



دانشگاه صنعتی امیرکبیر
(پلی تکنیک تهران)
دانشکده ریاضی و علوم کامپیوتر

پایان نامه کارشناسی ارشد
گرایش الگوریتم و نظریه محاسبه

ارزیابی قراردادهای هوشمند با محوریت
بهای سوخت بهینه و صحت در شبکه اتریوم

نگارش
محمدصادق مغاره

استاد راهنما
دکتر علی محدث خراسانی

خرداد ۱۴۰۳

بِسْمِ اللَّهِ الرَّحْمَنِ الرَّحِيمِ

صفحه فرم ارزیابی و تصویب پایان نامه - فرم تأیید اعضاء کمیته

دفاع

در این صفحه فرم دفاع یا تأیید و تصویب پایان نامه موسوم به فرم کمیته دفاع- موجود در پرونده آموزشی- را قرار دهید.

نکات مهم:

- نگارش پایان نامه/رساله باید به **زبان فارسی** و بر اساس آخرین نسخه دستورالعمل و راهنمای تدوین پایان نامه های دانشگاه صنعتی امیرکبیر باشد.(دستورالعمل و راهنمای حاضر)
- رنگ جلد پایان نامه/رساله چاپی کارشناسی، کارشناسی ارشد و دکترا باید به ترتیب مشکی، طوسی و سفید رنگ باشد.
- چاپ و صحافی پایان نامه/رساله بصورت **پشت و رو(دورو)** بلامانع است و انجام آن توصیه می شود.



دانشگاه صنعتی امیرکبیر
(پلی تکنیک تهران)

به نام خدا

تعهدنامه اصالت اثر

تاریخ: خرداد ۱۴۰۳

اینجانب **محمدصادق مغاره** متعهد می‌شوم که مطالب مندرج در این پایان‌نامه حاصل کار پژوهشی اینجانب تحت نظارت و راهنمایی اساتید دانشگاه صنعتی امیرکبیر بوده و به دستاوردهای دیگران که در این پژوهش از آنها استفاده شده است مطابق مقررات و روال متعارف ارجاع و در فهرست منابع و مآخذ ذکر گردیده است. این پایان‌نامه قبلاً برای احراز هیچ مدرک هم‌سطح یا بالاتر ارائه نگردیده است.

در صورت اثبات تخلف در هر زمان، مدرک تحصیلی صادر شده توسط دانشگاه از درجه اعتبار ساقط بوده و دانشگاه حق پیگیری قانونی خواهد داشت.

کلیه نتایج و حقوق حاصل از این پایان‌نامه متعلق به دانشگاه صنعتی امیرکبیر می‌باشد. هرگونه استفاده از نتایج علمی و عملی، واگذاری اطلاعات به دیگران یا چاپ و تکثیر، نسخه‌برداری، ترجمه و اقتباس از این پایان‌نامه بدون موافقت کتبی دانشگاه صنعتی امیرکبیر ممنوع است. نقل مطالب با ذکر مآخذ بلامانع است.

محمدصادق مغاره

امضا

تقدیم به:

به پدر بزرگوارم، که درس تلاش و زندگی و اخلاق را از او آموختم،

به مادرم، منظر صبر و مهربانی که مهربانی پایان از اوست،

به همسرم، که سایه مهربانش سایه ساز زندگی می باشد

و به خانواده مهربانم که در تمامی قدم های زندگی همواره سایه مهرشان مشوق تلاش من بوده است.

باعشق، حمایت و تشویق های بی پایان که از طرف شاعر نیران در طی سال های تحصیل دریافت کرده ام، این پایان نامه

را به شما تقدیم می کنم. تلاش ها و پشتیبانی های شما، نه تنها در اتمام این تحصیلات، بلکه در هر گام از مسیر تحصیلی اینجانب، برایم

بسیار ارزشمند و انگیزه بخش بوده است.

امید دارم که این پایان نامه نه تنها به ترقی علمی و شخصی خودم کمک کرده باشد بلکه بتواند در توسعه علم و معرفت به بشریت

نیز تاثیر کوچکی بگذارد.

سپاس‌گزاری

بر خود لازم می‌دانم از جناب آقای دکتر محدث که مرا در تهیه‌ی این پایان‌نامه یاری نمودند و همچنین به خاطر شخصیت بزرگ و تاثیرگذارشان در همه‌ی زمینه‌ها، که برای من باعث الهام و دلگرمی است و راهنمایی‌ها و همکاری ارزشمندشان تشکر و قدردانی نمایم.

همچنین، از مشاوران عزیز (سرکار خانم دکتر کتایون نوذری و جناب آقای دکتر روح‌الله خداکرمیان) که با راهنمایی‌ها و نظرات سازنده‌شان، به من کمک فراوانی در انجام این تحقیق دادند، سپاس‌گزاری می‌کنم.

محمدصادق مناره

خرداد ۱۴۰۳

چکیده

در دنیای زنجیره بلوکی و با توسعه روزافزون فضای آن، مفهوم توکن‌ها و قراردادهای هوشمند رشد چشمگیری داشته است. توجه به این نکته حائز اهمیت بسیار است که قراردادهای هوشمند پس از استقرار غیرقابل تغییر هستند، این مسئله سبب می‌گردد بهینه‌سازی بهای سوخت و صحت آن‌ها در این قراردادها به چالش کشیده شود. هدف اصلی این تحقیق، تحلیل اساسی‌ترین چالش‌های مسئله بهینه‌سازی بهای سوخت در قراردادهای هوشمند است. این تحقیق با بررسی معیارهای متعدد، به مسئله بهینه‌سازی بهای سوخت در قراردادهای هوشمند می‌پردازد. همچنین، در کنار موضوع بهینه‌سازی بهای سوخت، مسئله صحت قراردادها بعد از بهینه‌سازی نیز بطور مطرح گشته است چرا که در برخی مواقع این دو موضوع میتواند یک دوگانگی ایجاد کند و نکته ای که تضمین ایجاد میکند که با وجود بهینه سازی بهای سوخت سبب می‌گردد کارکرد قرارداد هوشمند تحت تاثیر قرار نگیرد بررسی صحت می‌باشد. با توجه به اهمیت وسیع این فضا و تأثیر مستقیم آن بر سرمایه‌گذاران و افراد، عدم اطمینان و ناپایداری در این سیستم‌ها می‌تواند آسیب‌های جبران ناپذیری ایجاد کند. بنابراین، صحت و پایداری قراردادهای هوشمند در فضای زنجیره‌ی بلوکی بسیار حیاتی است. در این تحقیق، با استفاده از تحلیل برنامه و عملکرد اصلی قراردادهای هوشمند مورد بررسی قرار گرفته اصولی‌ترین وظایف و ویژگی‌های آنها بررسی شده است تا امنیت و صحت آن‌ها، همراه با بهای سوخت بهینه تضمین شود.

واژه‌های کلیدی:

بهینه‌سازی، بهای سوخت، قراردادهای هوشمند، تأیید صحت، غیرقابل تغییر، آزمون‌های نرم‌افزاری، تحلیل برنامه

فهرست مطالب

صفحه

عنوان

۱	مقدمه	۱
۱-۱	مقدمه	۲
۱-۱-۱	بهای سوخت در شبکه‌های زنجیره بلوکی	۳
۱-۱-۲	ضرورت بهینه‌سازی بهای سوخت در شبکه‌های زنجیره بلوکی	۴
۱-۱-۳	صحت قرارداد هوشمند	۵
۲-۱	تعریف مسئله	۸
۳-۱	چالش مسئله	۹
۴-۱	انگیزه و ضرورت	۱۱
۵-۱	نوآوری	۱۲
۶-۱	ساختار پایان‌نامه	۱۳
۲	بررسی زیرساخت‌ها و ادبیات	۱۵
۱-۲	مقدمه	۱۶
۲-۲	توکن غیرقابل تعویض	۱۶
۱-۲-۲	توکن غیرقابل تعویض از منظر فنی	۱۶
۲-۲-۲	توکن غیرقابل تعویض از منظر بازار	۱۷
۳-۲	مولفه‌های فنی توکن غیرقابل تعویض	۱۸
۱-۳-۲	زنجیره بلوکی	۱۸
۲-۳-۲	قرارداد هوشمند	۱۹
۳-۳-۲	آدرس و تراکنش	۲۱
۴-۳-۲	کدگذاری داده	۲۲
۴-۲	انواع توکن‌ها در شبکه با محوریت استانداردهای مختلف	۲۳

۲۴	استاندارد شماره ۲۰	۱-۴-۲
۲۵	استاندارد شماره ۷۲۱	۲-۴-۲
۲۵	استاندارد شماره ۱۱۵۵	۳-۴-۲
۲۶	معرفی طرح کلی قرارداد هوشمند	۵-۲
۲۶	دید بالا به پایین	۱-۵-۲
۲۶	دید پایین به بالا	۲-۵-۲
۲۷	انواع ارزشها در شبکه های مختلف با محوریت هر شبکه	۶-۲
۲۷	ارز بومی و یا توکن قابل تعویض	۱-۶-۲
۲۸	توکن	۲-۶-۲
۲۸	ویژگی توکن های غیرقابل تعویض	۷-۲
۲۹	ماشین مجازی اتریوم	۸-۲
۳۱	زبان سالیدیتی	۱-۸-۲
۳۲	بازرسی قراردادهای هوشمند ^۱	۲-۸-۲
۳۳	تراکنش و اطلاعات ذخیره شده در زنجیره	۹-۲
۳۳	شماره بلاک و هش بلاک	۱-۹-۲
۳۳	هش تراکنش	۲-۹-۲
۳۳	آدرس مبدا و مقصد	۳-۹-۲
۳۳	مقدار	۴-۹-۲
۳۴	هش قرارداد هوشمند	۵-۹-۲
۳۴	بهای سوخت	۶-۹-۲
۳۴	مانده حساب	۷-۹-۲
۳۵	گره ها در زنجیره بلوکی	۱۰-۲
۳۵	جمع بندی	۱۱-۲

^۱Audit

۳۵	۱۱-۱ مسئله بهای سوخت
۳۵	۱۱-۲ مسئله صحت در قراردادهای هوشمند
۳۶	۳ پیشینه پژوهش
۳۷	۱-۳ مقدمه
۳۷	۲-۳ موضوع بهای سوخت در قراردادهای هوشمند
	۱-۲-۳ پژوهش صورت گرفته در خصوص بهای سوخت بالای توکن‌های غیرقابل
۳۷	تعویض
۳۸	۲-۲-۳ بهینه سازی بهای سوخت براساس ساختار کنترلی حلقه
۳۸	۳-۳ ارزیابی صحت قراردادهای هوشمند
۳۸	۱-۳-۳ چالش‌های امنیتی، راه حل‌ها
۴۰	۲-۳-۳ پژوهش‌ها در خصوص تحلیل‌های ایستا در توکن‌های غیرقابل تعویض
۴۰	۳-۳-۳ پژوهش صورت گرفته جهت بهینه‌سازی پروتکل‌های صحت سنجی
۴۱	۴-۳ پژوهش‌های جامع در حوزه تحلیل و آنالیز
۴۱	۱-۴-۳ پشتیبانی تست و امنیت در استقرار
۴۱	۵-۳ تحلیل استاتیک و دینامیک
۴۲	۱-۵-۳ روش استاتیک: تحلیل سورس برنامه بدون اجرا
۴۳	۲-۵-۳ روش دینامیک: آزمون و ارزیابی عملکرد
۴۴	۴ روش تحقیق و حل مسئله
۴۵	۱-۴ مقدمه روش‌ها
۴۵	۲-۴ تحلیل استاتیک
۴۵	۱-۲-۴ ۱. مدل‌سازی قرارداد هوشمند به عنوان یک گراف
۴۵	۲-۲-۴ ۲. تعریف محدودیت‌های ILP
۴۶	۳-۲-۴ ۳. تابع هدف

۴۷	۴-۲-۴ مزایای تحلیل استاتیک
۴۷	۳-۴ تحلیل دینامیک
۴۷	۴-۳-۱. آزمون بهای سوخت
۴۷	۴-۳-۲. جمع‌آوری داده‌های مصرف بهای سوخت
۴۷	۴-۳-۳. نحوه ارزیابی
۵۰	۴-۴ صورت مسئله اول بهینه سازی بهای سوخت
۵۰	۴-۴-۱ ویژگی‌های قرارداد هوشمند استاندارد
۵۱	۴-۴-۲ پارامترهای تاثیرگذار در بهای سوخت
۷۸	۴-۴-۳ ارائه دو نمونه تست واحد
۸۱	۵-۴ جمع بندی
۸۲	۵ نتایج پیاده سازی و تجزیه و تحلیل نتایج
۸۳	۵-۱ مقدمه
۸۴	۵-۲ نتایج حاصل از تحلیل استاتیک و دینامیک و دوگانه
۸۴	۵-۲-۱ نتایج تحلیل استاتیک
۸۵	۵-۲-۲ نتایج صورت مسئله اول به تفکیک هرکدام از پارامترها
۹۰	۵-۲-۳ کد در نظر گرفته شده
۱۰۱	۵-۳ خلاصه یافته‌های کلیدی و پیشنهادات
۱۰۱	۵-۳-۱ ارائه خلاصه‌ای از نتایج مهم
۱۰۲	۵-۴ جمع بندی
۱۰۳	۵-۵ پیشنهادات
۱۰۴	مراجع
۱۰۷	پیوست
۱۰۷	۱- پیش تاریخچه بلاک چین

۱-۱-	پیشینه مفهوم بلاکچین	۱۰۸
۲-۱-	پیدایش بیتکوین	۱۰۸
۳-۱-	تکامل زنجیره بلوکی	۱۰۸
۴-۱-	استفاده‌های گسترده	۱۰۸
۵-۱-	تکامل تکنولوژی	۱۰۹
۶-۱-	اعتراضات و چالش‌ها	۱۰۹
۲-	مفاهیم و تعاریف تکمیلی	۱۰۹
۳-	نرخ تبادل	۱۱۱
۴-	ساخت امضا و هش تراکنش	۱۱۲
۵-	ارسال تراکنش	۱۱۲
۶-	نمونه قرارداد هوشمند استفاده ضرب به جای توان	۱۱۳
۷-	نمونه قرارداد هوشمند استفاده از حلقه	۱۱۴
۸-	نمونه قرارداد هوشمند استفاده از حلقه در حالات بهینه و غیر بهینه	۱۱۴
۹-	نمونه قرارداد هوشمند کد مرده	۱۱۶
۱۰-	نمونه قرارداد هوشمند مقایسه رشته	۱۱۷
۱۱-	نمونه کد استفاده کد داخلی اسمبلی	۱۲۰
۱۲-	نمونه کد استفاده از بایت ۳۲ بجای رشته	۱۲۲
۱۳-	نمونه کد استفاده از رویداد	۱۲۳
۱۴-	نمونه کد استفاده از Assert	۱۲۳
۱۵-	نمونه کد استفاده از MemoryUsage انواع حافظه	۱۲۴
۱۲۷	واژه‌نامه‌ی فارسی به انگلیسی	
۱۲۹	واژه‌نامه‌ی انگلیسی به فارسی	

فهرست تصاویر

صفحه

شکل

۸	۱-۱	فرآیند تولید قرارداد هوشمند بهینه شده
۱۹	۱-۲	ساختار زنجیره بلوکی
۲۰	۲-۲	انواع تراکنش‌ها در زنجیره بلوکی
۲۰	۳-۲	انواع تراکنش‌ها در زنجیره بلوکی
۲۳	۴-۲	تفاوت توکن‌های مختلف
۲۷	۵-۲	جریان کار در توکن غیرقابل تعویض
۳۱	۶-۲	فرآیند کامپایل و استقرار قرارداد هوشمند در اتریوم
۴۶	۱-۴	ارائه یک گراف از بهای سوخت به ازای هر بلاک کد و جریان کنترل بلوک کد
۵۴	۲-۴	حافظه‌ها در قراردادهای هوشمند
۸۴	۱-۵	نمودار مقایسه مصرف گاز قبل و بعد از بهینه‌سازی
۱۰۷	۲	پیش تاریخچه رمزارزها

فهرست جداول

صفحه

جدول

۱-۱	مقایسه بین مراحل توسعه و استقرار قراردادهای هوشمند	۱۰
۱-۲	مقایسه استانداردهای توکن در اتریوم	۲۶
۱-۳	مشکلات امنیتی و راه حل ها	۳۹
۱-۴	قسمتی از جدول مربوط به آپ کدها	۵۲
۲-۴	جدول مقایسه بهای سوخت در دو حالت استفاده از رشته و استفاده بایت ۳۲	
	بجای رشته	۶۱
۱-۵	مقایسه مصرف گاز قبل و بعد از بهینه سازی	۸۴
۲-۵	جدول مقایسه بهای در دو حالت استفاده حلقه بهینه و غیر بهینه (براساس واحد بهای سوخت برحسب وی)	۸۵
۳-۵	جدول مقایسه بهای سوخت در دو حالت وجود کد مرده و عدم وجود آن (بر اساس زمان و برحسب میلی ثانیه)	۸۶
۴-۵	جدول مقایسه مصرف بهای سوخت با استفاده از هش و استفاده از رشته (برحسب واحد بهای سوخت)	۸۷
۵-۵	جدول مقایسه گس در دو حالت استفاده از بسته بندی ساختار و بدون استفاده از بسته بندی ساختار (برحسب اساس واحد بهای سوخت و برحسب وی)	۸۷
۶-۵	جدول مقایسه بهای سوخت در دو حالت استفاده از رویداد و بدون استفاده از رویداد	۸۸
۷-۵	جدول مقایسه گس در دو حالت استفاده از رویداد و بدون استفاده از رویداد	۸۹
۸-۵	جدول مقایسه گس از دید سطح دسترسی	۹۰
۹-۵	مصرف گس در مراحل مختلف	۹۲
۱۰-۵	مصرف گس در محل ذخیره سازی داده	۹۵

فهرست نمادها

فصل اول

مقدمه

۱-۱ مقدمه

مبحث قراردادهای هوشمند^۱، یکی از حوزه‌های زنجیره بلوکی^۲ است که هم از نظر علمی و هم از نظر کاربرد عملی توجه بسیار زیادی را به سمت خود جلب کرده است. به دلیل وجود شاخه‌های متنوع در این حوزه، تعریف‌های گوناگونی برای آن بیان گشته است. به عنوان یک تعریف کلی، میتوان گفت که زنجیره بلوکی رویکرد نوآورانه‌ای را ارائه می‌دهد که امکان ایجاد اعتماد در یک محیط باز را بدون نیاز به یک مرجع متمرکز فراهم می‌کند.^[۱]

تراکنش‌ها در شبکه‌های زنجیره بلوکی به دو دسته اصلی تقسیم می‌شوند: تراکنش‌های مستقیم و تراکنش‌های مبتنی بر قرارداد هوشمند. تراکنش‌های مستقیم، مانند انتقال بیت کوین از یک کیف پول به کیف پول دیگر که بدون هیچ شرط یا واسطه‌ای انجام می‌شود. این نوع تراکنش‌ها ساده، سریع و کم‌هزینه هستند. در مقابل، تراکنش‌های مبتنی بر قرارداد هوشمند، مانند تراکنش‌های توکن‌ها که پیچیده‌تر هستند. این تراکنش‌ها مشروط به برآورده شدن شرایط خاصی هستند که در قرارداد هوشمند تعریف شده‌اند. برای مثال، در مورد توکن غیرقابل تعویض، انتقال مالکیت پس از پرداخت مبلغ مشخص و تأیید توسط قرارداد هوشمند انجام می‌شود. هر دو نوع تراکنش نقش مهمی در فضای زنجیره بلوکی ایفا می‌کنند، اما کاربردها و پیچیدگی‌های متفاوتی دارند.

مسائل وابسته به قراردادهای هوشمند در زنجیره بلوکی به دسته‌های متفاوتی تقسیم می‌شوند. از جمله این مسائل میتوان به صرفه‌جویی در بهای سوخت^۳، صحت^۴، شفافیت^۵، کارآمدی^۶، دقت^۷، قابل اعتمادی^۸، سوابق غیرقابل تغییر^۹، نوآوری^{۱۰}، کاهش تقلبات^{۱۱}، و غیره اشاره کرد. هر یک از

¹ Smart Contracts² Blockchain³ Gas Savings⁴ Correctness⁵ Transparency⁶ Efficiency⁷ Accuracy⁸ Trustworthiness⁹ Immutable Records¹⁰ Innovation¹¹ Reduced Fraud

مسائل نیز میتوانند کاربردهای متفاوتی داشته باشند.

۱-۱-۱ بهای سوخت در شبکه‌های زنجیره بلوکی

بهای سوخت^{۱۲} در زنجیره بلوکی، به ویژه در اتریوم، مفهومی کلیدی است: بهای سوخت، هزینه‌ای است که کاربران برای اجرای تراکنش‌ها یا قراردادهای هوشمند در شبکه پرداخت می‌کنند. این هزینه به عنوان پاداش به ماینرها یا اعتبارسنج‌هایی داده می‌شود که تراکنش‌ها را پردازش و تأیید می‌کنند.

ویژگی‌های اصلی بهای سوخت:

- متغیر: بسته به تراکم شبکه تغییر می‌کند.
 - محاسبه: بر اساس پیچیدگی عملیات و منابع مورد نیاز محاسبه می‌شود.
 - واحد: معمولاً با ارز بومی شبکه (مثل اتر در اتریوم) پرداخت می‌شود.
 - کارکرد: از اسپم و سوءاستفاده از منابع شبکه جلوگیری می‌کند.
- بهای سوخت نقش مهمی در حفظ امنیت و کارایی شبکه‌های زنجیره بلوکی دارد.

یادداشت ۱-۱-۱. به طور کلی، تراکنش‌های مستقیم بهای سوخت کمتری نسبت به تراکنش‌های قرارداد هوشمند دارند. این تفاوت به دلیل پیچیدگی و نیاز به اجرای کد در قرارداد هوشمند است که باعث مصرف گس بیشتری می‌شود. انتخاب نوع تراکنش بستگی به نیازمندی‌های خاص برنامه یا کاربر دارد.

از رویکردهایی به جهت کاهش یا به اصطلاح بهینه‌سازی بهای سوخت استفاده می‌گردد

¹²Gas Fee

۱-۲- ضرورت بهینه‌سازی بهای سوخت در شبکه‌های زنجیره بلوکی

بهینه‌سازی بهای سوخت در شبکه‌های زنجیره بلوکی از اهمیت به‌سزایی برخوردار است. دلایل اصلی این ضرورت به شرح زیر تبیین می‌گردد:

۱. **کاهش هزینه‌های تراکنش:** بهینه‌سازی بهای سوخت منجر به کاهش قابل توجه هزینه‌های تراکنش می‌شود. این امر نه تنها موجب مقرون به صرفه‌تر شدن استفاده از شبکه برای کاربران می‌گردد، بلکه زمینه‌ساز گسترش پذیرش و استفاده از فناوری زنجیره‌بلوکی در سطح وسیع‌تری خواهد بود.

۲. **افزایش کارایی شبکه:** با بهینه‌سازی مصرف سوخت، تراکنش‌ها با کارایی بیشتری پردازش می‌شوند. این امر به معنای استفاده بهینه از منابع محاسباتی شبکه است که در نتیجه، ظرفیت شبکه برای پردازش تعداد بیشتری از تراکنش‌ها در واحد زمان افزایش می‌یابد.

۳. **ارتقاء تجربه کاربری:** بهینه‌سازی بهای سوخت موجب کاهش زمان انتظار برای تأیید تراکنش‌ها می‌شود. این بهبود در سرعت و کارایی، منجر به افزایش رضایتمندی کاربران و در نتیجه، تشویق استفاده گسترده‌تر از شبکه زنجیره بلوکی می‌گردد.

۴. **افزایش مقیاس‌پذیری:** با بهینه‌سازی بهای سوخت، امکان پردازش تعداد بیشتری از تراکنش‌ها در هر بلوک فراهم می‌شود. این امر به طور مستقیم بر قابلیت مقیاس‌پذیری شبکه تأثیر می‌گذارد و توانایی آن را برای پاسخگویی به تقاضای فزاینده در آینده افزایش می‌دهد.

۵. **کاهش تراکم شبکه:** بهینه‌سازی بهای سوخت نقش مهمی در جلوگیری از ازدحام و کندی شبکه، به ویژه در زمان‌های اوج تراکنش، ایفا می‌کند. این امر منجر به عملکرد پایدارتر و قابل اعتمادتر شبکه می‌شود.

۶. **تشویق توسعه برنامه‌های کارآمدتر:** با بهینه‌سازی بهای سوخت، توسعه‌دهندگان به نوشتن کدهای بهینه‌تر و کارآمدتر تشویق می‌شوند. این امر نه تنها به بهبود عملکرد

برنامه‌های غیرمتمرکز می‌انجامد، بلکه به ارتقاء کلی فضای زنجیره‌بلوکی نیز کمک می‌کند.

۷. **تقویت امنیت شبکه:** بهینه‌سازی بهای سوخت به عنوان یک مکانیسم دفاعی در برابر حملات مبتنی بر اسپم و سوءاستفاده از منابع شبکه عمل می‌کند. این امر به حفظ یکپارچگی و امنیت کلی شبکه زنجیره‌بلوکی کمک شایانی می‌نماید.

در مجموع، بهینه‌سازی بهای سوخت نه تنها برای بهبود عملکرد فنی شبکه‌های زنجیره‌بلوکی ضروری است، بلکه نقش حیاتی در تضمین پایداری، مقیاس‌پذیری و پذیرش گسترده این فناوری در آینده ایفا می‌کند.

۳-۱-۱ صحت قرارداد هوشمند

صحت قرارداد هوشمند به معنای تطابق کامل عملکرد قرارداد با مشخصات و اهداف از پیش تعیین شده است. این مفهوم شامل ابعاد متعددی است که عبارتند از:

۱. **درستی منطقی:** تضمین اینکه قرارداد دقیقاً مطابق با الگوریتم‌های تعریف شده و منطق کسب و کار مورد نظر عمل می‌کند.

۲. **امنیت:** اطمینان از مقاومت قرارداد در برابر انواع حملات سایبری و سوءاستفاده‌های احتمالی، با تأکید بر حفظ یکپارچگی داده‌ها و محرمانگی اطلاعات حساس.

۳. **کارایی:** بهینه‌سازی استفاده از منابع شبکه، شامل حافظه و قدرت پردازش، به منظور اجرای مؤثر و کارآمد عملیات قرارداد.

۴. **قابلیت اطمینان:** تضمین عملکرد صحیح و پایدار قرارداد در تمامی شرایط پیش‌بینی شده و حتی در مواجهه با ورودی‌های غیرمنتظره یا شرایط استرس شبکه.

اهمیت بررسی صحت پس از بهینه‌سازی بهای سوخت

بررسی مجدد صحت قرارداد هوشمند پس از اعمال بهینه‌سازی‌های مرتبط با بهای سوخت، از اهمیت حیاتی برخوردار است. دلایل این امر به شرح زیر تبیین می‌گردد:

۱. **حفظ یکپارچگی عملکردی:** بهینه‌سازی بهای سوخت نباید منجر به تغییر در منطق اصلی و عملکرد کلیدی قرارداد شود. بررسی صحت پس از بهینه‌سازی، تضمین می‌کند که الزامات کسب و کار و تعهدات قراردادی همچنان به طور کامل رعایت می‌شوند.

۲. **پیشگیری و شناسایی آسیب‌پذیری‌های نوظهور:** فرآیند بهینه‌سازی ممکن است به طور ناخواسته منجر به ایجاد نقاط ضعف امنیتی یا باگ‌های جدید در کد قرارداد شود. بررسی مجدد صحت، امکان شناسایی، ارزیابی و رفع این آسیب‌پذیری‌های بالقوه را فراهم می‌آورد.

۳. **ارزیابی اثربخشی بهینه‌سازی:** این فرآیند امکان سنجش دقیق تأثیر بهینه‌سازی‌ها بر کاهش مصرف گاز را فراهم می‌کند. همچنین، تأثیر این تغییرات بر سایر جنبه‌های عملکردی قرارداد، مانند زمان اجرا و استفاده از منابع، مورد ارزیابی قرار می‌گیرد.

۴. **تقویت اعتماد ذینفعان:** بررسی صحت پس از بهینه‌سازی، اطمینان خاطر لازم را برای کاربران، سرمایه‌گذاران و سایر ذینفعان فراهم می‌آورد. این امر به حفظ و تقویت اعتبار پروژه و اکوسیستم مربوطه کمک شایانی می‌کند.

۵. **انطباق با استانداردها و چارچوب‌های قانونی:** اطمینان حاصل می‌شود که قرارداد پس از بهینه‌سازی همچنان با استانداردهای صنعتی، پروتکل‌های امنیتی و الزامات قانونی مرتبط مطابقت دارد.

۶. **تضمین پایداری بهینه‌سازی:** این فرآیند تضمین می‌کند که بهبودهای ایجاد شده در مصرف گاز، تأثیر منفی بر سایر جنبه‌های حیاتی قرارداد، مانند قابلیت خوانایی کد یا قابلیت نگهداری آن، نداشته است.

۷. آمادگی برای توسعه‌های آتی: بررسی صحت اطمینان می‌دهد که ساختار قرارداد پس از بهینه‌سازی همچنان انعطاف‌پذیری لازم برای اعمال به‌روزرسانی‌ها و توسعه‌های آینده را حفظ کرده است.

یادداشت ۱-۱-۲. در نتیجه، بررسی صحت قرارداد هوشمند پس از بهینه‌سازی بهای سوخت، یک گام حیاتی در فرآیند توسعه و نگهداری قراردادهای هوشمند محسوب می‌شود. این فرآیند تعادل ضروری بین بهبود کارایی و حفظ عملکرد صحیح را تضمین می‌کند، که برای موفقیت و پایداری بلندمدت پروژه‌های مبتنی بر زنجیره بلوکی امری حیاتی است.

بررسی صحت^{۱۳} قرارداد هوشمند یکی از کاربردهای امنیت و قابل اعتمادی و کاهش تقلبات می‌باشد. همچنین کاهش بهای سوخت^{۱۴} از جمله کاربردهای صرفه‌جویی در هزینه می‌باشد. [۲] در حوزه کاربردی صحت و بررسی عملکرد قرارداد هوشمند سعی می‌گردد تا با ارائه روش‌هایی همانند تایید رسمی، منطق هور^{۱۵} و یا دوگان آن که منطق نادرستی^{۱۶} می‌باشد، به بررسی صحت و درستی کد قرارداد هوشمند و آنچنان که مطابق خواسته بوده است پرداخته شود. دلیل استفاده از این روش‌ها این موضوع است که در خیلی از ابزارهای موجود به دلیل وجود فرایند تجزیه و ترکیب‌های متعدد سبب می‌گردد تا کد دچار تغییر گردد. اما به دلیل تاثیر این تغییرات در عملکرد و نتیجتاً تغییرات ناخواسته در تراکنش‌های مالی در این حوزه نیاز به ضمانت است که هیچ موردی دستخوش تغییر ناخواسته و یا منفی نمی‌گردد. [۳] ابزارهای مختلف جهت تجزیه، تحلیل و ترکیب زبان برنامه نویسی مربوط به قراردادهای هوشمند وجود دارد. این ابزارها با هدف بهینه سازی^{۱۷} بوجود آمده اند. بهینه سازها، ابزاری مجهز به یک سری قوانین^{۱۸} هستند که سعی دارند تا قراردادهای هوشمندی که به زبان‌های مختلف برای مثال زبان سالیدیتی نوشته شده است را مورد تجزیه و تحلیل قرار داده تا بسته به هدف و علت بهینه سازی مدنظر انجام گردد. با توجه به هدف

¹³Correctness

¹⁴Gas Optimization

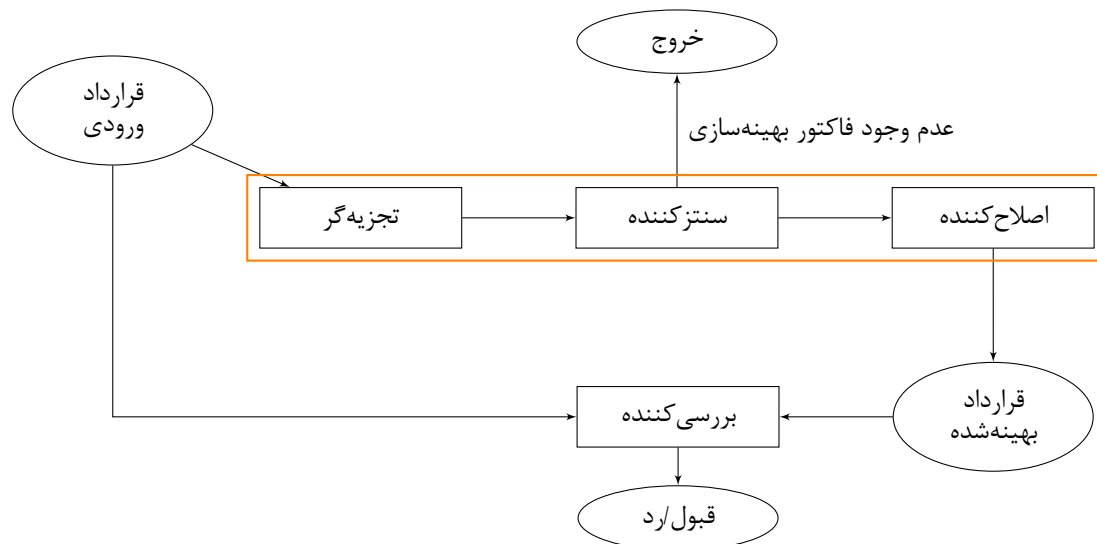
¹⁵Hoare Logic

¹⁶Incorrectness Logic

¹⁷Optimization

¹⁸Rule

و علت میتوان ابزارهای بهینه‌ساز قراردادهای هوشمند را در گروه‌هایی مجزا قرارداد. اما آنچه که مسلم است، این است که تمام ابزارهای بهینه‌ساز از قالب کلی پیروی میکنند. در شکل ۱-۱، جزییات معماری و ترتیب اجرای هر یک از واحدها نمایش داده شده است.



شکل ۱-۱: فرآیند تولید قرارداد هوشمند بهینه شده

۲-۱ تعریف مسئله

مراحل توسعه و استقرار قراردادهای هوشمند به دو بخش کلی تقسیم می‌شود. یک مرحله قبل از استقرار و تا زمانی است که توسعه به اتمام برسد و مرحله دوم پس از استقرار می‌باشد. تمامی تحلیل‌کننده‌ها و بهینه‌سازها در مرحله اول کاربرد دارند. چرا که مسئله‌ای که توسعه برنامه قرارداد هوشمند را از دیگر برنامه‌ها متمایز میکند؛ مسئله تغییرناپذیری^{۱۹} آن‌ها می‌باشد. این تفاوت بزرگ سبب می‌شود که اهمیت موضوعاتی همچون ضمانت درستی (صحت) و بهینه بودن بهای سوخت و دیگر مواردی که در بخش قبل اشاره شد، حائز اهمیت فراتر قرار گیرد.

درواقع یک قرارداد هوشمند که از منظر بهینه‌سازی بهای سوخت و صحت مورد بررسی قرار نگرفته، در صورت استقرار بر روی شبکه زنجیره بلوکی، می‌تواند منجر به ضرر و زیان مالی بسیاری

¹⁹Immutability

شود و این مسئله تنها به این جا ختم نمی‌گردد؛ چرا که هیچ گاه دیگر نمی‌توان آن قرارداد را مجدداً خطایابی و رفع خطا نمود و مجدداً به شبکه برگرداند. این مسئله سبب می‌شود که به موضوعاتی مانند صحت و بهینه بودن قراردادهای هوشمند توجه بسیاری شود. [۲] بنابر توضیحات ارائه شده در رابطه با اهمیت بهینه بودن و صحت قرارداد هوشمند، می‌توان گفت که هدف این مطالعه، پرداختن به مسئله بررسی قراردادهای هوشمند از منظر بهینه بودن بهای سوخت و صحت آن‌ها می‌باشد. درواقع هدف ارائه پارامترهای موثر در کاهش بهای سوخت در وهله اول و بررسی فاکتورهای متاثر در صحت پس از بهینه‌سازی بهای سوخت قراردادهای هوشمند در وهله دوم می‌باشد.

ارائه پارامترها و فاکتورهای متاثر در بحث کاهش بهای سوخت و صحت قرارداد هوشمند تنها هدف این مطالعه نیست. بلکه پیدایش ارتباطات و دوگانگی در این فاکتورها گاهی می‌تواند روی همدیگر تاثیر مثبت و یا منفی داشته باشد؛ که تا کنون کمتر به این مسئله پرداخته شده است و این مسئله سبب هزینه گزاف در شبکه زنجیره بلوکی شده است. [۴] برای رفع این مسئله، می‌توان از ابزارهای بهینه ساز کمک گرفت و با یک مقایسه میان پارامترها و فاکتورهای مختلف تلاش نمود تا ارتباط این موضوعات با یکدیگر را یافت.

۳-۱ چالش مسئله

اولین چالشی که می‌توان در مسئله کاهش بهای سوخت و صحت قرارداد هوشمند بیان کرد، این موضوع است، که متناسب با هدف باید مسئله دوگانگی بین صحت و بهای سوخت را مشخص نمود. با توجه به این نکته که در برخی موارد هدف صرفاً کاهش بهای سوخت می‌باشد، در چنین شرایطی باید تمامی فاکتورهای مدنظر را به کار گرفت تا با بیشترین میزان به بهبود بهای سوخت رسید. در چنین حالتی لزوماً بهترین امنیت اتفاق نخواهد افتاد، دلیل این امر این موضوع است که فاکتورهای مرتبط با میزان بهای سوخت در شرایط خاص که در فصول بعد به آن‌ها خواهیم پرداخت منجر به تغییر عملکرد خواهد شد و این موضوع مشکل صحت و امنیت را به مخاطره خواهد انداخت. [۵] پس مهم‌ترین قدم این می‌باشد که هدف اولیه به درستی مشخص گردد. اما با توجه به عملکرد قراردادهای هوشمند قطعاً صحت همواره بعنوان اصلی‌ترین مسئله مدنظر خواهد بود. در مقابل بهای

جدول ۱-۱: مقایسه بین مراحل توسعه و استقرار قراردادهای هوشمند

مرحله توسعه	مرحله استقرار
• تعریف مسئله و الزامات. • طراحی معماری قرارداد هوشمند. • نوشتن کد قرارداد هوشمند. • آزمایش قرارداد هوشمند برای عملکرد و آسیب‌پذیری‌های امنیتی. • بهینه‌سازی کد برای بهینه‌سازی مصرف گاز و کارایی هزینه. • بررسی و مرور کد برای رعایت استانداردها و بهترین روش‌ها. • تهیه اسناد قرارداد هوشمند برای مراجعه و نگهداری آینده.	• انتخاب پلتفرم زنجیره بلوکی مناسب برای استقرار. • استقرار قرارداد هوشمند در شبکه زنجیره بلوکی انتخابی. • تأیید استقرار و اطمینان از صحت آن. • تعامل با قرارداد هوشمند استقرار یافته از طریق تراکنش‌ها. • نظارت بر عملکرد و رفتار قرارداد هوشمند در محیط زنده. • اجرای به‌روزرسانی‌ها یا پیچ‌های لازم در صورت لزوم. • رعایت الزامات قانونی و مقرراتی.

سوخت نیز مسئله مهمی است، چرا که در صورتی که به نحوی قراردادهای هوشمند در نظر گرفته شوند که به کاهش بهای سوخت بی‌اهمیتی شود، این مسئله میتواند بوجود آورنده خللی برای از بین رفتن توکن و نتیجتاً به وجود آمدن هزینه گزاف گردد. در عمل چنین مسئله‌ای سبب بوجود آمدن خطای اتمام گاز صورت می‌گیرد. بطور مثال؛ از جمله مواردی که می‌تواند منجر به کاهش بهای سوخت در قراردادهای هوشمند شود، استفاده از حالت کد اسمبلی است، اما تعدادی از کنترل‌های امنیتی در حالت اسمبلی غیرفعال می‌گردند و اسمبلی، کد را مستعد خطا و ضعف امنیت و صحت می‌کند.

از جمله چالش‌های دیگر در این مسئله، می‌توان به استفاده از قابلیت دیده شدن توابع و یا سطوح دسترسی توابع می‌باشد. برای مثال اگر توابعی با حالت عمومی وجود داشته باشد آرگومان‌های ورودی در حافظه کپی خواهد شد و این اتفاق منجر به پرداخت بهای سوخت خواهد گشت اما در صورتی که به حالت بیرون^{۲۰} در این حالت مقادیر پارامترها و آرگومان‌ها به حافظه کپی نخواهد شد و این قضیه منجر به کاهش و بهبود بهای سوخت شود، اما باید بررسی نمود که آیا این قضیه خلل امنیتی خواهد داشت یا خیر؟

۴-۱ انگیزه و ضرورت

همانطور که از نام بهای سوخت و صحت در قراردادهای هوشمند پیداست، این مسئله تلاش میکند تا به بهبود میزان بهای سوخت و افزایش صحت قراردادهای هوشمند کمک داشته باشد. برای داشتن یک قرارداد هوشمند خوب باید عواملی را مدنظر قرار داد.

- امنیت: قرارداد دارای کمتری آسیب پذیری (در حالت ایده‌آل صفر) در مقابل حملات داشته باشد.

- هزینه: قرارداد دارای بهینه‌ترین گاز مصرفی را باشد؛ که شامل دو نوع است.

□ هزینه استقرار قرارداد

²⁰External

□ هزینه اجرا یا فراخوانی هر تابع از قرارداد

با در نظر گرفتن مسائل بالا با وجود یک منبع طبقه بندی شده از فاکتورهای تاثیرگذار در امنیت و بهای سوخت، میتوان به شناخت بهتر قراردادهای هوشمند رسید و از طریق آن و متناسب با اولویت و هدف مسئله بهترین قرارداد هوشمند ممکن را داشت. در حال حاضر این پارامترها قبل از استقرار با پرداخت هزینه توسط بازرسی های امنیتی صورت میپذیرد اما در بسیاری از موارد میتوان از ابزارهای موجود کمک گرفت تا از این هزینه اضافه صرف نظر نمود.

۵-۱ نوآوری

با توجه به چالش های مطرح شده در رابطه با امنیت و بهای سوخت قراردادهای هوشمند و مثال های اذعان شده، لازم است تا به یک مطالعه با هدف ساماندهی دانش در این پایان نامه صورت بگیرد و با بررسی مثال های متعدد به فاکتورهای تحت تاثیر در بهای سوخت و فاکتورهای تحت تاثیر در امنیت قرارداد هوشمند رسید. همچنین مهم تر از آن بررسی این مورد که دوگانگی های موجود را به چه شکل و به چه نحو می توان تعدیل نمود. راه حل های ارائه شده در این بخش استفاده از کدی در زبان پایتون بوده است، تا با بررسی ۷۲ قرارداد هوشمند و با مدنظر گرفتن پارامترهای متاثر در بهای سوخت به بررسی بهینه سازی آن ها پرداخته شود.

یادداشت ۱-۵-۱. در این مسئله، به بررسی های استاتیک و دینامیک خواهیم پرداخت. در تحلیل استاتیک، طولانی ترین مسیر و مصرف گاز برنامه را قبل و بعد از بهینه سازی، بدون نیاز به اجرای برنامه، مقایسه می کنیم. در تحلیل دینامیک، برنامه را با ورودی های مختلف اجرا کرده و نتایج بهینه سازی را ارزیابی می کنیم تا تأثیر بهینه سازی را در شرایط واقعی بررسی کنیم.

۱-۶ ساختار پایان نامه

در فصل دوم تمامی مفاهیم زیرساختی و ادبیات‌های مربوطه مورد بررسی قرار گرفته و به مرور در فصول بعدی با معرفی یک به یک آن‌ها، آن موارد نمادگذاری خواهند شد. همچنین در فصل سوم به پیشینه پژوهش پرداخته شده است.

در فصل چهارم به روش‌ها می‌پردازیم که فاکتورهای تاثیرگذار در بهای سوخت گاز را معرفی کنیم و همچنین تحلیل‌های ایستا و دینامیک و متدولوژی‌ها جهت بررسی صحت را بررسی می‌کنیم. فصل پنجم تحلیل نتایج و فصل ششم جمع‌بندی و پیشنهادات خواهیم پرداخت.

• **فصل دوم:** این فصل شامل بررسی زیرساخت‌ها و ادبیات‌های مطرح شده در حوزه تحلیل و بهینه‌سازی برنامه‌ها می‌باشد. تمامی مفاهیم پایه‌ای و نظری که در ادامه‌ی پژوهش مورد نیاز هستند، تشریح شده و به مرور در فصول بعدی با معرفی یک به یک آن‌ها، آن موارد نمادگذاری خواهند شد. این فصل بستری فراهم می‌کند تا خوانندگان با پیش‌زمینه‌های لازم برای درک بهتر مطالب آشنا شوند.

• **فصل سوم:** این فصل شامل مروری بر پیشینه پژوهش در حوزه بهای سوخت و صحت قرارداد هوشمند می‌باشد. در این فصل، به بیان جزئیات روش‌های پیشین پرداخته شده و شکاف‌های موجود در تحقیقات قبلی شناسایی می‌شوند. همچنین، این فصل به بررسی مطالعات انجام‌شده و نتایج حاصل از آن‌ها پرداخته و چارچوب نظری لازم برای ارائه روش‌های جدید در فصل‌های بعدی را فراهم می‌کند.

• **فصل چهارم:** این فصل به تشریح اطلاعات مرتبط با مجموعه داده‌ها، روش تحقیق و حل مسئله پرداخته شده است. در این فصل، فاکتورهای تاثیرگذار در بهای سوخت گاز شناسایی و معرفی می‌شوند. همچنین، تحلیل‌های ایستا و دینامیک و متدولوژی‌های بررسی صحت معرفی و بررسی می‌شوند. روش‌های تحلیل استاتیک و دینامیک، از جمله اجرای نمادین و برنامه‌ریزی خطی صحیح، به طور مفصل توضیح داده خواهند شد.

- **فصل پنجم:** این فصل به تجزیه و تحلیل و نتایج حاصل از اعمال روش‌های ارائه شده اختصاص دارد. نتایج به‌دست‌آمده از تحلیل‌های ایستا و داینامیک مقایسه شده و کارآیی بهینه‌سازی‌ها ارزیابی می‌شود. نمودارها، جداول و بحث‌های مفصل در این فصل به منظور ارائه دیدگاه‌های روشن‌تر از نتایج انجام‌شده استفاده خواهند شد.
- **فصل ششم:** این فصل شامل جمع‌بندی، نتیجه‌گیری و پیشنهادات می‌باشد. در این فصل، نتایج کلی پژوهش خلاصه شده و تأثیرات عملی و نظری آن‌ها مورد بحث قرار می‌گیرد. همچنین، پیشنهاداتی برای پژوهش‌های آینده و بهبود روش‌های فعلی ارائه خواهد شد.

فصل دوم

بررسی زیرساخت‌ها و ادبیات

۱-۲ مقدمه

در ابتدای نوشتار فصل دوم، به بررسی برخی تعاریف و مفاهیم مقدماتی و ادبیات مورد استفاده و همچنین زیرساخت‌های مورد استفاده در این پایان نامه پرداخته می‌شود تا با استفاده از آن بتوان مطالب را به صورت واضح بیان نمود. به همین منظور در این فصل به معرفی مفاهیمی از قبیل انواع توکن‌ها، دارایی‌های دیجیتال، قراردادهای هوشمند، مفاهیم مربوط به شبکه‌ها و انواع شبکه‌ها، مفاهیم زنجیره بلوکی و نرخ پرداخته می‌شود. توجه به این نکته جالب و حائز اهمیت است که زنجیره بلوکی حوزه‌ای نوظهور است اما اساس و پایه‌ی آن که منجر به پدید آمدن آن شده است بسیار قدیمی و ریشه دار می‌باشد. در قسمت ضمیمه می‌توان به نحوه شکل گیری حوزه زنجیره بلوکی نگاهی انداخت. در بخش پیش تاریخچه توکن‌ها بصورت مفصل تر پرداخته شده است.

۲-۲ توکن غیرقابل تعویض

محوریت اولیه این پژوهش، براساس قرارداد هوشمند توکن‌های غیرقابل تعویض بوده است، به همین منظور در خصوص این نوع خاص از توکن‌ها بررسی به عمل آمده است.

۱-۲-۲ توکن غیرقابل تعویض از منظر فنی

توکن غیرقابل تعویض^۱ یک نوع خاص از توکن و رمزارز^۲ یا دارایی‌های دیجیتال^۳ می‌باشد، که اولین بار توسط قرارداد هوشمند^۴ در شبکه اتریوم^۵ رمزنگاری شد. همچنین ایده‌ی ابتدایی توکن غیرقابل تعویض در پروپوزال بهبود اتریوم^۶ شماره ۷۲۱ مطرح گردید و در پروپوزال بهبود شماره ۱۱۵۵ توسعه یافت.

^۱ NFT(Non-Fungible Token)

^۲ Cryptocurrency

^۳ Digital Assets

^۴ Smart Contracts

^۵ Ethereum Network

^۶ EIP(Ethereum Improvement Proposals)

اناف‌تی در لغت به مفاهیم توکن غیر قابل تعوض، توکن غیر قابل معاوضه، توکن غیر مثلی، توکن یکتا و توکن بی همتا نیز تعبیر می‌گردد.

ارزهای دیجیتال کلاسیک همچون بیت‌کوین بصورت دو به دو معادل و غیرقابل تشخیص می‌باشد. خصیصه غیر قابل تعویض بودن این نوع ارز دیجیتال را از سایر ارزهای دیجیتال متمایز مینماید. این خصیصه سبب گشته تا این نوع ارز دیجیتال برای استفاده در سکوهایی که، کلیدهای دسترسی، بلیط‌های قرعه‌کشی، صندلی‌های شماره‌دار برای اماکنی همچون کنسرت و یا مسابقات ورزشی را ارائه می‌دهند بی بدیل نماید. این نوع توکن را می‌توان با ویژگی‌های مجازی/دیجیتال به‌عنوان شناسه‌های منحصر به فردشان پیوند داد. تمامی خواص گفته شده در خصوص توکن غیرقابل تعویض باید در چارچوب مشخصی باشد. به بیان ساده یک آفریننده توکن غیرقابل تعویض با استفاده از ویژگی یکتا بودن در فرم تصویر، ویدیو، اثر هنری، بلیط و یا هر شکل متمایزکننده‌ای امکان وجود، مالکیت و تمایز را ایجاد می‌کند. اما مدیریت این استانداردها در قالب یک شکل خاص که به استاندارد اتریوم^۷ تعبیر می‌شود، صورت می‌پذیرد. استاندارد مربوط به توکن غیرقابل تعویض در استاندارد شماره ۷۲۱^۸ منتشر شد. این استاندارد در ژانویه ۲۰۱۸ پیشنهاد و بصورت سرویس‌های تحت وب^۹ از طریق قراردادهای هوشمند توسعه و پیاده سازی شد.^[۶]

۲-۲-۲ توکن غیرقابل تعویض از منظر بازار

توکن‌های غیرقابل تعویض یک دارایی دیجیتال قابل معامله است، که در سال ۲۰۲۱ افزایش شگرفی در بازار ایجاد نمود. این دارایی دیجیتال در دوره‌ی پیشرفت شگرف خود درحالی که اطلاعات کمی درباره ساختار و سیر تکاملی بازار آن وجود داشت، در بازار جهانی ثبت رکورد کرده است. این ویژگی توکن غیرقابل تعویض سبب می‌شود تا علاقه پژوهشگران به تحلیل آماری و تحلیل بازار این ارز دیجیتال را افزایش دهد. اصلی‌ترین سوالاتی که ذهن تحلیل‌کنندگان بازار را درخصوص این نوع ارزهای دیجیتال درگیر می‌کرد این بوده است که چه ویژگی توکن غیرقابل تعویض سبب شد

^۷ERC(Ethereum Request for Comments)

^۸ERC-721

^۹API

تا توکن غیرقابل تعویض بر بازار هنر و بازارهای وابسته مسلط گردد؟ با بررسی بازارهای دیگر، این انقلاب به بازار هنر محدود نمی‌شود. در حالی که بهره‌برداری از توکن غیرقابل تعویض در بازی‌ها در زمینه معامله اشیاء درون بازی به افزایش چشم‌گیر است، صنایع دیگری همچون صنایعی که در تولید محتوای دیجیتال مانند موسیقی یا ویدیو دخیل هستند، با این فناوری درگیر گشته است. در چهار ماه اول سال ۲۰۲۱، حجم توکن غیرقابل تعویض بیش از ۲ میلیارد دلار بوده است، که ده برابر حجم کل معاملات این رمزارز در سال ۲۰۲۰ بوده است. [۷]

۳-۲ مولفه‌های فنی توکن غیرقابل تعویض

از منظر فنی دسترسی کامل به تاریخچه و تعامل آسان امکان می‌دهد تا توکن غیرقابل تعویض به عنوان یک راه‌حل موثر برای حفاظت از مالکیت معنوی^{۱۰} شود. به همین دلیل نیاز است تا مولفه‌های فنی را مورد بررسی قرار دهیم، تا با دید عمیق تری به نحوه‌ی حفاظت از مالکیت معنوی درک شود.

۱-۳-۲ زنجیره بلوکی

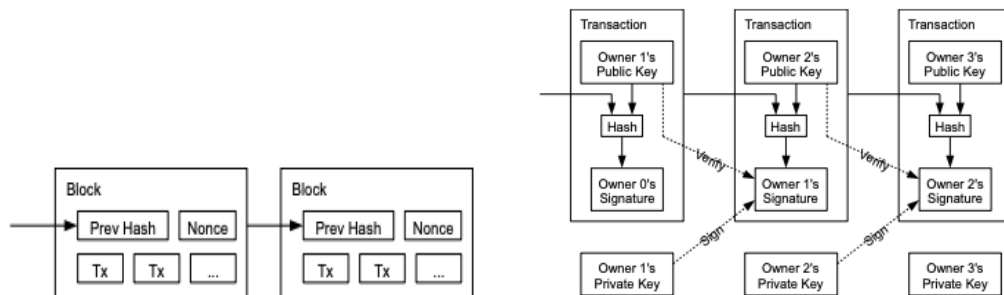
زنجیره بلوکی، ابتدا توسط شخصی به نام ناکاموتو پیشنهاد شد. [۸] که بیت‌کوین در این زنجیره از الگوریتم اثبات کار^{۱۱} [۴] برای رسیدگی به صحت در داده تراکنش‌ها در یک شبکه غیرمتمرکز استفاده می‌کند. زنجیره بلوکی به عنوان یک پایگاه داده توزیع شده است و تنها برای افزودن داده‌ها تعریف می‌شود که یک لیست از داده‌ها را حفظ می‌کند و با استفاده از پروتکل‌های رمزنگاری [۹] به هم متصل و محافظت می‌شوند. زنجیره بلوکی^{۱۲} راه‌حلی برای مشکل بیزانتین طولانی‌مدت [۱۰] فراهم می‌کند که با مشارکت یک شبکه بزرگ از شرکت‌کنندگان ناامن گردآوری شده است. یک‌بار که داده‌های مشترک در زنجیره بلوکی در اکثر گره‌های توزیع شده تأیید شود، غیرقابل تغییر می‌شود زیرا هر تغییری در داده‌های ذخیره‌شده باعث نامعتبر شدن تمام داده‌های بعدی می‌شود.

¹⁰ Intellectual Property Protection

¹¹ Proof of Work

¹² long-standing Byzantine problem

پراکندترین سکوی زنجیره بلوکی مورد استفاده، در طرح‌های مربوط به توکن غیرقابل تعویض، اتریوم [۱۱] است که محیط امنی را برای اجرای قراردادهای هوشمند استاندارد ۷۲۱ و دیگر استانداردها فراهم می‌کند. [۶]



(آ) ساختار امضا و صحت سنجی در زنجیره بلوکی (ب) ساختار زنجیره‌ی بلوکی از منظر تراکنش‌ها

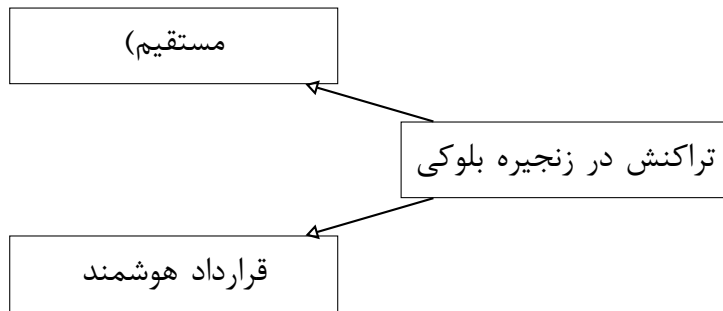
شکل ۱-۲: ساختار زنجیره بلوکی

[۸]

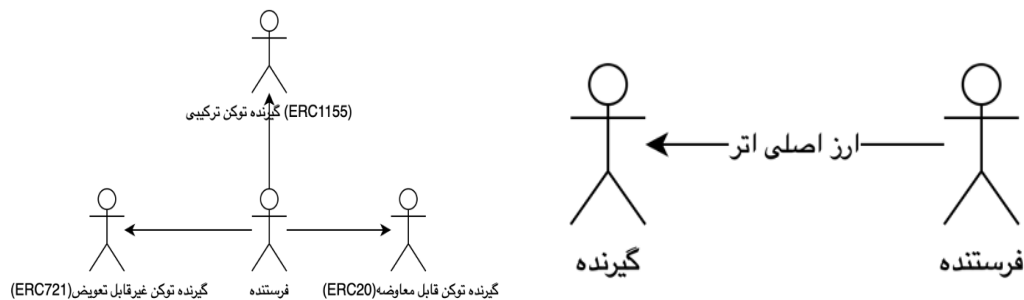
۲-۳-۲ قرارداد هوشمند

با در نظر گرفتن دو سناریو، می‌توان متوجه قرارداد هوشمند در زنجیره بلوکی شد. دو سناریو مختلف از تراکنش‌ها وجود دارد. در سناریو اول، تراکنش به صورت مستقیم و بدون شرط از یک کیف پول به کیف پول دیگر منتقل می‌شود، در این سناریو ارزهای بومی شبکه (مانند اتر در اتریوم) درگیر خواهد بود. در سناریو دوم، تراکنش شامل توکن‌های قابل تعویض و غیرقابل تعویض و ... است که با استفاده از قرارداد هوشمند و پس از رعایت قوانین مشخص انجام می‌شود. قرارداد هوشمند نقش مهمی در تضمین اجرای قوانین و شرایط تعیین شده برای تراکنش‌ها دارد. این قراردادها به صورت خودکار اجرا می‌شوند و به واسطه کدهای برنامه‌نویسی که در زنجیره بلوکی مستقر شده‌اند، اعتماد و امنیت بیشتری را فراهم می‌کنند. به این ترتیب، قراردادهای هوشمند می‌توانند بدون نیاز به واسطه‌های سنتی، تراکنش‌ها را به صورت مطمئن و شفاف انجام دهند.

زنجیره‌های بلوکی محبوب، از جمله اتریوم، اجازه اجرای برنامه‌های کاربردی به نام قراردادهای هوشمند را که در زنجیره بلوکی ذخیره می‌شوند، می‌دهند. قراردادهای هوشمند امکان انجام



شکل ۲-۲: انواع تراکنش‌ها در زنجیره بلوکی



(آ) سناریو تراکنش نظیر به نظیر (ب) سناریو تراکنش به واسطه قرارداد هوشمند

شکل ۲-۳: انواع تراکنش‌ها در زنجیره بلوکی

تراکنش‌های با پیچیدگی‌های دلخواه بین افراد غیرقابل اعتماد را به صورت امن بدون نیاز به واسطه (مانند مؤسسات مالی تجاری) با استفاده از رمزارزها فراهم می‌کنند. برنامه‌ها شامل امور مالی غیرمتمرکز و حراجی‌ها هستند. یک قرارداد هوشمند وضعیت دائمی را که در زنجیره بلوکی ذخیره شده است، مدیریت می‌کند. این قرارداد از مجموعه‌ای از توابع تشکیل شده است که وضعیت را تغییر می‌دهد. توابع می‌توانند توسط کاربران به صورت مستقیم یا به طور غیرمستقیم توسط سایر قراردادهای هوشمند از طریق تراکنش‌ها فراخوانی شوند. آنها امکان انجام عملیات با پیچیدگی‌های دلخواه با استفاده از رمزارزهای ذخیره شده در زنجیره را فراهم می‌کنند. [۱۲][۱۳]

۲-۳-۳ آدرس و تراکنش

آدرس و تراکنش در حوزه ارزهای دیجیتال مفاهیم بسیار مهمی هستند. آدرس، شناسه منحصر به فردی است که به کاربران امکان ارسال و دریافت دارایی‌ها را می‌دهد، مانند یک حساب بانکی. این آدرس از تعداد ثابتی از کاراکتر و عدد تشکیل شده است که از جفتی^{۱۳} کلید عمومی و کلید خصوصی تولید می‌شود. برای انتقال توکن غیرقابل تعویض یا دیگر ارزها، مالک باید مالکیت کلید خصوصی مربوطه را اثبات کند و دارایی‌ها را به آدرس(های) دیگری با امضای دیجیتال صحیح ارسال کند. این عملیات ساده معمولاً با استفاده از یک کیف پول رمزارزی انجام می‌شود و به عنوان ارسال یک تراکنش برای درگیر کردن قراردادهای هوشمند در استاندارد ۷۷۷^{۱۴} [۱۴] نمایش داده می‌شود.

آدرس صاحب حساب

به معنای "آدرس اکانت"^{۱۵} در مفهوم شبکه اتریوم است. این آدرس یک نوع حساب در شبکه اتریوم است که به صورت متعارف توسط افراد ایجاد می‌شود. این حساب‌ها دارای یک آدرس متشکل از ۴۰ حرف یا عدد هگزادسیمال (معمولاً با حروف کوچک) هستند. این آدرس به عنوان شناسه‌ای برای صاحب حساب در شبکه اتریوم عمل می‌کند و از آن برای ارسال و دریافت اتر و تعامل با قراردادهای هوشمند استفاده می‌شود.

این آدرس‌ها دارای کلیدهای خصوصی هستند که به صاحب حساب اجازه می‌دهد تراکنش‌هایی را انجام دهند و کنترل دارایی‌های خود را داشته باشند. امنیت حساب این آدرس‌ها بسیار مهم است و صاحب حساب باید محافظت کافی برای حفظ کلیدهای خصوصی خود داشته باشد تا جلوی دسترسی غیرمجاز به حساب او را بگیرد.

¹³Pair

¹⁴ERC-777

¹⁵Externally Owned Account (EOA)

آدرس قرارداد هوشمند

آدرس یک قرارداد هوشمند در شبکه نیز متشکل از ۴۰ حرف یا عدد هگزادسیمال است، مانند آدرس یک حساب که در پیش‌فرض با حروف کوچک نمایش داده می‌شود. این آدرس تابعی از کد باینری قرارداد هوشمند و مکان آن در شبکه است.

برای مثال، آدرس قرارداد هوشمند ممکن است به شکل می‌باشد :

0x742d35Cc6634C0532925a3b844Bc454e4438f44e

این آدرس‌ها بسیار مهم هستند و برای انجام تراکنش‌ها و تعامل با قراردادهای هوشمند در شبکه، نیاز به دانش دقیق در مورد آدرس مربوطه و عملکرد قرارداد است.

۲-۳-۴ کدگذاری داده

کدگذاری فرآیند تبدیل داده از یک شکل به شکل دیگر است. به طور معمول، بسیاری از فایل‌ها به شکل‌های کدگذاری شده تبدیل می‌شوند، یا به شکل‌های فشرده و کم حجم برای صرفه‌جویی در فضای دیسک یا به شکل‌های فشرده نشده برای کیفیت و وضوح بالا. در سیستم‌های زنجیره بلوکی معروف مانند بیت‌کوین و اتریوم، از مقادیر مبنای ۱۶ برای کدگذاری عناصر معاملاتی مانند نام‌های توابع، پارامترها و مقادیر بازگشتی استفاده می‌شود. این نشان می‌دهد که داده‌های خام توکن غیرقابل تعویض باید این قوانین را رعایت کنند. اگر کسی ادعا کند که مالک توکن غیرقابل تعویضی است، به عبارتی مالک مقدار داده مبنای ۱۶ کدگذاری ایجاد شده توسط خالق است. دیگران می‌توانند به طور آزاد داده‌های خام را کپی کنند، اما نمی‌توانند مالکیت اموال را ادعا کنند. بر اساس این، می‌توان گفت که فعالیت‌های مرتبط با توکن غیرقابل تعویض (مانند خرید/فروش/معامله/حراج) باید در چهار مرحله پردازش شوند، که مشابه روند پردازش اولیه قراردادهای هوشمند می‌باشد. [۶]

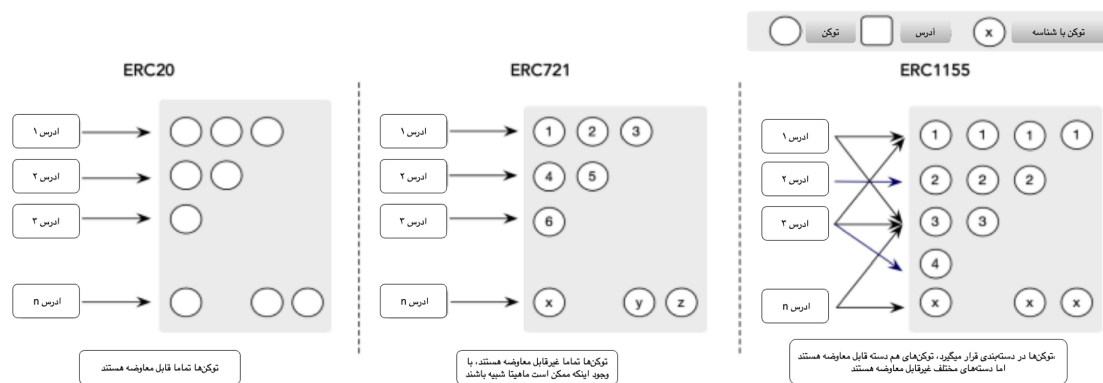
۴-۲ انواع توکن‌ها در شبکه با محوریت استانداردهای مختلف

توکن در زنجیره بلوکی یک واحد دیجیتالی از ارز است که به عنوان نماینده ارزشی خاص در شبکه عمل می‌کند. این توکن‌ها به عنوان دارایی‌های دیجیتال تعریف می‌شوند و می‌توانند نماینده ارزهای مختلف باشند.

مهمترین ویژگی اولیه توکن‌ها این است که آن‌ها قابل تبدیل و انتقال درون شبکه هستند. به عبارت دیگر، توکن‌ها می‌توانند بین کاربران منتقل شوند. این انتقال توکن‌ها باعث انجام تراکنش‌ها در شبکه می‌شود و از طریق آن مالکان توکن‌ها می‌توانند ارزش‌های خود را به دیگر اشخاص انتقال دهند.

شبکه‌های مختلف ممکن است از انواع مختلفی از توکن‌ها برای اهداف مختلف استفاده کنند. به عنوان مثال، در بلاکچین اتریوم، توکن‌ها معمولاً به عنوان ارزهای دیجیتالی (مانند اتر) و یا نماینده ارزهای دیگر (مانند ان اف تی‌ها) استفاده می‌شوند.

به طور کلی، توکن‌ها به عنوان واحدهای ارزشی و عامل اصلی انجام تراکنش‌ها و انتقال دارایی‌ها عمل می‌کنند و نقش مهمی ایفا می‌کنند. [۱۵] [۶]



شکل ۴-۲: تفاوت توکن‌های مختلف

استانداردهای توکن: استاندارد شماره ۲۰ یک استاندارد توکن در اتریوم است که توکن‌های قابل تعویض را معرفی کرد که در نوع و ارزش یکسان هستند و نقش مهمی در ارائه اولیه سکه ایفا کرد. از جمله توکن‌های قابل تعویض میتوان به توکن با ارزش متغیر همچون اتر (ارز بومی در اتریوم

و توکن قابل تعویض در دیگر شبکه‌ها) و یا توکن با ارزش ثابت همچون تتر اشاره داشت. استاندارد ۷۲۱ یک استاندارد توکن غیرقابل تعویض در اتریوم است که ایجاد توکن‌های منحصر به فرد که می‌توانند از یکدیگر متمایز شوند، امکان می‌دهد. این استاندارد امکان نمایش دارایی‌های فردی و غیرقابل تعویض را فراهم می‌کند. توکن‌های غیر قابل تعویض می‌توانند به هم‌دیگر از لحاظ ظاهری مشابه باشند، اما ارزش‌های کاملاً متفاوت داشته باشند. برای مثال؛ توکن‌هایی که در بازی‌ها به کار گرفته می‌شود مانند Axie Infinity که از جمله معروف ترین آن‌ها می‌باشد.

استاندارد ۱۱۵۵ یک استاندارد توکن است که نمایش توکن‌های قابل تعویض و غیرقابل تعویض را گسترش می‌دهد. این استاندارد رابطی برای نمایش هر تعداد توکن فراهم می‌کند و امکان ایجاد انواع مختلف توکن قابل پیکربندی در یک قرارداد تکی را فراهم می‌کند. بطور مثال؛ در بازی‌های خاصی نیاز به توکن‌هایی وجود دارد که واقعا از لحاظ ارزشی متغیر نمی‌باشد اما هرکدام از آنان به‌طور خاص دارای هویت و متمایز از دیگر توکن‌ها می‌باشد در چمنین حالتی از استاندارد ۱۱۵۵ کمک گرفته می‌شود. استاندارد ۷۲۱ و استاندارد ۱۱۵۵ به رویکردهای مختلفی در مورد استقرار توکن دارند. استاندارد ۷۲۱ یک گروه از توکن‌های غیرقابل تعویض را در یک قرارداد تکی استقرار می‌دهد، در حالی که استاندارد ۱۱۵۵ امکان نمایش مستقل انواع مختلف توکن در یک قرارداد را فراهم می‌کند. این استانداردهای توکن تأثیر قابل توجهی بر توسعه طرح‌های توکن غیرقابل تعویض دارند و پایه‌ای برای ایجاد، انتقال و مدیریت توکن‌های غیرقابل تعویض فراهم می‌کنند.

۲-۴-۱ استاندارد شماره ۲۰

استاندارد شماره ۲۰^{۱۶} یک استاندارد توکن اتریوم است که مجموعه‌ای از قوانین و عملکردها را برای ایجاد و مدیریت توکن‌های قابل معامله در زنجیره بلوکی اتریوم تعریف می‌کند. توکن‌های قابل معامله، توکن‌هایی هستند که با یکدیگر قابل تعویض هستند، به این معنا که یک توکن همیشه ارزش معادل یک توکن دیگر از همان نوع دارد. توکن‌های استاندارد اتریوم ۲۰ اساس بسیاری از

¹⁶ERC-20

رمزارزها، پیشنهادات سکه اولیه^{۱۷} و برنامه‌های کاربردی غیرمتمرکز ساخته شده بر پلتفرم اتریوم شده‌اند.

۲-۴-۲ استاندارد شماره ۷۲۱

استاندارد ۷۲۱ به عنوان استاندارد توکن غیرقابل تعویض در اتریوم شناخته می‌شود. این استاندارد به توسعه دهندگان امکان می‌دهد تا توکن‌های منحصر به فردی ایجاد کنند که از یکدیگر تمیز و قابل تمیز باشند. به عبارت دیگر، هر توکن ۷۲۱ دارای ویژگی‌ها و ویژگی‌های منحصر به فردی است و نمی‌تواند با دیگر توکن‌های همگانی جایگزین شود. این استاندارد معمولاً برای نمایش دارایی‌های مختلف مثل عناصر بازی، اثرات گرافیکی، یا اموال دیجیتالی استفاده می‌شود.

۳-۴-۲ استاندارد شماره ۱۱۵۵

توکن ۱۱۵۵ یک استاندارد توکن در اتریوم است که به توسعه دهندگان امکان می‌دهد تا توکن‌های قابل تعویض و غیرقابل تعویض را ایجاد کنند. این استاندارد از توکن ۲۰ و توکن ۷۲۱ اقدام کرده و امکان ایجاد توکن‌های چندگانه در یک قرارداد را فراهم می‌کند. به عبارت دیگر، با استاندارد ۱۱۵ می‌توان توکن‌هایی را تعریف کنید که به صورت همزمان می‌توانند توکن‌های قابل تعویض و توکن‌های غیرقابل تعویض باشند.

این توکن امکان ایجاد توکن‌های مختلف با ویژگی‌ها و قابلیت‌های متفاوت را در یک قرارداد فراهم می‌کند. این استاندارد برای مواردی که نیاز به تنوع در توکن‌ها دارند، مانند بازی‌های دیجیتالی، پلتفرم‌های تجاری، و دیگر موارد مشابه مناسب است. با این استاندارد توکن‌ها به صورت کاملاً قابل تنظیم و تنوع زیادی می‌توانند ایجاد شوند و به عنوان دارایی‌های دیجیتالی نمایش داده شوند.

¹⁷ICO: Initial Coin Offering

۱۱۵۵	۷۲۱	۲۰	ماهیت
ترکیب هر دو	غیر قابل معاوضه	قابل معاوضه	
یک قرارداد می‌تواند توکن‌های قابل معاوضه و غیرقابل معاوضه را ایجاد و مدیریت کند	یک قرارداد هوشمند فقط توکن غیرقابل معاوضه (ان اف تی) ایجاد می‌کند	هر قرارداد هوشمند یک توکن قابل معاوضه ایجاد و مدیریت می‌کند	محدودیت
ارزان	گران	ارزان	هزینه تراکنش
بالا	بالا	بالا	بهره‌وری
انجین	کریپتو کیتی	تتر	مثال

جدول ۱-۲: مقایسه استانداردهای توکن در اتریوم

۵-۲ معرفی طرح کلی قرارداد هوشمند

قرارداد هوشمند توکن غیرقابل تعویض از دو دیدگاه مختلف می‌تواند مورد تجزیه و تحلیل قرار گیرد: از پایین به بالا و از بالا به پایین.

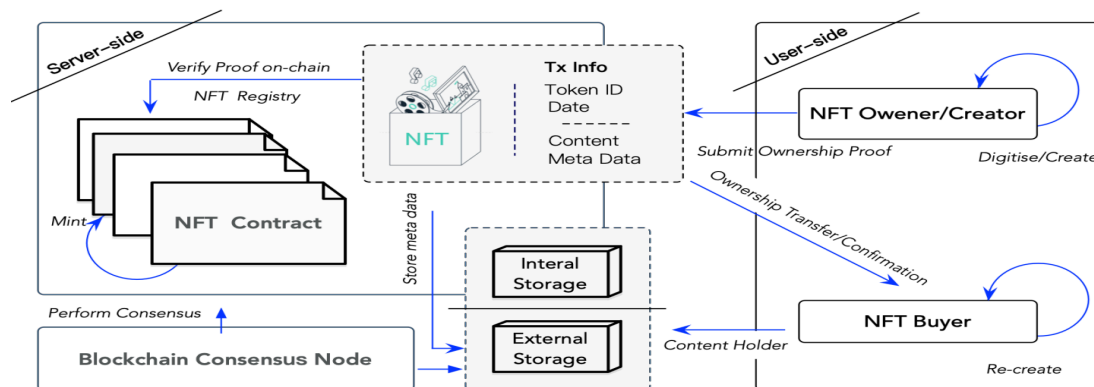
۱-۵-۲ دید بالا به پایین

در رویکرد از بالا به پایین، قرارداد توکن غیرقابل معاوضه از دو نقش تشکیل شده است: مالک توکن و خریدار توکن. پروتکل اطمینان از شفافیت و دسترسی به فعالیت‌های توکن غیرقابل تعویض را فراهم می‌کند، اجازه می‌دهد تا اطلاعات فنی توکن و مالکیت به صورت عمومی تأیید شود، و همچنین امکان ضرب، فروش و خرید توکن‌ها را فراهم می‌کند.

[۶]

۲-۵-۲ دید پایین به بالا

رویکرد پروتکل از پایین به بالا در توکن غیرقابل معاوضه شامل ایجاد توکن غیرقابل معاوضه‌ها توسط کاربران بر اساس یک الگو تعیین شده توسط خالق توکن غیرقابل معاوضه می‌شود. این طراحی امکان سفارشی‌سازی و خلاقیت کاربر در ایجاد توکن غیرقابل معاوضه‌های منحصر به فرد را فراهم



شکل ۲-۵: جریان کار در توکن غیرقابل تعویض

می‌کند.

در این پروتکل، خالق توکن از طریق یک قرارداد الگویی را آغاز می‌کند که قوانین و ویژگی‌های پایه‌ای برای توکن تعیین می‌کند.

سپس کاربران می‌توانند برای توکن پیشنهاد قیمت دهند و بر اساس مقادیر تصادفی، توکن غیرقابل معاوضه‌های منحصر به فرد خود را در محدوده‌های تعیین شده توسط الگو ایجاد کنند. پروتکل از پایین به بالا همچنین اطمینان از شفافیت و دسترسی را فراهم می‌کند، اجازه می‌دهد تا اطلاعات فنی توکن و مالکیت به صورت عمومی تأیید شود، و همچنین امکان ضرب توکن، فروش و خرید توکن را فراهم می‌کند. [۶]

۲-۶ انواع ارزها در شبکه‌های مختلف با محوریت هر شبکه

۲-۶-۱ ارز بومی و یا توکن قابل تعویض

اتریوم: اتر به عنوان سکه اصلی و واحد پول اصلی در شبکه اتریوم شناخته می‌شود. این سکه اصلی برای انجام معاملات، اجرای قراردادهای هوشمند و پرداخت هزینه‌های مختلف در شبکه استفاده می‌شود. لازم به ذکر است که این ارز در دیگر شبکه‌ها همچون اتر یا بایننس به عنوان یک توکن شناخته می‌شود.

بایننس: بایننس به عنوان سکه اصلی در شبکه بایننس شناخته می‌شود. این سکه اصلی برای معاملات، هزینه‌های تراکنش و انجام فعالیت‌های مختلف در اکوسیستم بایننس استفاده می‌شود.

ترون: ترون به عنوان سکه اصلی و واحد پول اصلی در شبکه ترون شناخته می‌شود. این سکه اصلی برای معاملات، اجرای قراردادهای هوشمند و ارسال تراکنش‌ها در ترون استفاده می‌شود.

۲-۶-۲ توکن

توکن‌ها در اتریوم: در اتریوم، توکن‌ها به عنوان واحدهای دیجیتالی دیگر از ارزهای رمزارزی و دارایی‌های دیجیتالی استفاده می‌شوند. این توکن‌ها ممکن است برای توکن‌های رمزارزی مختلف، پروژه‌های اختصاصی، امضای قراردادهای هوشمند و برنامه‌های دیگر در اتریوم ایجاد شوند. همچنین با الگوی مشابه برای مابقی شبکه‌ها نیز توکن تعریف می‌شود.

یادداشت ۲-۶-۱. لازم بذکر است که هرکدام از شبکه‌ها می‌تواند شامل هرکدام از انواع استاندارد توکن‌ها باشد و تمایزی بین استانداردهای مختلف در شبکه‌های مختلف وجود ندارد.

۲-۷ ویژگی توکن‌های غیرقابل تعویض

• ویژگی‌های مطلوب ان‌اف‌تی‌ها:

۱. **قابلیت تأیید^{۱۸}:** توکن‌های غیرقابل تعویض به تأیید عمومی اطلاعات توکن و مالکیت جواز می‌دهند، از این طریق شفافیت و اعتماد در سیستم ایجاد می‌کنند.
۲. **اجرای شفاف^{۱۹}:** فعالیت‌های توکن‌های غیرقابل تعویض، از جمله ایجاد، فروش و خرید، به صورت عمومی دسترسی‌پذیر است، این امکان را فراهم می‌کند که در اکوسیستم ان‌اف‌تی شفافیت و صحت ایجاد شود.

¹⁸Verifiability

¹⁹Transparent Execution

۳. **دسترسی مداوم**^{۲۰}: سیستم توکن غیرقابل تعویض به طور مداوم در دسترس است، تضمین می‌کند که تمام توکن‌ها و توکن‌های غیرقابل تعویض صادر شده در هر زمانی می‌توانند خریداری و فروخته شوند.
۴. **مقاومت در برابر اختلال و دستکاری**^{۲۱}: داده‌های همراه توکن‌های غیرقابل تعویض و تراکنش و معاملات به صورت پایدار ذخیره می‌شوند و پس از تأیید معاملات قابل دستکاری نیستند، این ویژگی راحتی و امکان تغییرناپذیری محیط شبکه‌های توکن‌های غیرقابل تعویض تضمین می‌کند.
۵. **قابلیت استفاده**^{۲۲}: هر توکن غیرقابل تعویض اطلاعات مالکیت به‌روزی دارد که برای کاربران قابل فهم و به‌وضوح ارائه می‌شود، این امکان را به کاربران می‌دهد که به راحتی با توکن‌های غیرقابل تعویض تعامل کنند.
۶. **فردیت**^{۲۳}: معاملات ان‌اف‌تی می‌توانند در یک تراکنش فردی و پایدار ACID تکمیل شوند، این ویژگی تمامیت و قابلیت اطمینان معاملات ان‌اف‌تی را تضمین می‌کند.
۷. **قابلیت معامله**^{۲۴}: توکن‌های غیرقابل تعویض و محصولات متناظر با آنها می‌توانند به آزادی معامله و تبادل شوند، این امکان را به بازار توکن‌های غیرقابل تعویض می‌دهد تا از نظر نقدینگی و انعطاف‌پذیری پیشرفت کند.

۸-۲ ماشین مجازی اتریوم

ماشین مجازی اتریوم یک بخش کلیدی در محیط شبکه اتریوم است که وظیفه اجرای قراردادهای هوشمند و اجرای تراکنش‌های زنجیره بلوکی را برعهده دارد. این بخش به عنوان محیط اجرایی برای اجرای کدهای برنامه‌نویسی توکن‌ها و قراردادهای هوشمند در زنجیره بلوکی اتریوم عمل می‌کند.

²⁰Availability

²¹Tamper-resistance

²²Usability

²³Atomicity

²⁴Tradability

این ماشین مجازی بر پایه معماری ماشین مجازی تورینگ کامل^{۲۵} طراحی شده است، که به برنامه‌نویسان این امکان را می‌دهد که قراردادهای هوشمند پیچیده‌تر و برنامه‌های توزیع‌شده را توسعه دهند. این به معنای این است که با استفاده از ماشین مجازی اتریوم، می‌توان برنامه‌هایی بسیار گسترده و متنوع را بر روی بلاکچین اجرا کرد.

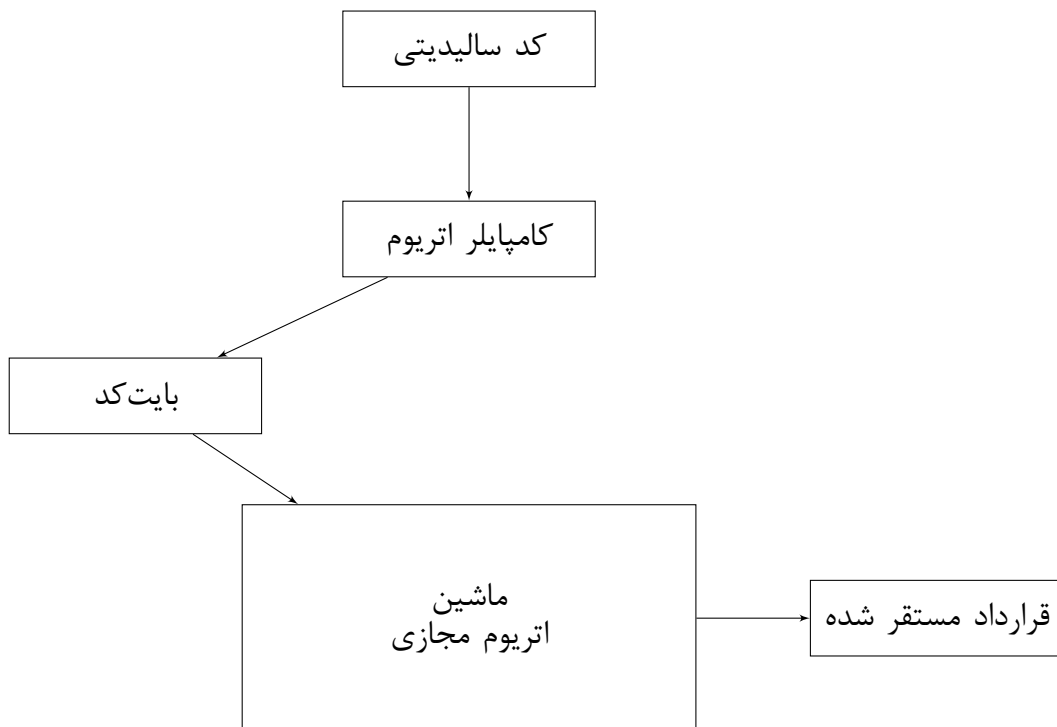
کارکرد این ماشین به این صورت است که تراکنش‌هایی که در شبکه اتریوم ارسال می‌شوند، به دستگاه‌های ماینر ارسال می‌شوند. این دستگاه‌ها تراکنش‌ها را بررسی و تایید می‌کنند و سپس توسط ماشین مجازی اتریوم اجرا می‌شوند. ماشین مجازی اتریوم به کمک یک زبان برنامه‌نویسی خاص به نام "سالیدیتی"^{۲۶} قطعه کدهای اجرا شونده و قراردادهای هوشمند را اجرا می‌کند.

بعد از اجرای موفقیت‌آمیز یک تراکنش یا قرارداد هوشمند توسط ماشین مجازی، تغییرات مرتبط با زنجیره بلوکی (مانند تغییرات در موجودی حساب‌ها) ثبت می‌شوند و بلاک جدیدی به زنجیره افزوده می‌شود.

به طور خلاصه، ماشین مجازی اتریوم مسئول اجرای قراردادهای هوشمند و تراکنش‌های زنجیره بلوکی در اتریوم است و بر پایه معماری ماشین مجازی تورینگ کامل عمل می‌کند که امکان اجرای برنامه‌های گسترده‌تر و پیچیده‌تر را فراهم می‌کند.

²⁵Turing-complete

²⁶Solidity



شکل ۲-۶: فرآیند کامپایل و استقرار قرارداد هوشمند در اتریوم

۲-۸-۱ زبان سالی‌دیتی

زبان سالی‌دیتی یک زبان برنامه‌نویسی مورد استفاده برای توسعه قراردادهای هوشمند در زنجیره بلوکی اتریوم است. این زبان به طور اساسی برای توسعه برنامه‌های غیرمتمرکز (توکن‌ها، بازی‌ها، برنامه‌های مالی، و غیره) در زنجیره بلوکی اتریوم مورد استفاده قرار می‌گیرد. در ادامه، توضیحات مختصری از زبان سالی‌دیتی را خواهیم داشت:

سالی‌دیتی یک زبان برنامه‌نویسی مناسب برای توسعه قراردادهای هوشمند در زنجیره بلوکی اتریوم است. این زبان به توسعه‌دهندگان اجازه می‌دهد تا قراردادهای هوشمندی بسازند که بر اساس اجرای کد در بلاکچین، تراکنش‌ها را اجرا می‌کنند و قابلیت انجام توافقات و تبادل ارزهای رمزی و دارایی‌های مختلف را دارا هستند.

زبان سالی‌دیتی شبیه به زبان‌های برنامه‌نویسی معمولی است و از مفاهیمی مانند متغیرها، توابع،

حلقه‌ها، و شرط‌ها پشتیبانی می‌کند. همچنین دارای ویژگی‌های خاصی برای تعریف قراردادهای هوشمند است که به توسعه‌دهندگان اجازه می‌دهد تا قوانین و قواعد خود را برای تبادل اطلاعات و دارایی‌ها تعریف کنند.

یکی از ویژگی‌های جذاب سالیدیتی، قابلیت تعریف توکن‌های قابل معامله است. این توکن‌ها می‌توانند نمایندگی رمزارز خاص، دارایی‌های دیجیتال، و حتی اجناس فیزیکی را داشته باشند و با استفاده از قراردادهای هوشمند توسط توسعه‌دهندگان بسیاری ساخته شوند.

قراردادهای هوشمند اتریوم با توجه به دینامیک و فضای بی‌نهایت ناشی از یک زبان تورینگ کاملی که دارند، امکان تحلیل‌های نرم افزاری متعدد را خواهند داشت. در دیگر زبان‌ها همچون سی یا جاوا، تحلیل می‌تواند به اجرای بهینه‌تر از منظر زمان کمک به‌سزایی داشته باشد. در سالیدیتی نیز این قضیه نمود هزینه مالی پیدا خواهد کرد.

۲-۸-۲ بازرسی قراردادهای هوشمند^{۲۷}

در زمینه قراردادهای هوشمند، بازرسی به معنای یک مرور دقیق و مستقل از کد قرارداد هوشمند، عملکرد، صحت و تطابق با بهترین شیوه‌ها و استانداردها است. هدف اصلی از یک مرور قرارداد هوشمند، شناسایی و حل مشکلات و آسیب‌پذیری‌های ممکن در کد قرارداد هوشمند قبل از اجرا در زنجیره بلوکی است.

مرورهای قراردادهای هوشمند به علت اینکه پس از اجرا در زنجیره بلوکی قابل تغییر نیستند (به معنایی که نمی‌توان آن‌ها را تغییر داد یا به‌روزرسانی کرد) بسیار حیاتی هستند. هر گونه آسیب‌پذیری یا مشکل در کد قرارداد می‌تواند منجر به از دست دادن مالی، نقض امنیت، یا عملکرد ناخواسته شود.

یادداشت ۲-۸-۱. ذکر این نکته حائز اهمیت است، که بطور معمول قراردادهای هوشمند قابلیت بروز شدن^{۲۸} و تغییر را ندارد، اما امروزه با استفاده از برخی روش‌ها از جمله استفاده از پروکسی

²⁷Audit

²⁸Upgradability

قابلیت ایجاد برخی تغییرات بصورت غیرمستقیم صورت پذیرفته است. اما بطور معمول این روش پیشنهاد نمی‌گردد که علت آن در تضاد بودن با بحث غیرمتمرکز بودن فضا زنجیره بلوکی می‌باشد.

۹-۲ تراکنش و اطلاعات ذخیره‌شده در زنجیره

در این بخش به مفاهیم مهم مرتبط با تراکنش‌ها و اطلاعاتی که در آن ذخیره می‌شود پرداخته خواهد شد.

۱-۹-۲ شماره بلاک و هش بلاک

شماره بلاک به عنوان یک شماره منحصر به فرد تمام تراکنش‌ها و رویدادها در زنجیره بلوکی مورد استفاده قرار می‌گیرد. هش بلاک نیز به عنوان یک امضای دیجیتال برای تأیید محتوای بلاک استفاده می‌شود.

۲-۹-۲ هش تراکنش

هش تراکنش یک مقدار دسته‌ای است که از اطلاعات تراکنش به وجود می‌آید. این هش معمولاً برای تأیید صحت تراکنش و جلوگیری از تغییر آن توسط اشخاص غیرمجاز استفاده می‌شود.

۳-۹-۲ آدرس مبدا و مقصد

آدرس مبدا تراکنش نشان‌دهنده آدرس کیف پول یا حسابی است که تراکنش از آن آغاز می‌شود. آدرس مقصد نیز نشان‌دهنده آدرسی است که تراکنش به آن ارسال می‌شود.

۴-۹-۲ مقدار

مقدار تراکنش نشان‌دهنده مقدار ارز یا دارایی‌ای است که انتقال داده می‌شود. این مقدار ممکن است برای انتقال ارزهای دیجیتال مثل بیت‌کوین یا ان‌اف‌تی‌ها مورد استفاده قرار گیرد.

۲-۹-۵ هش قرارداد هوشمند

قراردادهای هوشمند در زنجیره بلوکی دارای آدرس‌هایی هستند و می‌توانند تراکنش‌ها را انجام دهند. هش قرارداد هوشمند نشان‌دهنده آدرس مربوط به قرارداد ویژه است که در تراکنش‌های مرتبط با آن استفاده می‌شود.

۲-۹-۶ بهای سوخت

فی تراکنش مبلغی است که به عنوان کارمزد به ماینرهای شبکه پرداخت می‌شود تا تراکنش‌ها را تأیید کنند. وضعیت تراکنش نشان‌دهنده وضعیت فعلی تراکنش (مانند تأیید شده یا در حال انتظار و یا ناموفق) است.

۲-۹-۷ مانده حساب

مفهوم مانده حساب^{۲۹} در زنجیره بلوکی به معنای مقدار ارز دیجیتال یا توکن‌هایی است که یک آدرس کیف پول در شبکه دارد. هر آدرس در یک شبکه زنجیره بلوکی می‌تواند یک مقدار خاص از مانده حساب داشته باشد. مانده حساب نمایانگر مقدار ارز دیجیتالی یا توکن مربوط به آن آدرس در زنجیره بلوکی است.

به عبارت دیگر، مانده حساب نشان‌دهنده وضعیت مالی یک آدرس در یک شبکه زنجیره بلوکی است. اگر مانده حساب یک آدرس مثبت باشد، به معنای داشتن ارز یا توکن‌هایی در آن آدرس است. در صورتی که مانده حساب منفی باشد، به معنای بدهی در آن آدرس است که نشان‌دهنده انتقال توکن‌ها به آدرس‌های دیگر در یک تراکنش قبلی است. (مانده حساب منفی در حالت‌های خاصی اتفاق می‌افتد چرا که این قضیه عملاً به معنای بدهی می‌باشد) مفهوم مانده حساب در زنجیره بلوکی مهم است زیرا تمام تراکنش‌ها بر اساس مانده حساب‌ها انجام می‌شوند. برای انجام یک تراکنش، فرستنده باید مقدار کافی مانده حساب داشته باشد تا بتواند توکن‌های مورد نظر را به مقصد انتقال دهد. این مفهوم اساسی در طراحی و عملکرد شبکه‌های زنجیره بلوکی دخیل است.

²⁹Balance

۱۰-۲ گره‌ها در زنجیره بلوکی

گره‌ها در زنجیره بلوکی اشاره به اعضای شبکه‌ای دارند که در فرآیند تأیید و اجرای تراکنش‌ها و ایجاد بلاک‌ها در شبکه زنجیره بلوکی شرکت می‌کنند. این گره‌ها می‌توانند انواع مختلفی داشته باشند و وظایف مختلفی را انجام دهند. برخی از انواع گره‌ها عبارتند از:

۱۱-۲ جمع‌بندی

با توجه به بررسی مفاهیم و عناوین مورد استفاده در این نوشتار، می‌کوشیم تا مسئله اصلی مورد توجه در این تحقیق با دقت و به شیوه‌ای کامل و تئوری و عملی معرفی شود. این معرفی مسئله به خوانندگان این امکان را می‌دهد تا درک شفافی از ابعاد متعدد مسئله پیدا کنند و آمادگی لازم برای درک بهتر راه‌حل‌های ارائه شده در فصول بعدی را داشته باشند. در فصول آتی، به بررسی دقیق‌تر و بیشتر راه‌حل‌های مناسب و بهینه برای این مسئله پرداخته خواهد شد، به نحوی که خوانندگان با ارتباط و تداخل بین مسئله و راه‌حل‌ها آشنا شوند و این ارتباط به طور واضح بیان شود.

۱-۱۱-۲ مسئله بهای سوخت

در این نوشتار سعی شده است، با بررسی عمیقی پارامترهای درگیر در مسئله گس فی بهینه مورد بررسی به تفصیل قرار بگیرد و در فصول آتی اولین موضوع بحث شده در این خصوص بوده است.

۲-۱۱-۲ مسئله صحت در قراردادهای هوشمند

در کنار و گاهی در مقابل مسئله گاز مسئله صحت قراردادهای هوشمند از اهمیت بسزایی برخوردار است و در فصول آتی دومین مسئله بحث شده در این خصوص بوده است. با افزایش چشمگیر دزدی‌ها و تلف شدن اموال در قراردادهای هوشمند این موضوع یعنی بررسی صحت در قراردادهای هوشمند از مهمترین مسائل این حوزه خواهند بود.

فصل سوم

پیشینه پژوهش

۱-۳ مقدمه

در این قسمت به بررسی پیشینه پژوهش که در حیطه ی موضوعات مرتبط با این پایان نامه است خواهیم پرداخت. از آنجایی که ماهیت این نوشتار بر اساس کاربرد آن در دنیای واقعی است و مسائل مطرح شده در آن در سیستم های دنیای واقعی و به صورت عملیاتی در حال استفاده هستند، این فصل را به قسمت های مختلف بهای سوخت، صحت قرارداد هوشمند و تحلیل های جامع تقسیم کرده و درباره هر کدام جداگانه به بررسی خواهیم پرداخت. نکته حائز اهمیت این است که در پژوهش های صورت گرفته بخصوص در دوبرخ اول موارد و مسائل بصورت پیوسته و درهم تنیده مورد بحث و بررسی قرار گرفته است و ممکن است در منابع و مقالات پژوهشی در یک مقاله جامع هر کدام از موارد مورد بحث قرار گرفته شده باشد.

۲-۳ موضوع بهای سوخت در قراردادهای هوشمند

۱-۲-۳ پژوهش صورت گرفته در خصوص بهای سوخت بالای توکن های

غیرقابل تعویض

افزایش قیمت بهای سوخت به یکی از مشکلات اساسی برای بازارهای توکن های غیرقابل تعویض، به ویژه زمانی که توکن های غیرقابل تعویض به مقیاس بزرگی ایجاد می شوند و نیاز به آپلود داده های فنی به شبکه زنجیره بلوکی دارند، تبدیل شده است. هزینه استخراج یک توکن ممکن است بیش از ۶۰ دلار برسد و انجام یک معامله ساده توکن غیرقابل تعویض ممکن است بین ۶۰ دلار تا ۱۰۰ دلار هزینه به دنبال داشته باشد. این هزینه های گران قیمت که ناشی از عملیات پیچیده و ازدحام بالا هستند، به شدت منجر به محدود شدن استفاده از توکن های غیرقابل تعویض می شوند. [۶]

۳-۲-۲ بهینه سازی بهای سوخت براساس ساختار کنترلی حلقه

درخصوص موضوع بهینه سازی بهای سوخت براساس ساختار کنترلی حلقه یک رویکرد برای بهینه سازی بهای سوخت قراردادهای هوشمند ارائه شده است که منجر به صرفه جویی در میانگین مصرف بهای سوخت در هر معامله تقریباً ۲۳،۹۴۳ واحد بهای سوخت یا کاهش ۲۱٪ در هزینه های آن می شود. این رویکرد منجر به افزایش در هزینه های استقرار به دلیل افزودن توابع داخلی اضافی می شود، اما این قضیه تنها با میانگین ۱۶،۷۱۰ واحد بهای سوخت یا ۵٪ از کل هزینه اجرا (ضرب توکن) است. که این مقدار مقداری ناچیز می باشد. قراردادهای بهینه سازی شده تولید شود. معادل بودن قراردادهای تولید شده با قراردادهای اصلی با استفاده از اثبات های بهبود تایید می شود. [۱]

۳-۳ ارزیابی صحت قراردادهای هوشمند

۳-۳-۱ چالش های امنیتی، راه حل ها

در باره موضوع امنیت و حریم خصوصی مرتبط با توکن های غیرقابل تعویض، تناقضات امنیتی در سیستم های توکن غیرقابل تعویض شناسایی شده است، مانند جعل، دستکاری، تکذیب و افشای اطلاعات. گزینه ای از راه حل های دفاعی درباره این مسائل پیشنهاد شده، که از جمله تایید رسمی قراردادهای هوشمند و قراردادهای هوشمند حفظ حریم خصوصی اصلی ترین و مهم ترین راه حل های این حوزه بوده است. جدول مسائل امنیتی یک جمع بندی از بارزترین مشکلات و راه حل های را نشان داده است. [۶]

جدول ۳-۱: مشکلات امنیتی و راه حل ها

عنوان مشکل	مشکلات امنیتی	راه حل ها
جعل هویت	- یک مهاجم ممکن است ضعف های احراز هویت را مورد حمله قرار دهد. - یک مهاجم ممکن است کلید خصوصی کاربر را بدزدد.	- یک تایید رسمی در قرارداد هوشمند. - استفاده از کیف پول فیزیکی برای جلوگیری از دزدیده شدن کلید خصوصی.
عدم پایداری	داده های ذخیره شده خارج از زنجیره بلوکی ممکن است تغییر یابد.	ارسال همزمان داده اصلی و داده هش به خریدار توکن غیرقابل معاوضه در هنگام معامله.
تغییر آدرس مقصد	داده های هش ممکن است به آدرس مهاجم تغییر پیدا کند.	استفاده از یک قرارداد چندامضایی.
عدم محرمانگی	یک مهاجم می تواند به آسانی از هش و تراکنش به شناسایی خریدار یا فروشنده پیوند بزند.	استفاده از قراردادهای هوشمند حفظ حریم خصوصی به جای قراردادهای هوشمند عادی برای حفاظت از حریم خصوصی کاربر.
دسترسی	داده های توکن ممکن است در دسترس نباشند اگر دارایی خارج از زنجیره ذخیره شود.	استفاده از معماری زنجیره بلوکی ترکیبی با الگوریتم موافقت.
اختیارات دسترسی	یک قرارداد هوشمند ضعیف ممکن است باعث از دست رفتن احراز هویت شود.	تایید رسمی در قرارداد هوشمند.

۳-۳-۲ پژوهش‌ها درخصوص تحلیل‌های ایستا در توکن‌های غیرقابل

تعویض

با توجه به افزایش بکارگیری فناوری‌های زنجیره بلوکی و قراردادهای هوشمند، تایید رسمی^۱ و اعتبارسنجی قراردادهای هوشمند موضوع فعالیت تحقیقاتی فعال در سال‌های اخیر بوده است. در مطالعه‌ها، تجزیه و تحلیل جامع و دسته‌بندی روش‌های موجود برای مدل‌سازی رسمی، مشخصات و اعتبارسنجی قراردادهای هوشمند ارائه شده‌است. علاوه بر این، ویژگی‌های مشترک از حوزه‌های مختلف قراردادهای هوشمند شرح داده شده و آنها را با قابلیت‌های روش‌های اعتبارسنجی موجود مرتبط کرده است. یافته‌ها نشان می‌دهد که ترکیبی از مدل‌ها و مشخصات قرارداد با بررسی مدل به طور گسترده برای استدلال درباره صحت کارکردی قراردادهای هوشمند از تمام حوزه‌های مورد بررسی استفاده می‌کند. از سوی دیگر، نمایش‌های سطح برنامه اغلب از منظر ویژگی‌های امنیتی مورد تجزیه و تحلیل قرار می‌گیرند، که با استفاده از اجرای نمادین^۲، ابزارهای تأیید برنامه یا اثبات قضیه بررسی می‌شوند. مشکلات امنیتی همچنین می‌توانند در دنباله‌های اجرای قراردادهای هوشمند توسط تکنیک‌های اعتبارسنجی در زمان اجرا شناسایی شوند. دیدگاه کلی به تجزیه و تحلیل قراردادهای هوشمند که در مطالعات به کار رفته است، امکان دستیابی به محدودیت‌های موجود در اعتبارسنجی رسمی موثر قراردادهای هوشمند را می‌دهد و جهت‌های تحقیقات پیشنهاد می‌دهد. [۱۶]

۳-۳-۳ پژوهش صورت گرفته جهت بهینه‌سازی پروتکل‌های صحت سنجی

در قدم بعدی به جهت افزایش امنیت که در این حوزه نیاز به انجام دارد، یک تکنیک جبری برای مدلسازی و تایید ترکیب پروتکل‌های مالی غیرمتمرکز (تئوری مالی دیجیتال) پیشنهاد می‌شود که امکان تایید خصوصیات به صورت کارآمد صورت می‌پذیرد.

این تکنیک پیشنهادی شرایط خاصی را که باعث نقض خصوصیات صحت در ترکیب پروتکل‌ها

^۱ Formal Specification and Formal Verification

^۲ Symbolic Execution

و توکن‌ها می‌شوند، شناسایی می‌کند.

کارهای آینده در این حوزه شامل غنی‌سازی مدل‌ها برای در نظر گرفتن قابلیت‌های مرتبط با مکانیزم‌های حکومت در پروتکل‌ها، گسترش مجموعه خصوصیات برای پوشش آسیب‌پذیری‌های امنیتی و جنبه‌های رمزارز اقتصادی، و حل مشکل گسترش وضعیت از طریق تکنیک‌های تایید ترکیبی می‌باشد. [۱۷]

۳-۴ پژوهش‌های جامع در حوزه تحلیل و آنالیز

با توجه به این موضوع که حوزه زنجیره بلوکی و بطور خاص توکن‌های غیرقابل تعویض یک حوزه بسیار جدید می‌باشد، طبیعی است پژوهش‌ها به تازگی اقبال دارند تا از علوم مشابه مانند تحلیل و آنالیز برنامه‌ها در برنامه نویسی قراردادهای هوشمند به خدمت گرفته شوند. پس میتوان با رجوع به آنها ایده‌های جدیدی گرفت و از آنان برای بحث بهینه‌سازی بهای سوخت و همچنین آزمون‌ها و بهینه‌سازی قراردادهای توکن‌های غیرقابل تعویض کمک شایانی گرفت.

۳-۴-۱ پشتیبانی تست و امنیت در استقرار

در بحث استقرار نکات بسیار مهم و حائز اهمیت وجود دارد از جمله آن موارد، بحث تست و امنیت پیوسته پیش از استقرار، حین و بعد از استقرار می‌باشد. [۱۸] در مباحث مربوط به بازرسی و توسعه و استقرار این مسئله بسیار حائز اهمیت است که، در سیستم‌های در حال توسعه که قراردادهای هوشمند قراردادهای هوشمند از جمله آنها می‌باشد، همواره باید به موضوع تست و استقرار پیوسته توجه نمود. [۱۹]

۳-۵ تحلیل استاتیک و دینامیک

در حوزه بهینه‌سازی و صحت‌سنجی برنامه‌ها، دو روش اصلی وجود دارد: استاتیک و دینامیک. این دو روش برای بررسی و ارزیابی برنامه‌ها از جنبه‌های مختلف به کار می‌روند و هر کدام مزایا و معایب

خود را دارند.

۳-۵-۱ روش استاتیک: تحلیل سورس برنامه بدون اجرا

روش استاتیک به تحلیل و بررسی برنامه بدون اجرای کد می‌پردازد. در این روش تنها سورس برنامه مورد بررسی قرار می‌گیرد. یکی از کاربردهای اصلی این روش، پیدا کردن طولانی‌ترین مسیر برنامه است که بیشترین مصرف گاز را دارد. مراحل این کار به شرح زیر است:

۱. بررسی طولانی‌ترین مسیر در کد پیش از بهینه‌سازی

۲. بررسی طولانی‌ترین مسیر در کد پس از بهینه‌یازی

در نهایت بایستی مقایسه‌ای ما بین مراحل صورت بپذیرد.
در این روش، دو تکنیک اصلی وجود دارد:

- **تحلیل بر پایه درخت^۳:** در این روش از اجرای نمادین استفاده می‌شود تا مسیرهای مختلف برنامه شبیه‌سازی و تحلیل شوند.

- **تحلیل بر پایه گراف^۴:** این روش بر مبنای برنامه‌ریزی خطی صحیح است که برای یافتن مسیرهای بحرانی در گراف‌های کنترل جریان استفاده می‌شود.

در مقالاتی، طولانی‌ترین مسیر برای زمان محاسبه می‌شود و در قالب واحدهای زمانی مورد بحث قرار گرفته است اما در سالی‌دیتی میتوان گاز را مبنا قرارداد و پژوهش‌ها براساس واحدهای گاز مورد بررسی و تحلیل قرار داد.

به عنوان مثال، یک بیسیک بلاک (قطعه کد) ممکن است ۵ سیکل زمانی طول بکشد که می‌توان آن را به گس تبدیل کرده و با استفاده از تکنیک‌های مشابه آن را بررسی نمود.

³Symbolic Execution

⁴ILP (Integer Linear Programming)

۳-۵-۲ روش داینامیک: آزمون و ارزیابی عملکرد

در روش داینامیک، آزمون برنامه بر اساس داده‌های ورودی‌های امکان‌پذیر انجام می‌شود. برنامه با ورودی‌های مختلف تست می‌شود. به عنوان مثال، اگر برنامه دارای ۳ ورودی باشد، می‌توان تعداد بالا مثلاً ۸۰۰۰ بار ورودی مختلف را در برنامه اجرا کرد. این کار به دو صورت انجام می‌شود:

- اجرای برنامه بدون بهینه‌سازی

- اجرای برنامه با اعمال بهینه‌سازی

همچنین در نهایت می‌توان به شکل زیر ارزیابی و تحلیل نمود نتایج این تست‌ها مقایسه شده و به صورت زیر ارزیابی می‌شوند:

- درصد بهبودی

- درصد بدتر شدن

- درصد بدون تغییر

این روش منجر به محاسبه متوسط عملکرد بهینه‌سازی می‌شود. اگر بهبود بیشتری مشاهده شود، این بهینه‌سازی موفق‌تر خواهد بود. روش داینامیک بیشتر در صنعت مورد استفاده قرار می‌گیرد، در حالی که روش استاتیک بیشتر در حوزه‌های آکادمیک کاربرد دارد. [۲۰]

فصل چهارم

روش تحقیق و حل مسئله

۴-۱ مقدمه روش‌ها

در این پژوهش، از دو روش اصلی برای تحلیل و بهینه‌سازی مصرف بهای سوخت در قراردادهای هوشمند اتریوم استفاده می‌شود: **تحلیل استاتیک و تحلیل دینامیک**. این روش‌ها ارزیابی جامعی از صرفه‌جویی‌های بالقوه و واقعی بهای سوخت که از طریق تکنیک‌های بهینه‌سازی مختلف می‌توان به دست آورد، ارائه می‌دهند. در ادامه جزئیات و کاربرد هر یک از این روش‌ها را توضیح می‌دهیم.

۴-۲ تحلیل استاتیک

تحلیل استاتیک شامل بررسی کد قرارداد هوشمند بدون اجرای آن است. این روش از اجرای نمادین^۱ و برنامه‌ریزی خطی عدد صحیح^۲ برای تعیین طولانی‌ترین مسیر و تخمین هزینه بهای سوخت در سناریوهای مختلف استفاده می‌کند.

۴-۲-۱. مدل‌سازی قرارداد هوشمند به عنوان یک گراف

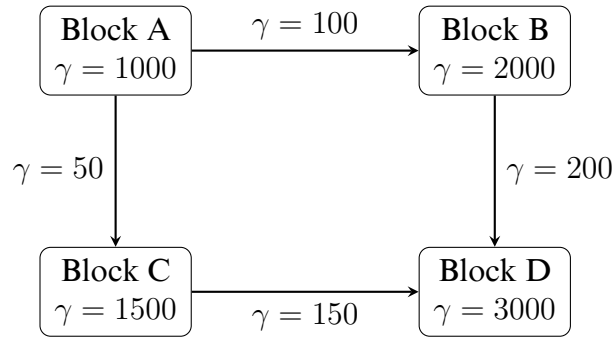
- **گره‌ها:** نشان‌دهنده بلوک‌های کد که ریزدانی آن‌ها تا یک یا چند خط می‌تواند باشد.
- **یال‌ها:** نشان‌دهنده جریان کنترل بین بلوک‌ها یا به عبارت دیگر توالی اجرا دستورات را نشان می‌دهد.
- هر گره و یال با هزینه بهای سوخت γ مشخص می‌شود.

۴-۲-۲. تعریف محدودیت‌های ILP

- برقراری روابط و توالی جریان کنترل بین بلوک‌های کد.

¹Symbolic Execution

²ILP



شکل ۴-۱: ارائه یک گراف از بهای سوخت به ازای هر بلاک کد و جریان کنترل بلوک کد

- حصول اطمینان از اینکه همه مسیرهای احتمالی اجرا در نظر گرفته شده‌اند.

۳-۲-۴. تابع هدف

- به حداقل رساندن هزینه بهای سوخت کل از نقطه ورود تا خروج قرارداد هوشمند.

فرمول‌بندی ILP: فرض کنید γ_i هزینه بهای سوخت مربوط به بلوک i باشد.

- هدف: حداقل رساندن $\sum_i \gamma_i$

Algorithm 1 ILP Formulation for Static Analysis

- 1: Define the objective function coefficients: $c = [\text{gas_cost_block1}, \text{gas_cost_block2}, \dots, \text{gas_cost_blockN}]$
 - 2: Define the equality constraints matrix A_{eq} and vector b_{eq} :
 - 3: $A_{eq} = \begin{bmatrix} 1 & -1 & 0 & \dots & 0 \\ 0 & 1 & -1 & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \end{bmatrix}$
 - 4: $b_{eq} = [0, 0, \dots, 0]$
 - 5: Bounds for each variable x_i (gas cost for each block must be non-negative):
 $x_bounds = [(0, \text{None}) \text{ for } _ \text{ in range(len(c))}]$
 - 6: Solve ILP:
`linprog(c, A_eq=A_eq, b_eq=b_eq, bounds=x_bounds, method='highs')`
 - 7: Output optimal gas costs:
`print(result) = 0`
-

۴-۲-۴ مزایای تحلیل استاتیک

- ارائه حداکثر صرفه‌جویی تئوریک در گاز
- شناسایی نقاط تنگنا^۳ در کد.

۴-۳ تحلیل دینامیک

تحلیل دینامیک شامل اجرای قرارداد هوشمند با ورودی‌های مختلف و اندازه‌گیری مصرف واقعی بهای سوخت است. این روش داده‌های تجربی در مورد اثرگذاری بهینه‌سازی‌ها در شرایط واقعی ارائه می‌دهد.

۴-۳-۱. ۱. آزمون بهای سوخت

- نوشتن تست‌هایی که طیف وسیعی از داده‌های ورودی را پوشش دهند.
- استفاده از ابزارهای تست مانند Truffle یا Hardhat.

۴-۳-۲. ۲. جمع‌آوری داده‌های مصرف بهای سوخت

- اجرای تست‌ها همراه با بهینه‌سازی و بدون بهینه‌سازی.
- ثبت مصرف بهای سوخت برای هر سناریو.

۴-۳-۳. ۳. نحوه ارزیابی

- محاسبه میانگین صرفه‌جویی در بهای سوخت.
- شناسایی سناریوهایی با بهبود یا تضعیف قابل توجه.

³Bottleneck

نمونه اسکریپت تست:

Algorithm 2 Sample Test Script for Dynamic Analysis

- 1: Deploy contract instance β
 - 2: Execute Without Optimization:
 - 3: Call function `functionWithoutOptimization` on β
 - 4: Record transaction object as δ_1
 - 5: Get receipt object as ϵ_1
 - 6: Extract gas used from ϵ_1 as γ_1
 - 7: Execute With Optimization:
 - 8: Call function `functionWithOptimization` on β
 - 9: Record transaction object as δ_2
 - 10: Get receipt object as ϵ_2
 - 11: Extract gas used from ϵ_2 as γ_2
 - 12: Compare Gas Costs:
 - 13: Compare γ_2 with γ_1
 - 14: Assert that $\gamma_2 < \gamma_1 = 0$
-

استقرار قرارداد:

- α نمایانگر مجموعه‌ای از قراردادها است.
- β نمونه‌ای از قرارداد مستقر شده است.

اجرای بدون بهینه‌سازی:

- δ_1 تراکنش برای تابع بدون بهینه‌سازی است.
- ϵ_1 رسید شامل میزان بهای سوخت مصرف‌شده است.

اجرای با بهینه‌سازی:

- δ_2 تراکنش برای تابع با بهینه‌سازی است.
- ϵ_2 رسید شامل میزان بهای سوخت مصرف‌شده است.

مقایسه هزینه‌های بهای سوخت:

- مقایسه بهای سوخت مصرف‌شده در تابع بهینه‌شده $(\epsilon_2 \cdot \gamma)$ با بهای سوخت مصرف‌شده در تابع غیر بهینه $(\epsilon_1 \cdot \gamma)$.

مزایای تحلیل دینامیک

- ارائه داده‌های واقعی مصرف بهای سوخت.
- تاثیر بهینه‌سازی‌ها بر ورودی‌های مختلف را ثبت می‌کند.
- برای اعتبارسنجی پیش‌بینی‌های تئوریک مفید است.

محدودیت‌های تحلیل دینامیک

- نیاز به پوشش بهای سوخت‌ترده تست برای مؤثر بودن.
- ممکن است تمامی مسیرهای اجرای ممکن را شامل نشود.
- برای فضای ورودی بزرگ می‌تواند زمان‌بر باشد.

رویکرد ترکیبی

با ترکیب تحلیل استاتیک و دینامیک، می‌توانیم درک جامعی از رفتار مصرف بهای سوخت در قراردادهای هوشمند به دست آوریم. تحلیل استاتیک به شناسایی حداکثر صرفه‌جویی تئوریک و نقاط تنگنا کمک می‌کند، در حالی که تحلیل دینامیک شواهد تجربی از اثرگذاری این بهینه‌سازی‌ها در سناریوهای واقعی ارائه می‌دهد. این رویکرد دوگانه اطمینان حاصل می‌کند که بهینه‌سازی‌ها نه تنها در تئوری مناسب هستند، بلکه در محیط‌های عملی نیز مزایای ملموسی به همراه دارند.

۴-۴ صورت مسئله اول بهینه سازی بهای سوخت

قراردادهای هوشمند اجرای کدهای قابل پیاده سازی روی زنجیره بلوکی را امکان پذیر می کنند. هزینه اجرای کد قراردادهای هوشمند با استفاده از واحد بهای سوخت یا گاز^۴ اندازه گیری می شود - مقدار دقیق آن بر اساس پیچیدگی محاسباتی قرارداد هوشمند متناظر با آن است. بنابراین، بهینه سازی کد قراردادهای هوشمند برای کاهش مصرف بهای سوخت و در برخی موارد، جلوگیری از حملات مخرب بسیار ضروری است. در این تحقیق، رویکردهای لازم برای بهینه سازی مصرف بهای سوخت در قراردادهای هوشمند ارائه شده است، به خصوص در ساختارهای حلقه ای و کنترل حلقه. ما یک پیاده سازی نمونه از این رویکرد با استفاده از ابزارها برای قراردادهای هوشمند سالییدی ارائه داده ایم. تکنیک خود را با استفاده از بیش از ۳۰ قرارداد هوشمند سالییدی ارزیابی کرده ایم. ارزیابی ها نشان داد که میانگین صرفه جویی در هزینه بهای سوخت در هر تراکنش حدود ۲۴۰۰۰ واحد بهای سوخت یا کاهش معادل ۲۱٪ در هزینه های بهای سوخت است. با وجود اینکه این رویکرد باعث افزایش طراحی کمی هزینه به دلیل توابع داخلی اضافی می شود، اما این تنها حدود ۱۷۰۰۰ واحد بهای سوخت در میانگین یا ۵٪ از کل هزینه های استقرار است. این افزایش تا حد زیادی قابل قبول باقی می ماند، مقایسه با صرفه جویی در هزینه بهای سوخت برای هر تراکنش، و همچنین کارآمدی، کاربردی بودن و اثربخشی روش پیشنهادی را تایید می کند. [۱]

۴-۴-۱ ویژگی های قرارداد هوشمند استاندارد

از یک قرارداد هوشمند خوب به دو مورد اساسی و کلی می توان اشاره داشت:

• صحت

• هزینه

□ هزینه مربوط به استقرار

□ هزینه فراخوانی یا اجرا هر تابع از قرارداد

نکته حائز اهمیت درخصوص بحث هزینه، این می‌باشد که عموماً نمی‌توان هر دو هزینه را با یکدیگر به حداقل رساند، برای مثال گاهی نیاز است، برای کم کردن هزینه محاسبات سمت کاربر (حین اجرا)^۵ نیاز می‌شود، هزینه اولیه استقرار^۶ را افزایش داد و در یک تعارض (دوگانگی)^۷ قرار می‌گیرد. این مسئله اهمیت این موضوع را زیاد می‌کند که باید بدانیم با چه هدفی و در چه زمانی می‌خواهیم چه هزینه‌ای را بهینه کنیم؟

بطور مثال وقتی هزینه‌های مربوط به بخش کاربر بهینه می‌شود، هزینه‌های مربوط به استقرار زیاد می‌گردد. پس در پارامتر مربوط به انجام بهینه سازی برای بهای سوخت هدف باید کاملاً مشخص باشد. برای اعمال تاثیر مستقیم یا معکوس، توابع هدف به شکل زیر در خواهد آمد:

$$\text{OptimizationFunction} = w_1 \cdot (\sum \text{PositiveFactors}) + w_2 \cdot (\sum \text{NegativeFactors})$$

۲-۴-۴ پارامترهای تاثیرگذار در بهای سوخت

در این بخش تمامی پارامترهای تاثیرگذار بر بهای سوخت را یک به یک مورد بررسی قرار داده تا در بخش‌های مربوط به پیاده سازی از آنها استفاده شود.

همانطور که در **جدول ۱-۴** نیز مشاهده می‌شود، در این جدول اطلاعاتی به هر آپ‌کد درج گشته است و برای بیش از ۱۰۰ آپ‌کد مقدار بهای سوخت مصرفی و اطلاعات دیگری مثل نحوه ذخیره سازی عملوندها قابل دستیابی می‌باشد.

آشنایی با جدول معرفی و هزینه اجرای آپ‌کد

مانند زبان ماشین در برنامه نویسی مربوط به قراردادهای هوشمند نیز، با یک جدول آپ‌کد سر و کار خواهیم داشت. [۱۳]

⁵Runtime

⁶Deploy

⁷Trade-off

جدول ۴-۱: قسمتی از جدول مربوط به آپ‌کدها

پشته	نام	بهای سوخت	متغیرهای پشته در ابتدا
۰۰	STOP	۰	-
۰۱	ADD	۳	a, b
۰۲	MUL	۵	a, b
۰۳	SUB	۳	a, b
۰۴	DIV	۵	a, b
۰۵	SDIV	۵	a, b

بعنوان یک مثال از این قسمت، می‌توان مشاهده کرد که استفاده از آپ‌کد ضرب بجای آپ‌کد مربوط به توان، ما را با یک عملیات بهینه‌تر مواجه می‌کند و بطور مثال؛ در این مثال با کاهش حدود ۲۰۰ واحد بهای سوخت (وی^۸) همراه خواهیم بود. این مقدار شاید در ابتدا مقدار چشم‌گیری نبوده باشد اما در عملیات سنگینی همچون یک حلقه که این تابع را فراخوانی می‌کند، می‌تواند تاثیر به‌سزایی ایجاد نماید.

تحلیل نمادین و برنامه‌ریزی خطی عدد صحیح (تحلیل استاتیک)

سناریوی غیربهینه: استفاده از power در سالی‌دیتی

کد مربوط به توان و ضرب را مدنظر قرار داده

تحلیل نمادین

• فراخوانی تابع: `powerExample(x)`

• عملیات: x^2

• نماد بهای سوخت: $\gamma_{\text{pow}}(x)$

⁸Way

فرمول بندی برنامه ریزی خطی عدد صحیح

$$\min \Gamma = \gamma$$

با در نظر گرفتن:

$$\gamma_{\text{pow}} \geq 0$$

سناریوی بهینه: استفاده از ضرب به جای power در سالی دیتی

تحلیل نمادین

• فراخوانی تابع: `multiplicationExample(x)`

• عملیات: $x * x$

• نماد بهای سوخت: $\gamma_{\text{mul}}(x)$

فرمول بندی برنامه ریزی خطی عدد صحیح

$$\min \Gamma = \gamma_{\text{mul}}$$

با در نظر گرفتن:

$$\gamma_{\text{mul}} \geq 0$$

بسط تحلیل استاتیک بهای سوخت در جدول آپ کد

عملیات های مربوط به آپ کدهای مختلف را می توان بسط داده و با بقیه موارد در جدول آپ کد

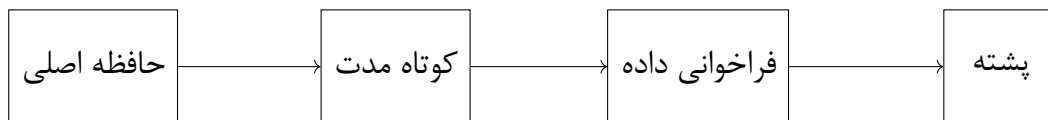
ترکیب نمود

$$\gamma_{\text{total_pow}} = \gamma_{\text{pow}} + \gamma_{\text{add}} + \gamma_{\text{mload}}$$

در چنین حالتی بهای سوخت کلی یک بلوک کد، می‌تواند مجموع بهای سوخت تک تک موارد باشد.

هزینه استفاده از هر نوع حافظه

در قراردادهای هوشمند چهار نوع مختلف از حافظه وجود دارد. محل خوانده شدن دیتا و محل ثبت شدن دیتا اهمیت بسیار زیادی در هزینه خواهد داشت.



شکل ۴-۲: حافظه‌ها در قراردادهای هوشمند

- حافظه پشته: این حافظه یک صف (آخرین وارد شده، اولین خروجی داده شده)^۹ است که توسط ماشین مجازی برای بازیابی آرگومان‌های ورودی دستورات و ذخیره آنها استفاده می‌شود. این حافظه کمترین میزان هزینه را دارد.
- حافظه فراخوانی داده: ^{۱۰} حافظه ای است که آرگومان‌هایی که توابع دریافت می‌کنند، موجود می‌باشند.
- حافظه کوتاه مدت: ^{۱۱} حافظه یک مکان است که می‌توان به طور موقتی متغیرها را ایجاد و ذخیره کنیم.
- حافظه اصلی: حافظه‌ای اصلی که اطلاعات مربوط به وضعیت‌ها و داده‌های دیگر در آن ذخیره سازی می‌گردد.

در زبان قراردادهای هوشمند اتریوم، هزینه‌های مربوط به حافظه‌ها به عوامل مختلفی بستگی دارد. چندین عاملی که هزینه‌ها را تحت تأثیر قرار می‌دهند را بررسی می‌شود:

^۹LIFO

^{۱۰}Call Data

^{۱۱}Memory

موارد کم هزینه

۱. خواندن ثابت‌ها و متغیرهای تغییرناپذیر: خواندن مقادیر ثابت ^{۱۲} معمولاً ارزان‌تر است.
۲. خواندن و نوشتن متغیرهای محلی ^{۱۳}: عملیات خواندن و نوشتن متغیرهای محلی در حافظه از ارزان‌ترین عملیات‌ها هستند.
۳. خواندن و نوشتن متغیرهای حافظه: عملیات‌های خواندن و نوشتن در حافظه کوتاه مدت، مانند آرایه‌ها و ساختارها، نسبت به سایر نوع‌های داده معمولاً ارزان‌تر هستند.
۴. خواندن متغیرهای فراخوانی داده‌ها: عملیات خواندن متغیرهای فراخوانی داده‌ها مانند آرایه‌ها و ساختارها اغلب ارزان‌تر از نوشتن آنها هستند.
۵. فراخوانی‌های داخلی ^{۱۴}: فراخوانی‌های داخلی به صورت مستقیم اجرا می‌شوند و معمولاً ارزان‌تر از فراخوانی‌های خارجی ^{۱۵} هستند.

موارد پرهزینه:

۱. خواندن و نوشتن متغیرهای وضعیت ^{۱۶} عملیات‌های خواندن و نوشتن متغیرهای ذخیره‌شده در ذخیره‌گاه قرارداد هزینه‌بر هستند.
۲. فراخوانی‌های خارجی: فراخوانی‌های به قراردادهای دیگر (خارجی) اغلب هزینه‌بر هستند.
۳. حلقه‌ها: اجرای حلقه‌ها ممکن است هزینه‌بر باشد، مخصوصاً اگر حلقه‌ها طولانی باشند یا تعداد بزرگی از عملیات در داخل حلقه انجام شود.

تحلیل نمادین استفاده از حافظه‌های مختلف

کد مربوط به استفاده از حافظه‌های مختلف را مدنظر قرار داده و برطبق آن تحلیل انجام می‌شود.

$$\text{Storage costs: } \gamma_{total_storages} = \gamma_{stack} + \gamma_{calldata} + \gamma_{memory} + \gamma_{storage}$$

¹²immutable variables

¹³Local

¹⁴Internal Function Calls

¹⁵External Function Calls

¹⁶state variables

Symbolic: Let S be the set of storage operations, then $\gamma_{storage} = \sum_{s \in S} cost(s)$

حلقه‌ها

حلقه‌ها در صورتی که استانداردهای لازم را رعایت نکرده باشند، مصرف کننده بهای سوخت بالایی خواهند بود.

در این **قطعه کد** ابتدا یک متغیر در حافظه اصلی مقدار دهی شده است و سپس در یک تابع یک حلقه در حال افزایش مقدار این متغیر عمومی است، این کد دارای خروجی بهای سوخت بسیار بالا و غیر بهینه‌ای است و می‌تواند منجر به یک فاجعه گردد.

یادداشت ۴-۴-۱. عملیات هزینه‌بر در حلقه: به دلیل آپ‌کدهای گران‌قیمت خواندن و نوشتن، مدیریت یک متغیر در حافظه اصلی به مراتب گران‌تر از مدیریت متغیرها در حافظه کوتاه مدت است. به همین دلیل، متغیرهای حافظه اصلی در حلقه‌ها نباید به کار گرفته شود.

الگوریتم ۳ محاسبه جمع بدون متغیر عمومی

```
0: sum ← 0
0: function p3(x)
0:   for i ← 0 to x - 1 do
0:     sum ← 1
0:   end for
0: end function=0
```

یادداشت ۴-۴-۲. **قطعه کد** الگوریتم نوشته شده در حلقه نامناسب

در این مثال خاص با استفاده از یک متغیر موقتی یک کپی از متغیر اصلی گرفته شد و عملیات روی این متغیر با هزینه کمتر صورت گرفته و در نهایت مقدار دهی صورت می‌گیرد. که این هزینه از هزینه کد قبلی به مراتب کمتر خواهد بود.

الگوریتم ۴ محاسبه جمع و بهروزرسانی متغیر عمومی

```

0:  $sum \leftarrow 0$ 
0: function p3( $x$ )
0:    $temp \leftarrow 0$ 
0:   for  $i \leftarrow 0$  to  $x - 1$  do
0:      $temp \leftarrow temp + i$ 
0:   end for
0:    $sum \leftarrow sum + temp$ 
0: end function=0
    
```

یادداشت ۴-۴-۳. قطعه کد در زبان سالییدی الگوریتم نوشته شده در حلقه به کمک متغیر داخلی

تحلیل نمادین حلقه‌ها

Loops: $\gamma_{loop} = n \cdot \gamma_{iteration}$, where n is loop iterations

ILP: Constraint: $n \leq MAX_ITERATIONS$

مینیمم سازی داده‌های ذخیره شده روی زنجیره

در حد امکان از ذخیره سازی اطلاعات غیر ضروری در زنجیره بلوکی باید پرهیز شود. پیشنهاد می‌شود برای ذخیره سازی این اطلاعات در مولفه های دیگر ذخیره سازی صورت بگیرد و یا از پلتفرم‌های ذخیره سازی داده^{۱۷} کمک گرفته شود.

تحلیل نمادین ذخیره داده روی زنجیره بلوکی

در زنجیره بلوکی به ازای بایت هزینه بهای سوخت می‌تواند شامل تغییر شود.

Minimize data: $\gamma_{data} = size(data) \cdot cost_per_byte$

Symbolic: Let D be the set of data elements, $\gamma_{data} = \sum_{d \in D} size(d) \cdot cost$

$_per_byte$

¹⁷IPSF

کد مرده یا کد دزدیده شده

کدی است که هرگز اجرا نخواهد شد زیرا اجرای آن به شرطی وابسته است که همواره به صورت نادرست باز می‌گردد و به این دلیل، این کد کدی بلااستفاده خواهد بود و سبب مصرف بهای سوخت بسیار می‌شود.

الگوریتم ۵ کد مرده

```
0: procedure deadtode( $x$ )
0:   if  $x > 1$  then
0:     if  $x = 2$  then
0:       return  $x$ 
0:     end if
0:   end if
0: end procedure=0
```

یادداشت ۴-۴-۴. **قطعه کد** مربوط به شرطهای متناقض

الگوریتم ۶ الگوریتم برای تابع opaquePredicate

```
0: procedure opaquePredicate( $x$ )
0:   if  $x > 1$  then
0:     if  $x > 0$  then
0:       return  $x$ 
0:     end if
0:   end if
0: end procedure=0
```

یادداشت ۴-۴-۵. **قطعه کد** مربوط به تابع مبهم

تحلیل نمادین کد مرده یا بلااستفاده در شرطها

Dead code: $\gamma_{dead} = 0$ (after removal)

ILP: Remove constraints related to dead code

استفاده از توابع هش به جهت مقایسه رشته‌ها

در قراردادهای هوشمند روش تعبیه‌شده‌ای برای مقایسه رشته‌ها وجود ندارد. مقایسه رشته‌ها با استفاده از یک کتابخانه ممکن است به طرز دور از انتظاری گران‌تر باشد. در اغلب موارد، بهترین راه ایجاد یک تابع برای مقایسه رشته‌هاست که بر اساس یک تابع هش مقایسه کند. محاسبه و مقایسه هش‌های رشته نسبت به هزینه بهای سوخت روی ماشین مجازی اتریوم نسبتاً ارزان است و کافی است در صورت نیاز رشته‌ها با شرط برابری مقایسه و چک شوند.

الگوریتم ۷ `compareStringsWithLib` تابع

```
0: procedure compareStringsWithLib
0:   return areStringsEqual("totally different1", "totally different2")
0: end procedure=0
```

یادداشت ۴-۴-۶. [قطعه کد](#) مربوط به تابع مقایسه رشته‌ها با کتابخانه

الگوریتم ۸ `compareStringsWithHash` تابع

```
0: procedure compareStringsWithHash
0:   return areStringsEqual("totally different1", "totally different2")
0: end procedure=0
```

یادداشت ۴-۴-۷. [قطعه کد](#) مربوط به تابع مقایسه رشته‌ها با هش

تحلیل نمادین کد مرده یا بلااستفاده در شرط‌ها

Hash comparisons: $\gamma_{hash_cmp} < \gamma_{string_cmp}$

Symbolic: Replace string comparison operations with hash comparisons

استفاده از دستور کدهای اسمبلی

در مواقعی که اهمیت بهای سوخت در کد بسیار زیاد می‌شود، می‌توان بجای سالی‌دیتی معمولی از قطعه کد های داخلی اسمبلی استفاده نمود. این کار می‌تواند با ضعف و منفعت هایی همراه شود.

برای مثال، برای استفاده از اسمبلی باید خطراتی همچون غیرفعال شدن برخی کنترل‌های صحتی را بدنبال خواهد داشت و کد مستعد ضعف‌های صحتی خواهد بود. بطور تجربی میتوان گفت که استفاده از کدهای اسمبلی چیزی در حد ۲۰ الی ۳۰ درصد بهای سوخت‌ها را کاهش می‌دهد.

الگوریتم ۹ الگوریتم برای بهینه‌سازی بهای سوخت با استفاده از دستور کدهای داخلی اسمبلی

```
0: procedure sumArrayWithAssembly(arr)
0:   sum ← 0
0:   assembly:
0:     len ← mload(arr)
0:     data ← add(arr, 32)
0:     for i ← 0 to len - 1 do
0:       val ← mload(data)
0:       sum ← sum + val
0:       data ← add(data, 32)
0:     end for
0:   return sum
0: end procedure=0
```

یادداشت ۴-۴-۸. **قطعه کد** مربوط به استفاده از توابع داخلی اسمبلی

تحلیل نمادین استفاده از اسمبلی کد یا کد سطح پایین

Assembly code: $\gamma_{assembly} < \gamma_{high_level}$

ILP: Replace high-level operations with optimized assembly in constraints

استفاده از بایت ۳۲ بجای رشته

اگر رشته‌های کوتاه (کمتر از ۳۲ بایت) بهتر است از بایت ۳۲ بجای نوع داده رشته استفاده گردد.

جدول ۴-۲: جدول مقایسه بهای سوخت در دو حالت استفاده از رشته و استفاده بایت ۳۲ بجای رشته

یادداشت ۴-۴-۹. این قاعده برای رشته‌هایی با سایز ثابت بهتر عمل خواهد نمود.
نمونه کد

تحلیل نمادین استفاده از اسمبلی کد یا کد سطح پایین

32-byte vs string: $\gamma_{32byte} < \gamma_{string}$

Symbolic: Replace string variables with 32-byte variables where possible

استفاده از بسته‌بندی ساختار^{۱۸}

زمانی که ساختارهای حافظه قرار است بسته شود، بهتر است ساختارها به نحوی بسته‌بندی گردد که کلمات و یا واحدهای حافظه بصورت ۲۵۶ بیتی بسته‌بندی گردد. این الگوریتم برای بهینه‌سازی استفاده از حافظه در سالی‌دیتی استفاده می‌شود. واحدهای حافظه در سالی‌دیتی ۲۵۶ بیتی هستند، بنابراین وقتی ساختارهای حافظه بسته شوند، مناسب است که ساختارها به گونه‌ای بسته‌بندی شوند که کلمات یا واحدهای حافظه به صورت ۲۵۶ بیتی بسته‌بندی شوند. این کار می‌تواند به بهبود عملکرد و کاهش مصرف حافظه کمک کند.

تحلیل نمادین بسته‌بندی ساختار

Struct packing: $\gamma_{packed} < \gamma_{unpacked}$

ILP: Minimize storage slots used by structs

¹⁸Struct Packing

ارزیابی‌های ارزیابی اتصال کوتاه

عملگرهای $||$ و $\&$ از قوانین اتصال کوتاه پیروی می‌کنند. این به این معناست که در عبارت $f(x) || g(y)$ ، اگر $f(x)$ درست ارزیابی شود، $g(y)$ حتی اگر نادرست باشد اعمال نخواهد شد. بنابراین، اگر یک عمل منطقی شامل یک عملیات بهای سوخت بالا و یک عملیات بهای سوخت پایین باشد، به ترتیبی که عملیات بهای سوخت بالا را به صورت اتصال کوتاه انجام داد، می‌تواند منجر به کاهش مصرف بهای سوخت شود.

اگر $f(x)$ بهای سوخت پایین داشته باشد و $g(y)$ گران بهای سوخت بالا داشته باشد، ترتیب انجام دادن عملیات‌های منطقی به صورت زیر، در صورت انجام عملیات اتصال کوتاه، منجر به بیشترین صرفه‌جویی در مصرف بهای سوخت خواهد شد:

• OR: $f(x) || g(y)$ (در صورت درست بودن تابع اول اتصال کوتاه می‌شود)

• AND: $f(x) \& g(y)$ (در صورت غلط بودن تابع اول اتصال کوتاه انجام می‌شود)

اگر $f(x)$ احتمال غلط قابل ملاحظه‌ای نسبت به $g(y)$ داشته باشد، ترتیب دادن به عملیات‌های AND به صورت $f(x) \& g(y)$ ممکن است منجر به صرفه‌جویی بیشتر در مصرف بهای سوخت در حالت اتصال کوتاه شود.

اگر $f(x)$ احتمال درست بسیار بیشتری نسبت به $g(y)$ داشته باشد، ترتیب دادن به عملیات‌های OR به صورت $f(x) || g(y)$ ممکن است منجر به صرفه‌جویی بیشتر در مصرف بهای سوخت در حالت اتصال کوتاه شود.

تحلیل نمادین ارزیابی کوتاه (اتصال کوتاه)

Short circuit: $\gamma_{short_circuit} \leq \gamma_{full_evaluation}$

Symbolic: Evaluate conditions left-to-right, stop at first false (AND) or true (OR)

تحلیل نمادین برای سناریوی کد مرده

در این سناریو، کد مرده قطعه‌ای از کد است که هرگز اجرا نمی‌شود و صرفاً فضای اضافی در قرارداد هوشمند را اشغال می‌کند. این کد نه تنها بی‌فایده است بلکه هزینه‌های اضافی گس را نیز تحمیل می‌کند.

Algorithm 10 Symbolic Execution for Identifying Dead Code

- 1: Input: Smart contract
 - 2: Output: Identification and removal of dead code
 - 3: Identify all functions present in the contract
 - 4: for each function in the contract do
 - 5: Check the number of calls to the function
 - 6: if number of calls to the function == 0 then
 - 7: Mark the function as dead code
 - 8: end if
 - 9: end for
 - 10: Remove all functions marked as dead code =0
-

فرمول‌های تحلیل نمادین

• اجرای تابع `usefulFunction`

$$\gamma_{\text{usefulFunction}} = \text{GAS_LOAD} + \text{GAS_RETURN}$$

□ $\gamma_{\text{usefulFunction}}$ نشان‌دهنده هزینه گس برای اجرای تابع `usefulFunction` است.

□ عملیات‌ها:

✱ `GAS_LOAD`: هزینه بارگذاری متغیر `data` از حافظه اصلی.

✱ `GAS_RETURN`: هزینه بازگرداندن مقدار.

• اجرای تابع uselessFunction

$$\gamma_{\text{uselessFunction}} = \text{GAS_PURE}$$

□ $\gamma_{\text{uselessFunction}}$ نشان‌دهنده هزینه گس برای اجرای تابع uselessFunction است.

□ عملیات:

✱ GAS_PURE: هزینه اجرای یک تابع خالص که هیچ‌گونه اثری ندارد.

• هزینه کلی قرارداد

$$\gamma_{\text{total}} = \gamma_{\text{constructor}} + \gamma_{\text{usefulFunction}} + \gamma_{\text{uselessFunction}}$$

□ γ_{total} نشان‌دهنده هزینه گس کل قرارداد است.

□ اگر کد مرده (یعنی تابع uselessFunction) را حذف کنیم، هزینه کلی کاهش می‌یابد:

$$\gamma_{\text{total_optimized}} = \gamma_{\text{constructor}} + \gamma_{\text{usefulFunction}}$$

نتیجه‌گیری

با حذف کد مرده (تابع uselessFunction)، هزینه گس کلی قرارداد کاهش می‌یابد. تحلیل نمادین به ما کمک می‌کند تا تأثیر کد مرده را شناسایی کرده و بهینه‌سازی‌های لازم را انجام دهیم. این بهینه‌سازی منجر به کاهش هزینه‌های گس و بهبود عملکرد قرارداد هوشمند می‌شود.

استفاده از متغیر سائز ثابت بجای سائز متغیر

استفاده از متغیر سائز ثابت نسبت به متغیر با سائز متغیر منجر به بهینه شدن بهای سوخت خواهد شد.

یادداشت ۴-۴-۱۰. دلیل این امر این می‌باشد که در حالت سایز ثابت نیاز نیست برنامه پردازش‌هایی از قبیل محاسبه طول متغیر و موارد دیگر را انجام دهد.

بهتر است از `bytesX` به جای `byte[]` استفاده شود، زیرا این گزینه بهینه‌تر است. در واقع، `byte[]` بین عناصرش ۳۱ بایت پرکننده اضافه می‌کند. بنابراین استفاده از یکی از انواع مقادیر `bytes1` تا `bytes32`، منجر به صرفه‌جویی در هزینه بهای سوخت خواهد شد.

تحلیل نمادین در این سناریو

Static vs dynamic: $\gamma_{static} < \gamma_{dynamic}$

ILP: Prefer static size variables in constraints

عدم استفاده از کتابخانه‌های سنگین غیر ضروری

در استفاده از کتابخانه‌های سالی‌دیتی بعضاً قرار است یک عمل ساده توسط کتابخانه‌های پیچیده‌ای از جمله `openzeppelin` و یا کتابخانه‌های چند امضایی که از قراردادهای تودرتو^{۱۹} استفاده می‌کنند، استفاده شود، این عمل باعث مصرف بی‌رویه بهای سوخت خواهد شد. در چنین حالت‌هایی بهتر است، آن توابع دلخواه را پیاده سازی نمود و از کتابخانه تا حد امکان استفاده نکرد.

تحلیل نمادین در این سناریو

Avoid libraries: $\gamma_{inline} < \gamma_{library_call}$

Symbolic: Replace library calls with inline code

¹⁹Nested

استفاده از حافظه‌های خارجی^{۲۰}

استفاده از دسترسی خارجی^{۲۱} برای توابعی که تنها از بیرون فراخوانی می‌شوند، اجبار می‌کند که calldata به عنوان مکان پارامتر استفاده شود و این باعث صرفه‌جویی در مصرف بهای سوخت در زمان اجرای تابع می‌شود.

برای تمام توابع عمومی، پارامترهای ورودی به صورت خودکار در حافظه کپی می‌شوند و این هزینه بهای سوخت ایجاد می‌کند. اگر تابع شما تنها از بیرون فراخوانی می‌شود، باید به صورت صریح به عنوان تابع عمومی مشخص شود. در توابع عمومی، پارامترهای ورودی در حافظه کپی نمی‌شوند، بلکه مستقیماً از calldata خوانده می‌شوند. این بهینه‌سازی کوچک در کد سالی‌دیتی می‌تواند در صرفه‌جویی بزرگی در مصرف بهای سوخت کمک کند بخصوص زمانی که پارامترهای ورودی تابع بسیار بزرگ باشد.

یادداشت ۴-۴-۱۱. درست است که این کار سبب می‌گردد بهای سوخت بهینه‌تر داشته باشیم، اما از لحاظ صحتی گاهی امن نمی‌باشد.

تحلیل نمادین در این سناریو

External visibility: $\gamma_{external} < \gamma_{public}$ for functions called externally

ILP: Use external visibility for functions in optimization constraints

استفاده از مقدار بازگشتی مستقیم و عدم استفاده از متغیر اضافه

یک بهینه‌سازی ساده این است که مقدار بازگشتی تابع را نام‌گذاری کنیم. نیازی به ایجاد یک متغیر محلی نیست. بعنوان مثال؛ این بهینه‌سازی ۵ بهای سوخت در هزینه اجرا صرفه‌جویی می‌کند. این مقدار زیادی نیست، اما اگر کد به طور چشمگیری بر اینگونه توابع متکی باشد، ممکن است

²⁰ External Visibility

²¹ External

ارزشمند تلقی شود. توجه به این نکته که می‌توان دستور بازگشت (return) را نیز حذف کرد، اما این تأثیری بر مصرف بهای سوخت ندارد، اما از لحاظ صحتی کد امن‌تری را به ارمغان خواهد آورد.

تحلیل نمادین در این سناریو

Direct return: $\gamma_{direct_return} < \gamma_{variable_then_return}$

Symbolic: Replace variable assignments followed by returns with direct returns

استفاده از منطق جایگزین بجای آرایه‌های طول متغیر در حلقه

در ابتدا این منطق ممکن است پرهزینه نباشد، اما در بعضی مواقع خصوصاً در حلقه این قضیه ممکن است سبب تمام شدن بهای سوخت شود.

تحلیل نمادین در این سناریو

Swap vs array: $\gamma_{swap} < \gamma_{array_operation}$ for simple swaps

ILP: Replace array operations with swap operations in constraints

استفاده از ابزارهای کاهنده بهای سوخت

بهتر است از فعال بودن بهینه‌ساز سالی‌دیتی اطمینان حاصل شود. برای استقرار قرارداد بایستی بهینه‌سازی بهای سوخت انجام شود (که استقرار یک قرارداد کمترین هزینه را دارد)، در این حالت سطح بهینه‌ساز سالی‌دیتی بایستی در مقدار پایین تنظیم شود. در مقابل اگر بایستی برای هزینه‌های بهای سوخت زمان اجرا بهینه‌سازی انجام شود (زمانی که توابع بر روی یک قرارداد فراخوانی می‌شوند)، سطح بهینه‌ساز در مقدار بالا تنظیم می‌شود.

تحلیل نمادین در این سناریو

Gas analyzer tools: Used to estimate γ_{total} for the contract

Both: Use tool output to refine ILP constraints and symbolic execution paths

استفاده از رویدادها بعنوان حافظه

داده‌هایی که تنها یک بار تولید می‌شود و هیچ گاه دیگر به‌روز نمی‌شود را می‌توان در رویدادها ذخیره نمود. داده‌هایی که نیازی به دسترسی در زنجیره ندارند، می‌توانند در رویدادها ذخیره شوند تا بهای سوخت صرفه‌جویی شود. اگرچه این تکنیک ممکن است کار کند، اما توصیه نمی‌شود - رویدادها برای ذخیره داده‌ها طراحی نشده‌اند.

تحلیل نمادین در این سناریو

Events vs storage: $\gamma_{event} < \gamma_{storage}$ for logging

Symbolic: Replace storage operations with events where possible

استفاده از مقادیر دقیق عدد بجای متغیرهای نیاز به محاسبه

در صورتی که متغیری که از حاصل چند متغیر دیگر داریم بهتر است مقدار دقیق آن را محاسبه نماییم، ضرب چند عدد در یکدیگر توسط کامپایلر هندل می‌گردد اما متغیرهای نیاز به محاسبه باعث هزینه محاسبه اضافه می‌شوند.

تحلیل نمادین در این سناریو

Statics vs variables: $\gamma_{static} < \gamma_{variable}$

ILP: Prefer static values in optimization constraints

استفاده از مقدار هش بجای تابع محاسبه هش

حتی الامکان باید سعی شود از مقادیر هش بصورت مستقیم استفاده شود و از توابع مخصوص محاسبه هش Keccak استفاده نشود.

بهینه سازی ترتیب تعریف متغیرها

متناسب با آنچه که پیش تر در قسمت استفاده از **بسته بندی ساختار** صحبت شد، سالیدیتی متغیرها را در کلمات ۲۵۶ بایتی قرار می دهد، بهتر است ترتیبها به نحوی مرتب سازی شده باشد تا کلمات حافظه به بهینه ترین شکل وجود داشته باشد.

تحلیل نمادین در این سناریو

Direct hash vs keccak: $\gamma_{direct_hash} < \gamma_{keccak}$

Symbolic: Replace keccak operations with pre-computed hashes where possible

استفاده ابزار گزارش دهنده بهای سوخت^{۲۲}

این ابزار هر بار که قرار داد اجرا خواهد شد، مصرف بهای سوخت هر تابع را نمایش می دهد و کمک می کند در حین کار متوجه بهای سوخت بالای غیر معمولی برخی توابع گردد که نیاز به بهینه سازی آنها وجود دارد. علاوه بر این پیشنهاد می شود قراردادهای پیش از استقرار چند بار با هدف بهینه سازی بهای سوخت خوانده و بهبود یابند و حتما بازرسی صورت پذیرد.

²²Eth-Gas-Reporter

استاندارد ۲۵۳۵ (الماس)

استاندارد ۲۵۳۵^{۲۳} که به عنوان استاندارد الماس^{۲۴} نیز شناخته می‌شود، یک استاندارد در زمینه توسعه قراردادهای هوشمند در زنجیره بلوکی اتریوم است. این استاندارد به توسعه‌دهندگان امکان می‌دهد تا قراردادهای هوشمند پیچیده‌تر و بهای سوخته‌تر را با بهره‌گیری از تکنیک‌های مشابهی به صورت بهینه و ساختارمند توسعه دهند.

استاندارد الماس از الگوی معماری قراردادهای هوشمند قابل تغییر و انعطاف‌پذیر استفاده می‌کند تا از پرداخت مجدد هزینه‌های توسعه جلوگیری کند. این الگو از قرارداد اصلی قرارداد پایه و قراردادهای کمکی^{۲۵} تشکیل شده است. قرارداد اصلی اصول و ویژگی‌های اساسی را دارا می‌باشد و به تمامی قراردادهای کمکی ارث‌بری می‌کند. این قراردادهای کمکی همچنین قابلیت اضافه کردن و تغییر ویژگی‌ها و توابع را دارند.

از اهداف اصلی این استاندارد افزایش بهره‌وری، افزایش قابلیت انعطاف‌پذیری، و کاهش هزینه‌ها در توسعه و تغییر قراردادهای هوشمند در شبکه اتریوم است. این استاندارد به توسعه‌دهندگان امکان می‌دهد تا قراردادهای هوشمند پیچیده‌تر و بهای سوخته‌تری را ایجاد کرده و از مزایای بهره‌برداری از این الگو در زمینه‌هایی مانند صحت، بهره‌وری، و قابلیت انعطاف‌پذیری بهره‌برند.

تحلیل نمادین در این سناریو

2535 diamond standard: $\gamma_{diamond} < \gamma_{monolithic}$ for large contracts

Both: Model contract as separate facets in ILP and symbolic execution

²³EIP-2535 (ERC-2535)

²⁴Diamond Standard

²⁵Facet Contracts

عدم استفاده از متغیرهای موقت برای مبادله

استفاده از متغیرهای موقت برای جابجایی دو مقدار می‌تواند منجر به افزایش پیچیدگی کد و مصرف منابع شود. به جای این روش، می‌توان از یک تجزیه تغییر یافته و چندتایی‌ها^{۲۶} استفاده کرد. این روش باعث کاهش مصرف بهای سوخت و بهبود خوانایی کد می‌شود.

عدم استفاده بی‌رویه از رویدادها

مطابق آنچه پیشتر درباره استفاده از **رویداد** نسبت به استفاده از حافظه‌ها گفته شد، اما باید دانست که استفاده بی‌رویه از رویداد نیز خود می‌تواند سبب مصرف بیش از حد بهای سوخت شود.

نمونه کد

عدم استفاده از توابع Assertion

تا حد امکان از استفاده از توابع Assertion بایستی پرهیز نمود، چرا که هزینه بهای سوخت بسیار بالایی دارند.

نمونه کد

نام گذاری و ترتیب توابع

مطابق آنچه پیش‌تر درباره ترتیب تعریف **متغیرها** گفته شد، ترتیب توابع نیز حائز اهمیت می‌باشد. علاوه بر این نام توابع نیز تاثیرگذار خواهد بود، نامی که یک تابع در قرارداد هوشمند خواهد داشت، به هش تبدیل می‌شود و سپس هش‌های توابع به ترتیب هگزادسیمال قرار خواهند گرفت (به ترتیب آنچه واقعا در قرارداد هوشمند وجود دارند نیستند)

به ازای هر تابع ۲۲ بهای سوخت اضافه‌تر نسبت به قبلی مصرف خواهد داشت. ترتیب به مبنای شناسه تابع است. بنابراین، اگر تابعی که بیشترین دسترسی را دارد را به ترتیب شناسه توابع به جلوتر منتقل شود، می‌توان هزینه بهای سوخت را کاهش داد.

²⁶Tuple

یادداشت ۴-۴-۱۲. ابزارهایی جهت نامگذاری توابع وجود دارد که پیشنهاد می‌کنند برای توابع از چه نامی استفاده شود تا بهای سوخت بهینه‌تر باشد.

فشرده‌سازی ورودی‌ها در قرارداد هوشمند

بهای سوخت پایه برای ارسال تراکنش ۲۱,۰۰۰ است. هزینه بهای سوخت افزایش می‌یابد اگر اندازه داده‌ها نیز افزایش یابد، که نرخ آن ۶۸ بهای سوخت برای هر بایت است و اگر بایت 0x00 باشد، ۴ بهای سوت خواهد بود. بنابراین، می‌توان با فشرده‌سازی داده‌های ورودی هزینه بهای سوخت را کاهش داد.

تحلیل نمادین در این سناریو

Avoid unnecessary assignments: $\gamma_{no_assignment} < \gamma_{unnecessary_assignment}$

Symbolic: Remove paths with unnecessary variable assignments

مقدار اولیه متغیرها

مقدار اولیه متغیرهایی که در ابتدا باید ۰ باشد بهتر است مقداردهی نشوند، چرا که در سالیدیتی این موارد بصورت پیش‌فرض ۰ در نظر گرفته خواهد شد. اگر یک متغیر مقداردهی نشده باشد، به عنوان مقدار پیش‌فرض (۰، false، 0x00 و غیره بستگی به نوع داده) در نظر گرفته می‌شود. اگر به صورت صریح با مقدار پیش‌فرض آن مقداردهی شوند، فقط بهای سوخت به هدر خواهد رفت.

تحلیل نمادین در این سناریو

Avoid unnecessary assignments: $\gamma_{no_assignment} < \gamma_{unnecessary_assignment}$

Symbolic: Remove paths with unnecessary variable assignments

استفاده از نگاشت به جای آرایه‌ها

بیشتر مواقع استفاده از نگاشت^{۲۷} به جای یک آرایه به سبب کاهش بهای سوخت استفاده می‌شود. اما، زمانی که از نوع داده‌های کوچکتر استفاده می‌شود، آرایه ممکن است انتخاب صحیح‌تری باشد. عناصر آرایه مانند دیگر متغیرهای ذخیره‌سازی در نظر گرفته می‌شوند و فضای کمتری اشغال می‌کنند که می‌تواند هزینه عملیات گران‌تر آرایه را جبران کند. این مورد بیشتر زمانی مفید است که قرار است با آرایه‌های بزرگ کار شود.

تحلیل نمادین در این سناریو

Map vs loops: $\gamma_{map_access} < \gamma_{loop}$ for lookups

ILP: Replace loop constraints with direct map access where possible

استفاده از رشته Require

در سالیدیتی، Require یک عبارت پیش‌تعریف شده است که برای بررسی شرایط پیش‌فرضی استفاده می‌شود. اگر شرط داده شده اجبار شده تا برآورده نشود (یعنی شرایطی که باید صحیح باشد نادرست تلقی شود)، اجرای تراکنش متوقف می‌شود و هزینه‌ای به ازای اجرای ناکام تراکنش از دست می‌رود.

به علاوه، می‌توان رشته‌ای جهت دلیل خطا به همراه آن اظهاریه‌ای ارسال شود تا به راحتی قابل درک باشد که چرا تراکنشی با خطا مواجه شده است. با این وجود، باید به خاطر داشت که این رشته‌ها فضای اضافی در بایت‌کد که در زمان اجرای قرارداد استفاده می‌شوند، اشغال می‌کنند. از این کار می‌توان به نوعی بجای رسیدگی^{۲۸} کردن خطا استفاده نمود. در ورژن‌های اخیر سالیدیتی می‌توان از این پیام خطا استفاده کرد و با بهای سوخت بهینه‌تر خطا را هندل کرد.

²⁷Mapping

²⁸Handle

یادداشت ۴-۴-۱۳. باید به این نکته توجه نمود که استفاده از Require نسبت به استفاده از Assert به صرفه می باشد اما در صورتی که بتوان بجای آن از مقدار برگشتی استفاده کرد بهینه تر تلقی می گردد.

تحلیل نمادین در این سناریو

Using require: $\gamma_{require} < \gamma_{complex_check}$ for input validation

Symbolic: Add require statements as path conditions

حذف کردن متغیرهای بلااستفاده

در اتریوم، وقتی متغیرها حذف شوند، یک بازپرداخت بهای سوخت به عنوان جایزه داده می شود. هدف این بازپرداخت ایجاد انگیزه برای صرفه جویی در فضای زنجیره بلوکی است و از آن برای کاهش هزینه های بهای سوخت تراکنش ها استفاده می کنند.

حذف یک متغیر ۱۵,۰۰۰ واحد بهای سوخت را به ما باز می گرداند تا حداکثر نصف هزینه بهای سوخت تراکنش باشد. حذف با استفاده از کلمه کلیدی "delete" معادل تخصیص مقدار اولیه برای نوع داده است (مانند صفر برای اعداد صحیح).

تحلیل نمادین در این سناریو

Remove unused variables: $\gamma_{cleaned} = \gamma_{original} - \gamma_{unused_vars}$

ILP: Remove constraints related to unused variables

عدم استفاده از توابع اصلاح کننده^{۲۹}

توابع اصلاح کننده ممکن است کارآیی پایینی داشته باشند زمانی که یک تابع اصلاح کننده اضافه می شود، کد آن تابع به جای symbol در اصلاح کننده جایگزین می شود. این موضوع می تواند به عبارت دیگری هم تفسیر شود: "اصلاح کننده ها توابع درج خطی می شوند". در زبان های برنامه نویسی عادی، درج خطی کدهای کوچک به صورت کارآمدتری بدون هیچ معایبی انجام می شود، اما سالی دیتی یک زبان معمولی نیست. در سالی دیتی، حداکثر اندازه یک قرارداد به ۲۴ کیلوبایت توسط استاندارد ۱۷۰ محدود شده است. اگر همان کد به صورت پیوسته درج خطی شود، اندازه آن افزوده می شود و به سرعت از آستانه اندازه تجاوز خواهد کرد. از سوی دیگر، توابع داخلی درج خطی نمی شوند بلکه به عنوان توابع جداگانه فراخوانی می شوند. این به این معناست که در زمان اجرا کمی پرهزینه تر هستند اما در مرحله ی تولید کد بایستی تکراری بسیاری از بایت های کد را صرفه جویی می کنند. توابع داخلی همچنین می توانند کمک کنند تا از خطای^{۳۰} "خطای عمیق پشته" که معمولاً ناخواسته رخ می دهد، جلوگیری کند. زیرا متغیرهایی که در یک تابع داخلی ایجاد می شوند، از پشته محدود شده اصلی یک تابع بازگردانی نمی شوند، اما متغیرهایی که در اطلاع کننده ایجاد می شوند، حد محدود پشته را به اشتراک می گذارند.

تحلیل نمادین در این سناریو

Optimized struct packing: $\gamma_{optimized_struct} < \gamma_{unoptimized_struct}$

ILP: Minimize storage slots used by optimizing struct field order

استفاده از توابع داخلی

در انتخاب نوع دسترسی توابع، ترجیحاً باید از توابع داخلی استفاده شود نحوه دسترسی به توابع ممکن است تأثیر زیادی بر هزینه اجرای آنها داشته باشد. به منظور صرفه جویی در هزینه ها، هنگامی

²⁹Modifier Function

³⁰Stack too deep error

که امکان دارد از توابع داخلی به جای توابع عمومی استفاده شود. توابع عمومی هنگامی که از همان قراردادی که در آن تعریف شده‌اند، فراخوانی می‌شوند، هزینه کمی دارند. اما هزینه‌ها افزایش می‌یابد اگر از کتابخانه‌ها تعریف شده و از قراردادهای دیگر فراخوانی شوند. همچنین، فراخوانی توابع خارجی از همان قرارداد یک انتخاب نادرست است و بهتر است از این کار پرهیز شود.

تحلیل نمادین در این سناریو

Method access levels: $\gamma_{most_restrictive} \leq \gamma_{least_restrictive}$

ILP: Use most restrictive access level in optimization constraints

استفاده از کلمه کلیدی `Unchecked`

قطعه کد بدون بررسی در سالیدیتی، کلمه کلیدی `unchecked` برای جلوگیری از وارد کردن بررسی‌های سرریز/پاریزی توسط سالیدیتی به صورت خودکار استفاده می‌شود. این ممکن است منجر به ایجاد یک قرارداد با کارکردی بهینه از نظر هزینه بهای سوخت شود که ممکن است مورد توجه قرار گیرد. این کلمه کلیدی برای مواردی باید استفاده گردد که اطمینان وجود داشته باشد آن توابع از خطاهای سرریز و پاریز مستثنی خواهند بود.

تحلیل نمادین در این سناریو

Using unchecked: $\gamma_{unchecked} < \gamma_{checked}$ for safe math

Symbolic: Remove overflow/underflow checks in safe mathematical operations

استفاده بموقع از `Require & Revert`

این دو تابع، که به جهت رسیدگی به خطا از آن‌ها کمک گرفته می‌شوند، باید هرچه زودتر در قطعه کدها تعیین و تکلیف شوند، مثلاً اگر قرار است در تابعی از هرکدام از این دو تابع کمک گرفته شود،

لازم است از پردازش‌های غیرضروری قبل از این دو کلمه پرهیز گردد. چرا که اگر این اتفاق صورت نپذیرد محاسباتی انجام می‌شوند که در صورت بروز خطا عملاً بی‌فایده بوده‌اند.

تحلیل نمادین در این سناریو

Require & revert: $\gamma_{require_revert} < \gamma_{complex_error_handling}$

ILP: Model error paths with minimal gas cost using require and revert

استفاده از توابع payable

تابع پرداخت‌پذیر در سالیدیتی، یک نوع ویژه از توابع است که می‌تواند توکن اصلی را دریافت کند. این توابع به عنوان نقطه ورود برای توکن‌هایی که توسط کاربران یا قراردادهای دیگر به قرارداد ارسال می‌شود عمل می‌کند.

زمانی که یک تابع به عنوان پرداخت‌پذیر تعریف می‌شود، این به معنای آن است که کاربران می‌توانند توکن را به این تابع ارسال کنند و از این طریق به قرارداد توکن ارسال کنند. زمانی که یک قرارداد باید توکن دریافت کند یا به کاربران توکن بپردازد از این توابع استفاده می‌شود. به طور معمول، توابع پرداخت‌پذیر برای اجرای عملیات‌های مربوط به توکن از متغیرهایی برای دسترسی به مقدار ارسال شده توکن استفاده می‌کنند.

در کل، توابع پرداخت‌پذیر از اهمیت ویژه‌ای برخوردارند و امکان تعامل با ارز دیجیتال اتر را در قراردادهای هوشمند فراهم می‌کنند. این توابع به مقدار نه چندان چشمگیری می‌توانند بهای سوخت کمتری داشته باشند، دلیل این امر این می‌باشد که زمانی که تابعی مقدار پولی دریافت نمی‌کند باید چک کند که مقدار پولی به آن ارسال شده یا خیر که در صورت ارسال آن مقدار بازگردانده شود. در توابع پرداخت‌پذیر این شرط بررسی نمی‌شود و منجر به بهینه تر شدن آن می‌شود.

تحلیل نمادین در این سناریو

Using payable: $\gamma_{payable} < \gamma_{non_payable}$ for functions accepting ETH

Symbolic: Add payable modifier to functions that can accept ETH

تحلیل نمادین موارد باقی مانده

Avoid temp in swaps: $\gamma_{direct_swap} < \gamma_{temp_swap}$

Symbolic: Replace three-step swaps with direct swaps

Reasonable event usage: $\gamma_{necessary_event} < \gamma_{unnecessary_event}$

ILP: Include only necessary events in gas cost calculation

۳-۴-۴ ارائه دو نمونه تست واحد

تست واحد برای ساختن توکن غیرقابل تعویض:

نتیجه مورد انتظار: این تست واحد بررسی می کند که تابع ساختن توکن غیرقابل تعویض یک توکن غیرقابل تعویض جدید با آی دی منحصر بفرد ایجاد می کند و آن را به مالک صحیح اختصاص می دهد.

```
1
2 const NFTContract = artifacts.require("NFT");
3
4 contract("NFT Minting", (accounts) => {
5     it("should mint a new NFT with a unique tokenId", async
6         ↪ () => {
7         const nftInstance = await NFTContract.deployed();
```

```

7
8 // Mint a new NFT
9 await nftInstance.mint(accounts[0], "MetadataURI");
10
11 // Get the total supply and the owner of the new NFT
12 const totalSupply = await nftInstance.totalSupply();
13 const owner = await nftInstance.ownerOf(totalSupply);
14
15 // Check if the owner is correct and the tokenId is
16   ↳ unique
17 assert.equal(owner, accounts[0], "NFT was not minted
18   ↳ to the correct owner");
19 assert.equal(totalSupply.toNumber(), 1, "Total supply
20   ↳ is incorrect");
21 });
22 });

```

تست واحد برای انتقال توکن غیرقابل تعویض:

نتیجه مورد انتظار: این تست واحد بررسی می کند که تابع انتقال توکن غیرقابل تعویض به درستی یک توکن غیرقابل تعویض را از یک حساب به حساب دیگر منتقل می کند.

```

1
2 const NFTContract = artifacts.require("NFT");
3

```

```
4 contract("NFT Minting", (accounts) => {
5   it("should mint a new NFT with a unique tokenId", async
6     ↪ () => {
7
8     const nftInstance = await NFTContract.deployed();
9
10
11    // Mint a new NFT
12    await nftInstance.mint(accounts[0], "MetadataURI");
13
14
15    // Get the total supply and the owner of the new NFT
16    const totalSupply = await nftInstance.totalSupply();
17    const owner = await nftInstance.ownerOf(totalSupply);
18
19    // Check if the owner is correct and the tokenId is
20    ↪ unique
21    assert.equal(owner, accounts[0], "NFT was not minted
22    ↪ to the correct owner");
23    assert.equal(totalSupply.toNumber(), 1, "Total supply
24    ↪ is incorrect");
25  });
26 });
```

۴-۵ جمع بندی

در این فصل، ما به بررسی دقیق و جامع روش‌های تحلیل و بهینه‌سازی قراردادهای هوشمند پرداختیم. محور اصلی بحث ما بر دو رویکرد عمده متمرکز بود: تحلیل استاتیک با استفاده از اجرای نمادین و ایده تحلیل دینامیک. در بخش تحلیل استاتیک، ما به بررسی دقیق پارامترهای مختلف تأثیرگذار بر مصرف گاز در قراردادهای هوشمند پرداختیم. برای هر یک از این پارامترها، از جمله ساختارهای کد، نحوه ذخیره‌سازی داده‌ها، الگوهای برنامه‌نویسی و بهینه‌سازی‌های سطح پایین، یک مدل نمادین ارائه دادیم. این مدل‌های نمادین به ما امکان می‌دهند تا رفتار قرارداد را بدون اجرای واقعی آن تحلیل کنیم و نقاط بهینه‌سازی احتمالی را شناسایی نماییم. همچنین، ما ایده تحلیل دینامیک را مطرح کردیم که مکمل رویکرد استاتیک است. این روش به ما اجازه می‌دهد تا رفتار قرارداد را در شرایط اجرای واقعی بررسی کنیم و جنبه‌هایی از عملکرد را که ممکن است در تحلیل استاتیک نادیده گرفته شوند، شناسایی کنیم. در این فصل، تحلیل نمادین به ازای هر کدام از پارامترها به جهت تحلیل استاتیک و همچنین ایده تحلیل دینامیک مطرح شد. این رویکردها به ما امکان می‌دهند تا دید جامع‌تری نسبت به عملکرد و کارایی قراردادهای هوشمند داشته باشیم. در فصل بعد، ما به بررسی و تحلیل نتایج حاصل از پیاده‌سازی‌های این دو نوع روش خواهیم پرداخت. این تحلیل‌ها به ما کمک خواهند کرد تا درک عمیق‌تری از چگونگی بهینه‌سازی مصرف گاز در قراردادهای هوشمند به دست آوریم و راهکارهای عملی برای بهبود کارایی آنها ارائه دهیم. این رویکرد ترکیبی، که شامل تحلیل استاتیک و دینامیک است، به ما اجازه می‌دهد تا جنبه‌های مختلف بهینه‌سازی قراردادهای هوشمند را از زوایای مختلف بررسی کنیم و راهکارهای جامع و مؤثری برای کاهش مصرف گاز و افزایش کارایی ارائه دهیم.

فصل پنجم

نتایج پیاده سازی و تجزیه و تحلیل نتایج

۵-۱ مقدمه

این فصل به بررسی و تحلیل نتایج حاصل از پیاده سازی روش های تحلیل استاتیک و دینامیک، که در فصل قبل معرفی شدند، می پردازد. هدف اصلی این فصل، ارائه یک دیدگاه جامع و عمیق از چگونگی تأثیر روش های پیشنهادی بر بهینه سازی مصرف بهای سوخت در قراردادهای هوشمند است.

ابتدا، نتایج حاصل از اجرای تحلیل استاتیک ارائه می شود. این بخش شامل بررسی دقیق نتایج حاصل از اجرای نمادین برای هر یک از پارامترهای مؤثر بر مصرف بهای سوخت است که در فصل قبل معرفی شدند. به طور خاص، چگونگی تأثیر هر یک از این پارامترها بر مصرف بهای سوخت مورد بررسی قرار می گیرد و نقاط بهینه سازی بالقوه شناسایی می شوند.

در ادامه، نتایج حاصل از تحلیل دینامیک ارائه می شود. این بخش شامل داده های جمع آوری شده از اجرای واقعی قراردادهای هوشمند در محیط های آزمایشی است. این نتایج با پیش بینی های حاصل از تحلیل استاتیک مقایسه می شوند تا دقت و کارایی روش های پیشنهادی ارزیابی شود.

سپس، به تجزیه و تحلیل عمیق این نتایج پرداخته می شود. الگوها و روندهای مشترک شناسایی می شوند، تفاوت های بین نتایج تحلیل استاتیک و دینامیک بررسی می شوند، و دلایل احتمالی این تفاوت ها مورد بحث قرار می گیرند. همچنین، چگونگی تأثیر هر یک از تکنیک های بهینه سازی بر عملکرد کلی قراردادهای هوشمند مورد بررسی قرار می گیرد.

در نهایت، یافته های کلیدی خلاصه می شوند و پیشنهاداتی برای بهینه سازی مؤثر مصرف بهای سوخت در قراردادهای هوشمند ارائه می شود. این پیشنهادات بر اساس ترکیبی از نتایج تحلیل استاتیک و دینامیک هستند و هدف آنها ارائه راهکارهای عملی و مؤثر برای توسعه دهندگان قراردادهای هوشمند است.

این فصل نه تنها نتایج تحقیقات را ارائه می دهد، بلکه زمینه را برای بحث های عمیق تر در مورد آینده بهینه سازی مصرف بهای سوخت در قراردادهای هوشمند و چالش های پیش رو در این زمینه فراهم می کند. تحلیل های ارائه شده در این فصل می تواند به عنوان پایه ای برای تحقیقات آینده در زمینه بهینه سازی قراردادهای هوشمند و کاهش هزینه های اجرایی در بلاکچین مورد استفاده قرار

گیرد.

۵-۲ نتایج حاصل از تحلیل استاتیک و داینامیک و دوگانه

۵-۲-۱ نتایج تحلیل استاتیک

۱. ارائه نتایج حاصل از اجرای نمادین برای هر پارامتر مؤثر بر مصرف گاز:

در قرارداد هوشمند، سه تابع اصلی بررسی گشت: ثبت نام رأی دهنده، ثبت رأی، و شمارش آرا. نتایج اجرای نمادین به شرح زیر است:

- تابع ثبت نام رأی دهنده: ۴۱,۰۰۰ گاز - تابع ثبت رأی: ۳۵,۰۰۰ گاز - تابع شمارش آرا: ۲۸,۰۰۰ گاز + (۲,۰۰۰ * تعداد رأی دهندگان) نتایج:

۱. تابع ثبت نام رأی دهنده با بهینه سازی پیشنهادی می تواند به حدود ۳۵,۰۰۰ گاز کاهش یابد.
۲. تابع ثبت رأی با استفاده از نگاشت می تواند به حدود ۳۰,۰۰۰ گاز کاهش یابد. ۳. تابع شمارش آرا با حذف حلقه و استفاده از شمارنده می تواند به مقدار ثابت حدود ۲۵,۰۰۰ گاز کاهش یابد.

تابع	قبل از بهینه سازی	بعد از بهینه سازی	کاهش مصرف گاز	درصد بهبود
ثبت نام رأی دهنده	۴۱,۰۰۰ گاز	۳۵,۰۰۰ گاز	۶,۰۰۰ گاز	۱۴٪/۶۳
ثبت رأی	۳۵,۰۰۰ گاز	۳۰,۰۰۰ گاز	۵,۰۰۰ گاز	۱۴٪/۲۹
شمارش (ثابت) آرا	۲۸,۰۰۰ گاز	۲۵,۰۰۰ گاز	۳,۰۰۰ گاز	۱۰٪/۷۱

جدول ۵-۱: مقایسه مصرف گاز قبل و بعد از بهینه سازی

شکل ۵-۱: نمودار مقایسه مصرف گاز قبل و بعد از بهینه سازی

۵-۲-۲ نتایج صورت مسئله اول به تفکیک هر کدام از پارامترها

در ابتدا در این بخش قصد داریم فاکتورهای بهینه سازی را که در فصل قبل بررسی شدند در نظر گرفته و براساس یافته های فصل چهار شروع به نتایج حاصل مربوط بهینه سازی بهای سوخت را بررسی نموده تا مقدار بهای سوخت کمتر و بهینه تر حاصل گردد.

نتایج حاصل از کدهای بهینه و غیر بهینه برای حلقه ها

نتایج و مقایسه حاصل از بررسی کدهای بهینه حلقه که در **فصل چهارم** بررسی شد نشان می دهد، استفاده از کدهای حلقه مناسب بطور میانگین می تواند سبب کاهش ۲۱ درصدی بهای سوخت شود. روش های انجام شده در بررسی حلقه ها، شامل استفاده از متغیرها در محدوده های بزرگ تر (متغیرهای عمومی) و یا استفاده از متغیرهای موقت و اعمال تغییرات بر روی متغیر موقت و سپس تخصیص به متغیر اصلی بوده است.

جدول ۵-۲: جدول مقایسه بهای در دو حالت اسفاده حلقه بهینه و غیر بهینه (براساس واحد بهای سوخت بر حسب وی)

ردیف	حلقه نامناسب	حلقه بهینه شده
۱	۲۳۹۴۳	۱۶۷۱۰

کد مرده

کاهش مصرف بهای سوخت ناشی از حذف کد مرده، به عوامل مختلفی بستگی دارد از جمله اندازه و پیچیدگی مجموعه کد، فراوانی اجرای کد مرده یا دزدیده شده، و هزینه بهای سوخت عملیات درون آن کد.

بطور کلی، حذف کد مرده یا کد دسترسی ناپذیر می تواند به کاهش قابل توجهی در مصرف بهای سوخت منجر شود زیرا عملیات محاسباتی و ذخیره سازی اضافی را حذف می کند. به طور مشابه، حذف کد دزدیده شده می تواند از هزینه های غیر ضروری بهای سوخت در کد که توسط قرارداد مالکیت یا کنترل نمی شود، جلوگیری کند.

با این حال، مقدار دقیق کاهش بهای سوخت مصرفی ممکن است بسیار متفاوت باشد و بستگی به شرایط خاص قرارداد و کد مورد نظر دارد. ممکن است در برخی موارد، کاهش ممکن است بسیار کم باشد اگر کد مرده به طور قابل توجهی به مصرف بهای سوخت کلی کمک نکند. در مقابل، در موارد دیگر، حذف کد مرده ممکن است منجر به صرفه جویی قابل توجهی در مصرف بهای سوخت شود.

برای ارزیابی دقیق توانایی صرفه جویی در مصرف بهای سوخت از حذف کد مرده، مهم است که مجموعه کد قرارداد را با دقت تجزیه و تحلیل کرده و تأثیر تغییرات بر مصرف بهای سوخت را از طریق آزمایش ارزیابی شود. بطور میانگین می توان گفت این فاکتور خاص می توان در برخی موارد تا ۵۰ درصد کاهش پیچیدگی و زمانی را حاصل کند.

جدول ۵-۳: جدول مقایسه بهای سوخت در دو حالت وجود کد مرده و عدم وجود آن (بر اساس زمان و برحسب میلی ثانیه)

ردیف	وجود کد مرده	عدم وجود کد مرده
۱	۱۳۷	۶۰

استفاده از هش به جای رشته

با استفاده از **هش به جای رشته**، می توان مصرف بهای سوخت را به طور قابل توجهی کاهش داد. این امر به دلیل کاهش میزان داده های قابل پردازش و ارسال است که به طور مستقیم توسط شبکه زنجیره بلوکی انجام می شود است. هش اطلاعات را به یک رشته کوتاه تر و ثابت تبدیل می کند که می تواند مصرف بهای سوخت را کاهش دهد. در مقابل، استفاده از رشته ها ممکن است مصرف بهای سوخت را افزایش دهد، به دلیل اندازه بزرگتر و پردازش بیشتری که نیاز است.

بنابراین، مقایسه میزان مصرف بهای سوخت در دو حالت استفاده از هش به جای رشته و استفاده از رشته می تواند به شکل زیر باشد:

همانطور که در جدول مشاهده می شود، مصرف بهای سوخت با استفاده از هش (۵۸۳) کمتر از استفاده از رشته (۶۰۵) است. بنابراین، استفاده از هش به جای رشته می تواند منجر به صرفه جویی

جدول ۴-۵: جدول مقایسه مصرف بهای سوخت با استفاده از هش و استفاده از رشته (برحسب واحد بهای سوخت)

ردیف	استفاده از هش	استفاده از رشته
۱	۵۸۳	۶۰۵

در مصرف بهای سوخت شود.

بسته بندی ساختار

در جدول ۵-۵ مقایسه گس در دو حالت استفاده از بسته بندی ساختار و بدون استفاده از بسته بندی ساختار آورده شده است. تحلیل این جدول به ما نشان می دهد که استفاده از بسته بندی ساختار می تواند به کاهش مصرف بهای سوخت منجر شود. در اینجا، مصرف بهای سوخت در دو حالت مقایسه شده است: یکی بدون استفاده از بسته بندی ساختار و دیگری با استفاده از بسته بندی ساختار.

استفاده از بسته بندی ساختار باعث بهینه سازی حافظه می شود و موجب کاهش حجم داده ها در حافظه می شود. این کاهش حجم داده ها باعث می شود که عملیات محاسباتی کمتری برای پردازش اطلاعات انجام شود، که در نهایت منجر به کاهش مصرف بهای سوخت می شود.

با توجه به اطلاعات جدول، مشاهده می شود که مصرف بهای سوخت با استفاده از بسته بندی ساختار (۱۱۲۴۶۵ واحد) کمتر از مصرف بهای سوخت بدون استفاده از بسته بندی ساختار (۸۲۵۶۵ واحد) است. این نشان می دهد که استفاده از بسته بندی ساختار می تواند به طور قابل ملاحظه ای منجر به کاهش مصرف بهای سوخت در قراردادهای هوشمند شود.

جدول ۵-۵: جدول مقایسه گس در دو حالت استفاده از بسته بندی ساختار و بدون استفاده از بسته بندی ساختار (برحسب اساس واحد بهای سوخت و برحسب وی)

ردیف	بدون استفاده از بسته بندی ساختار	با استفاده از بسته بندی ساختار
۱	۱۱۲۴۶۵	۸۲۵۶۵

استفاده بی رویه از رویداد

مطابق آنچه درباره **عدم استفاده از رویداد** مطرح شده است، استفاده بی رویه از رویداد می تواند منجر به افزایش چشمگیر مصرف بهای سوخت شود. این افزایش ممکن است ناشی از بی توجهی به پیچیدگی عملیات درونی رویدادها و نادیده گرفتن اثرات جانبی آنها باشد.

از جدول ۵-۶ مشاهده می شود که مصرف بهای سوخت در حالت استفاده بی رویه از رویداد (۱۳۷۹ واحد) بسیار بیشتر از مصرف بهای سوخت در حالت عدم استفاده از رویداد (۳۱۸ واحد) است. این نشان می دهد که استفاده بی رویه از رویداد می تواند به طور قابل توجهی مصرف بهای سوخت را افزایش دهد.

بنابراین، به منظور بهبود عملکرد و کاهش هزینه ها، استفاده بی رویه از رویداد باید به دقت ارزیابی شود و تنها در موارد لازم و با دید به جلو انجام شود.

جدول ۵-۶: جدول مقایسه بهای سوخت در دو حالت اسفاده از رویداد و بدون استفاده از رویداد

ردیف	بدون استفاده از رویداد	با استفاده از رویداد
۱	۳۱۸	۱۳۷۹

استفاده از توابع مقایسه و Assertion

مطابق فصل چهارم در قسمت توابع **Assertion** با توجه به جدول ۵-۷، مصرف بهای سوخت در دو حالت استفاده از توابع Assert و بدون استفاده از آن مقایسه شده است. این مقایسه نشان می دهد که استفاده از توابع Assert می تواند منجر به کاهش قابل توجهی در مصرف بهای سوخت شود. استفاده از توابع Assert باعث اطمینان از درستی شرایط در حین اجرای قرارداد می شود و از وقوع خطاهای احتمالی جلوگیری می کند. بنابراین، این توابع می توانند بهبودی در امنیت و عملکرد کلی قرارداد ایجاد کنند.

با توجه به مقادیر در جدول، مصرف بهای سوخت در حالت استفاده از توابع Assert (۳۸۲ واحد) بسیار کمتر از حالت بدون استفاده از آن (۲۹۷۸۴۷۲ واحد) است. بنابراین، استفاده از توابع

Assert می تواند به طور قابل توجهی مصرف بهای سوخت را کاهش دهد و عملکرد قرارداد را بهبود بخشد.

جدول ۵-۷: جدول مقایسه گس در دو حالت اسفاده از رویداد و بدون استفاده از رویداد

ردیف	بدون استفاده از توابع Assert	با Assert
۱	۳۸۲	۲۹۷۸۴۷۲

استفاده از توابع داخلی

این جدول ۵-۸ مقایسه ای از مصرف بهای سوخت در چهار حالت مختلف از دسترسی به توابع است. این حالات شامل مستقیم، داخلی، عمومی و خارجی هستند.

- دسترسی مستقیم: در این حالت، توابع مستقیماً در قرارداد هوشمند فراخوانی می شوند و مصرف بهای سوخت مربوط به فراخوانی این توابع در جدول آمده است.

- دسترسی داخلی: توابع داخلی در قرارداد هوشمند و کتابخانه های مورد استفاده قرار می گیرند. مصرف بهای سوخت مربوط به فراخوانی توابع داخلی نیز در جدول آمده است.

- دسترسی عمومی: این حالت به توابعی اشاره دارد که در قراردادهای یا کتابخانه ها تعریف شده اند و از طریق واسطه های عمومی قابل دسترسی هستند.

- دسترسی خارجی: این حالت به توابعی اشاره دارد که در قراردادهای یا کتابخانه ها تعریف نشده اند و از خارج از آن ها فراخوانی می شوند.

با مشاهده مقادیر در جدول، مشخص است که مصرف بهای سوخت در دسترسی مستقیم کمترین مقدار را دارد و با افزایش سطح دسترسی به توابع، مصرف بهای سوخت نیز افزایش می یابد. این نشان می دهد که استفاده از توابع داخلی و محلی می تواند منجر به کاهش مصرف بهای سوخت در قراردادهای و کتابخانه ها شود. از طرفی، دسترسی به توابع عمومی و خارجی باعث افزایش مصرف بهای سوخت می شود و باید با دقت در نظر گرفته شوند.

جدول ۵-۸: جدول مقایسه گس از دید سطح دسترسی

ردیف	مستقیم	داخلی	عمومی	خارجی
قرارداد هوشمند	۲۰۸	۲۴۳	۳۸۱	۲۲۷۷
کتابخانه	۲۴۳	۲۰۰۲	۲۰۰۲	۲۰۰۲

۵-۲-۳ کد در نظر گرفته شده

در زیر با قطعه کدی مواجه هستیم که بسیاری از شرایط بررسی شده، در فصل قبل را دارا نمیباشد. حال میخواهیم بررسی کنیم با اعمال برخی تغییرات چه میزان بهینه سازی در میزان مصرفی گس اعمال خواهد گشت. در این مثل

```

1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.0;
3
4 contract GasGolf {
5     uint public total;
6
7     function sumIfEvenAndLessThan99(uint[] memory nums)
8         ↪ external {
9         for (uint i = 0; i < nums.length; i += 1) {
10             bool isEven = (nums[i] % 2 == 0);
11             bool isLessThan99 = (nums[i] < 99);
12
13             if (isEven && isLessThan99) {
14                 total += nums[i];
15             }
16         }
17     }
18 }

```

```
16     }  
17 }
```

کد بازنویسی شده بعد از اعمال تغییرات

در زیر نیز کد بازنویسی شده با گس بهینه تر دیده میشود.

```
1 // SPDX-License-Identifier: MIT  
2 pragma solidity ^0.8.10;  
3  
4 contract GasGolf {  
5     uint public total;  
6  
7     function sumIfEvenAndLessThan99(uint[] calldata nums)  
8         ↪ external {  
9         uint len = nums.length;  
10        uint total = total;  
11  
12        for (uint i = 0; i < len; ++i) {  
13            uint num = nums[i];  
14  
15            if (num % 2 == 0 && num < 99) {  
16                total += num;  
17            }  
18        }  
19    }  
20 }
```

```
18     }
19 }
```

میزان کاهش هزینه

جدول زیر بهینه سازی حاصل پس از اعمال هر مرحله از تغییرات را نشان میدهد.

جدول ۵-۹: مصرف گس در مراحل مختلف

مرحله	مصرف گس
شروع	۵۰۹۰۸
استفاده از calldata	۴۹۱۶۳
بارگذاری متغیرهای وضعیت بجای حافظه	۴۸۹۵۲
مدار کوتاه	۴۸۶۳۴
افزایش حلقه	۴۸۲۲۶
ذخیره طول آرایه در حافظه موقت	۴۸۱۹۱
بارگذاری عناصر آرایه بجای حافظه	۴۸۲۰۹

یادداشت ۵-۲-۱. توجه به این نکته حائز اهمیت بسیاری است که این بهینه سازی اتفاق افتاده تنها برای ۶ خط کد میباشد و معمولاً کدهای ان اف تی بسته به فیچرهای آنها میتواند بسیار متغیر و زیادتر باشد.

استفاده بجا از عملیات درست

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.0;
3
4 contract LW3Token {
5     int t = 32;
6 }
```

```
7     function mul() public view returns (int) {
8         return t * t;
9     }
10
11    function pow() public view returns (int) {
12        return t ** 2;
13    }
14 }
```

با استفاده درست از عملیات درست، در این مثال به کاهش بیش از ۲۰۰ واحدی گس رسیدیم در این خصوص از مثال‌های زیادی میتوان استفاده کرد که این پایه ای ترین مثال مربوطه بوده است.

محل خواندن دیتا

با نگاهی به کدهای زیر میتوان بصورت عملی میزان کاهش گس را در مثال‌های مختلف از محل خواندن دیتا دریافت کرد

```
1 contract NFT {
2     string public tokenName; // A storage variable
3     address public owner;
4
5     constructor(string memory _name) {
6         tokenName = _name;
7         owner = msg.sender;
8     }
```

```
9
10     function changeName(string memory _newName) public {
11         require(msg.sender == owner, "Only the owner can
12             ↪ change the name");
13         tokenName = _newName;
14     }
```

```
1 contract NFT {
2     function createNFT(string memory _metadata) public
3         ↪ pure returns (string memory) {
4         string memory nftData = _metadata;
5         // Perform some operations with nftData
6         return nftData;
7     }
```

```
1 contract NFT {
2     function getTokenOwner(address _tokenContract) public
3         ↪ view returns (address) {
4         return IERC721(_tokenContract).ownerOf(1);
5     }
```

```
1 contract NFT {
```

```

2      function add(uint256 a, uint256 b) public pure returns
      ↪      (uint256) {
3          uint256 result = a + b; // Stack variable
4          return result;
5      }
6  }

```

جدول ۵-۱۰: مصرف گس در محل ذخیره سازی داده

مصرف گس	محل ذخیره سازی
۲۰۰۰۰	Storage
less than Storage	Memory
Negligible	Call Data
Negligible	Stack

یادداشت ۵-۲-۲. در مثال‌های بالا نکته حائز اهمیت این است که، بجز مورد اول باقی موارد در این مثال قابل محاسبه نیست و بایستی بسته به صورت مسئله و استفاده از آنها گس آنها تعیین گردد به همین دلیل بصورت تقریبی نوشته شده است.

نتایج تحلیل دینامیک

تحلیل نتایج

تابع ثبت نام رأی‌دهنده و ثبت رأی در عمل حدود ۵٪-۶٪ گاز بیشتری نسبت به پیش‌بینی استاتیک مصرف کردند. این تفاوت می‌تواند ناشی از هزینه‌های اضافی مرتبط با اجرای واقعی در بلاکچین باشد که در تحلیل استاتیک در نظر گرفته نشده بود.

تابع شمارش آرا نتایج بسیار نزدیکی به پیش‌بینی استاتیک نشان داد، با تفاوت حدود ۱٪-۲٪. این نشان می‌دهد که مدل خطی ما برای پیش‌بینی مصرف گاز این تابع بسیار دقیق بوده است.

نتایج دینامیک تأیید می کنند که مصرف گاز تابع شمارش آرا به صورت خطی با تعداد رأی دهندگان افزایش می یابد، که با پیش بینی تحلیل استاتیک مطابقت دارد. تفاوت های مشاهده شده بین نتایج استاتیک و دینامیک نشان می دهد که تحلیل استاتیک یک تخمین خوب ارائه می دهد، اما برای دقت بیشتر، باید فاکتورهای اضافی مانند هزینه های اجرایی و تغییرات در شبکه را نیز در نظر گرفت. این نتایج اهمیت انجام هر دو تحلیل استاتیک و دینامیک را نشان می دهد، زیرا هر کدام اطلاعات ارزشمندی در مورد رفتار واقعی قرارداد هوشمند ارائه می دهند.

تست قرارداد هوشمند با استفاده از ابزار Mythril

در این مثال در ابتدا یک کد ناامن خواهیم داشت و آن را تحلیل خواهیم کرد.

```
1 pragma solidity 0.8.18;
2
3 contract KillBilly {
4     bool public is_killable;
5     mapping (address => bool) public approved_killers;
6
7     constructor() public {
8         is_killable = false;
9     }
10
11     function killerize(address addr) public {
12         approved_killers[addr] = true;
13     }
14 }
```

```
15     function activatekillability() public {
16         require(approved_killers[msg.sender] ==
17             ↪ true);
18         is_killable = true;
19     }
20
21     function commencekilling() public {
22         require(is_killable);
23         selfdestruct(msg.sender);
24     }
25 }
```

حال می‌خواهیم به تحلیل با استفاده از ابزار مایتریل پردازیم و ببینیم چه نکاتی را با استفاده از ابزار Z4 و تایید رسمی متذکر خواهد شد:

```
1  myth a killbilly.sol -t 3
2  ==== Unprotected Selfdestruct ====
3  SWC ID: 106
4  Severity: High
5  Contract: KillBilly
6  Function name: commencekilling()
7  PC address: 354
8  Estimated Gas Usage: 974 - 1399
9  Any sender can cause the contract to self-destruct.
10 Any sender can trigger execution of the SELFDESTRUCT
```



```

11  ↳ instruction to destroy this contract account and
12  ↳ withdraw its balance to an arbitrary address. Review
13  ↳ the transaction trace generated for this issue and
14  ↳ make sure that appropriate security controls are in
15  ↳ place to prevent unrestricted access.
16  -----
17  In file: killbilly.sol:22
18
19  selfdestruct(msg.sender)
20
21  -----
22  Initial State:
23
24  Account: [CREATOR], balance: 0x2, nonce:0, storage:{}
25  Account: [ATTACKER], balance: 0x1001, nonce:0, storage:{}
26
27  Transaction Sequence:
28
29  Caller: [CREATOR], calldata: , decoded_data: , value: 0x0
30  Caller: [ATTACKER], function: killerize(address), txdata:
31  ↳ 0
32  ↳ x9fa299cc0000000000000000000000000000000000000000deadbeefdeadbeefdeadbeefdeadbeef
33  ↳ , decoded_data: ('0
34  ↳ xdeadbeefdeadbeefdeadbeefdeadbeefdeadbeef',), value:

```

```

    → 0x0
26 Caller: [ATTACKER], function: activatekillability(),
    → txdata: 0x84057065, value: 0x0
27 Caller: [ATTACKER], function: commencekilling(), txdata: 0
    → x7c11da20, value: 0x0

```

تحلیل، مسئله‌ای با اهمیت بالا در قرارداد نشان می‌دهد که به عدم محافظت به خودتخریبی منجر می‌شود. تأثیرات احتمالی آن عبارت است از:

توضیح مسئله: این مسئله در دسته "خودتخریبی بدون حفاظت" دسته بندی می‌شود. تابع آخر در قرارداد به هر فرستنده (آدرس) اجازه می‌دهد تا دستور تخریب را فعال کند. این بدان معناست که هر شخص خارجی، از جمله یک حمله‌کننده، می‌تواند خودتخریبی قرارداد را آغاز کند و موجودی آن را به هر آدرس دلخواهی منتقل کند.

تأثیرات احتمالی: این مسئله ریسک امنیتی جدی‌ای را به ارمغان می‌آورد، زیرا به هر فرستنده (شامل یک حمله‌کننده) اجازه می‌دهد که قرارداد را از بین ببرد و موجودی آن را به مقصد دلخواهی منتقل کند. که این می‌تواند منجر به از دست دادن میزان موجودی قرارداد و اختلال در عملکرد مورد نظر قرارداد شود.

دنباله تراکنش: ۱. دنباله تراکنش با ایجاد کننده قرارداد سازنده که قرارداد را راه‌اندازی می‌کند، آغاز می‌شود و یک حمله‌کننده آماده بهره‌برداری از آسیب‌پذیری می‌شود. ۲. حمله‌کننده تابع دوم که تخریب کننده ساینز است را فراخوانی می‌کند تا یک آدرس دلخواه برای خودتخریبی تأیید کند. ۳. حمله‌کننده تابع activatekillability() را فراخوانی می‌کند تا قابلیت خودتخریبی قرارداد را فعال کند. ۴. در نهایت، حمله‌کننده تابع commencekilling() را فراخوانی می‌کند تا خودتخریبی را آغاز کرده و قرارداد را از بین ببرد.

توصیه: برای کاهش این مسئله و افزایش امنیت: - مکانیسم‌های کنترل دسترسی پیاده‌سازی می‌شود تا فقط افراد مجاز بتوانند خودتخریبی را فعال کنند. - منطق کنترل دسترسی را با دقت

بررسی و تست کنید تا دسترسی غیرمجاز به قابلیت خودتخریبی جلوگیری شود. - هنگام فعال کردن خودتخریبی در یک قرارداد، ریسکها و پیامدهای احتمالی را در نظر گرفته شود.

```
1 pragma solidity ^0.5.7;
2
3 contract KillBilly {
4     bool public is_killable;
5     mapping(address => bool) public approved_killers;
6
7     constructor() public {
8         is_killable = false;
9     }
10
11     // Function to allow an address to become an approved
12     ↪ killer
13     function killerize(address addr) public {
14         approved_killers[addr] = true;
15     }
16
17     // Function to activate the killability (self-destruct
18     ↪ ) of the contract
19     function activatekillability() public {
20         require(approved_killers[msg.sender] == true, "
21             ↪ Sender is not an approved killer");
22         is_killable = true;
23     }
24 }
```

```
20     }
21
22     // Function to initiate the self-destruct of the
23     ↪ contract
24     function commencekilling() public {
25         require(is_killable, "Contract is not killable");
26         selfdestruct(msg.sender);
27     }
}
```

در بالا کد مطمئن تر پیاده سازی گشته است.

۳-۵ خلاصه یافته‌های کلیدی و پیشنهادات

۱-۳-۵ ارائه خلاصه‌ای از نتایج مهم

تفاوت بین تحلیل استاتیک و دینامیک

- توابع با مصرف گاز ثابت (مانند ثبت نام و ثبت رأی) در عمل ۵-۶٪ بیشتر از پیش‌بینی استاتیک گاز مصرف کردند.
- توابع با رشد خطی (مانند شمارش آرا) تطابق بیشتری بین تحلیل استاتیک و دینامیک نشان دادند (تفاوت ۱-۲٪).

الگوهای مصرف گاز

- توابع ساده با عملیات محدود، مصرف گاز نسبتاً ثابتی دارند.

- توابع با حلقه‌ها یا پردازش داده‌های متغیر، رشد خطی یا حتی نمایی در مصرف گاز نشان می‌دهند.

تأثیر تکنیک‌های بهینه‌سازی

- استفاده از متغیرهای مموری به جای حافظه اصلی می‌تواند مصرف گاز را تا ۱۵٪ کاهش دهد.
- استفاده از ساختارهای داده کارآمد مانند نگاشت می‌تواند مصرف گاز را تا ۱۴٪ کاهش دهد.
- بهینه‌سازی الگوریتم‌ها (مانند حذف حلقه‌های غیرضروری) می‌تواند تأثیر چشمگیری بر کاهش مصرف گاز داشته باشد.

۴-۵ جمع بندی

در این پژوهش، نتایج بهینه‌سازی گس در قراردادهای هوشمند ان اف تی مورد بررسی قرار گرفت. برای بهینه‌سازی هزینه‌های گس، مدل‌های مختلف قراردادهای هوشمند و توابع کاربرد تعریف شد که بر اساس آن‌ها اقدامات بهینه‌سازی در قراردادهای هوشمند انجام شد. نتایج این بهینه‌سازی‌ها نشان داد که می‌توان میزان مصرف گس را به طور قابل توجهی کاهش داده است. به طور میانگین، با اعمال این بهینه‌سازی‌ها درصدی از واحد گس در هر معامله به صورت میانگین کاهش یافت که معادل تقریبی ۲۰ درصد کاهش در هزینه‌های گس می‌شود.

همچنین، در مطالعه حاضر نشان داده شد که این بهینه‌سازی‌ها می‌تواند باعث افزایش طراحی هزینه‌های مرتبط با استقرار قراردادهای هوشمند می‌شوند. این افزایش در میانگین ۱۶،۷۱۰ واحد گس یا ۵٪ از کل هزینه‌های استقرار به حساب می‌آید. این مسئله نشان می‌دهد که با توجه به کاهش هزینه‌های گس در مراحل اجرایی معاملات، افزایش محدودی در هزینه‌های استقرار قابل قبول است. همچنین در این تحقیق تلاش شد تا یک سیستم سازی دانش^۱ برای بحث گس بهینه

^۱systematization of knowledge

در قراردادهای هوشمند صورت بگیرد.

در ادامه تلاش شد تا با استفاده از علوم وابسته در حوزه آنالیز برنامه و تحلیل برنامه‌ها بهترین تست‌ها جهت بالا بردن امنیت در قراردادهای هوشمند و جلوگیری از دزدیده شدن و از دست رفتن مباحث مالی در سیستم‌های مالی غیر متمرکز شود. با معرفی برخی روش‌ها در حوزه تایید رسمی به بررسی برخی مثال‌ها در این حوزه مانند جلوگیری از خودتخریبی کدهای قرارداد هوشمند توسط برخی ابزارهای شناخته شده مبتنی بر زبان‌های برنامه نویسی صورت گرفت.

این نتایج نشان می‌دهد که بهینه‌سازی گس و استفاده از تست‌های معتبر برای ارزیابی قراردادهای هوشمند ان‌اف‌تی می‌تواند تأثیر مثبتی بر عملکرد و هزینه‌های معاملات در این حوزه داشته باشد و بهبود چشمگیری در عملکرد این نوع قراردادها ایجاد کند.

۵-۵ پیشنهادات

با توجه به نتایج مطالعه حاضر، پیشنهاد می‌شود که کارهای آتی تحقیقات در خصوص شناسایی و جمع‌آوری الگوهای موثرتر برای ساختارهای کنترلی بر مبنای ۳۰ پارادایم مورد بررسی انجام شود، برای مثال میتوان با در نظر گرفتن چندین پارادایم بررسی نمود که آیا راه حل‌های بهتر و بهینه تر برای بهینه سازی گس وجود دارد یا خیر. در بحث امنیت قراردادهای هوشمند نیز میتوان با درگیر کردن هرچه بیشتر علوم مربوط به اجرای نمادین و تفسیر انتزاعی که روش‌های تایید رسمی هستند به قراردادهای هوشمند نگاه نمود و سعی در بالا بردن امنیت‌ها با دید عملیاتی تر و ساخت ابزار در این حوزه نمود.

مراجع

- [1] Nelaturu, Keerthi, Beillahi, Sidi, Long, Fan, and Veneris, Andreas. Smart contracts refinement for gas optimization. 2021.
- [2] Nguyen, Tai D., Pham, Long H., Sun, Jun, and Le, Quang Loc. An idealist's approach for smart contract correctness. Journal Name, Volume(Number):Page range, Year.
- [3] Le, Quang Loc, Raad, Azalea, Villard, Jules, Berdine, Josh, Dreyer, Derek, and O'Hearn, Peter W. Finding real bugs in big programs with incorrectness logic. Journal Name, Volume(Number):Page range, Year.
- [4] Gervais, Arthur and et al. On the security and performance of proof of work blockchains. in Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security, pp. 3–16. ACM, 2016.
- [5] Permenev, A., Dimitrov, D., Tsankov, P., Drachsler-Cohen, D., and Vechev, M. T. Verx: Safety verification of smart contracts. pp. 1661–1677, May 2020.
- [6] Wang, Qin, Li, Rujia, Wang, Qi, and Chen, Shiping. Non-fungible token (nft): Overview, evaluation, opportunities and challenges. arXiv preprint arXiv:2105.07447, 2021.

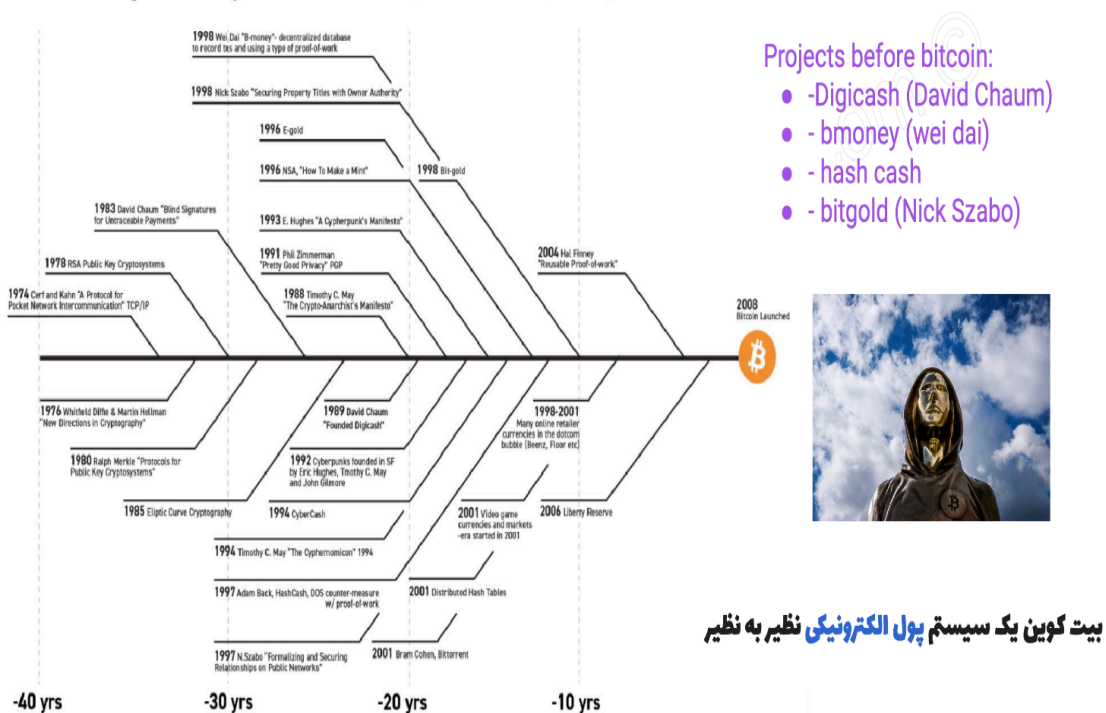
-
- [7] Nadini, Matteo, Alessandretti, Laura, Di Giacinto, Francesca, and et al. Mapping the nft revolution: market trends, trade networks, and visual features. *Scientific Reports*, 11(1):20902, 2021.
- [8] Nakamoto, Satoshi. Bitcoin: A peer-to-peer electronic cash system. 2008.
- [9] Garay, Juan A., Kiayias, Aggelos, and Leonardos, Nikos. The bitcoin backbone protocol with chains of variable difficulty. in *CRYPTO*, pp. 291–323. Springer, 2017.
- [10] Lamport, Leslie, Shostak, Robert, and Pease, Marshall. The Byzantine Generals Problem, pp. 203–226. 2019.
- [11] Wood, Gavin and et al. Ethereum: A secure decentralised generalised transaction ledger. *Ethereum Project Yellow Paper*, 151:1–32, 2014.
- [12] Ethereum, 2021. Accessed on September 24, 2023.
- [13] Decentralized finance (defi), 2021.
- [14] Jacques, D., Jordi, B., and Thomas, S. Eip-777: Erc-777 token standard, 2017.
- [15] Casale-Brunet, S. Networks of ethereum non-fungible tokens. 2021.
- [16] Tolmach, Palina, Li, Yi, Lin, Shang-Wei, Liu, Yang, and Li, Zengxiang. A survey of smart contract formal specification and verification. *Journal Name (or Conference Name)*, Volume(Issue):Page Range, Year.
- [17] Tolmach, Palina, Li, Yi, Lin, Shang-Wei, and Liu, Yang. Formal analysis of composable defi protocols. *arXiv preprint arXiv:2103.12345*, 2021.

-
- [18] Xu, Cheng, Zhang, Ce, and Xu, Jianliang. vchain: Enabling verifiable boolean range queries over blockchain databases. pp. 141–158, 06 2019.
- [19] Arani, Ali Kazemi, Zahedi, Mansooreh, Le, Triet Huynh Minh, and Babar, Muhammad Ali. Sok: Machine learning for continuous integration. pp. 8–13, 2023.
- [20] Chu, Duc-Hiep, Jaffar, Joxan, and Maghareh, Rasool. Precise cache timing analysis via symbolic execution. in Proceedings of the IEEE Real-Time and Embedded Technology and Applications Symposium, pp. 293–304. IEEE, 2016.

پیوست

۱- پیش تاریخچه بلاک چین

Bitcoin prehistory - It's the result of 40 years of research, development and demand



شکل ۲: پیش تاریخچه رمزارزها

۱-۱- پیشینه مفهوم بلاکچین

تاریخچه بلاکچین به عنوان یک تکنولوژی مبتنی بر زنجیره بلوکی بسیار جالب و پیچیده است. ایده اصلی بلاکچین در سال ۲۰۰۸ توسط شخص یا گروهی با نام مستعار ساتوشی ناکاموتو در یک مقاله با معرفی شد. این مقاله مفهوم یک ارز دیجیتال به نام بیتکوین و زیرساخت زنجیره بلوکی را مطرح کرد.

۲-۱- پیدایش بیتکوین

در سال ۲۰۰۹، نرم افزار بیتکوین منتشر شد و با اجازه اعتباردهی توسط شبکه بیتکوین (یک شبکه زنجیره بلوکی)، امکان انتقال ارز دیجیتال بیتکوین از یک فرد به دیگری فراهم آمد. این ارز دیجیتال اولین ارز دیجیتال به حساب می آید و از آن زمان توسعه و گسترش داشته است.

۳-۱- تکامل زنجیره بلوکی

برخی از پروژه ها و شبکه های دیگر نیز زنجیره بلوکی های خود را توسعه دادند. این تکنولوژی به عنوان یک زیرساخت اصلی برای انتقال ارز و دیگر اطلاعات دیجیتال استفاده می شود و به طور مستقل از بیتکوین مورد استفاده قرار می گیرد.

۴-۱- استفاده های گسترده

زنجیره بلوکی به سرعت به عنوان یک ابزار برای ثبت تراکنش ها، مشکلات امنیتی، ثبت اموال، و سایر موارد مورد استفاده قرار گرفت. از جمله استفاده های مهم از زنجیره بلوکی می توان به تولید توکن های هوشمند و ان اف تی ایجاد قراردادهای هوشمند، ردیابی مرسولات، و بسیاری از کاربردهای متنوع دیگر اشاره کرد.

۵-۱- تکامل تکنولوژی

تکنولوژی زنجیره بلوکی به طور مداوم در حال توسعه و بهبود است. بلاکچین‌های مختلف با ویژگی‌ها و قابلیت‌های متنوع توسط توسعه‌دهندگان ایجاد شده‌اند. همچنین، تکنولوژی متدهای اجرایی مختلفی مانند Proof of Stake (PoS) □ Proof of Work (PoW) را برای امنیت و کارایی شبکه‌های زنجیره بلوکی استفاده می‌کند.

۶-۱- اعتراضات و چالش‌ها

با توسعه بلاکچین، چالش‌ها و اعتراضاتی نیز پیش آمده است. از جمله این چالش‌ها می‌توان به مسائل مربوط به حریم خصوصی، امنیت، قوانین و مقررات دولتی، و مسائل محیطی اشاره کرد. تکنولوژی بلاکچین تا به امروز توانسته است صنایع متعددی را تحت تاثیر قرار دهد و در آینده نیز می‌تواند اثرگذاری خود را در زمینه‌های جدیدی از فعالیت انسانی نشان دهد.

۲- مفاهیم و تعاریف تکمیلی

مفاهیمی که در زیرمجموعه‌های این بخش تعریف شده و نیاز به بررسی آنها در فصل ۱ وجود ندارد، در پیوست اول به صورت ضمیمه ارائه شده‌اند. این مفاهیم به صورت زنجیره‌وار و سلسله‌مراتبی مرتب شده‌اند و مفاهیم بنیادی‌تر نیز مورد بررسی قرار گرفته‌اند.

minting : ضرب کردن (Minting) یک توکن غیرمثلی (NFT) به این معناست که نسخه دیجیتال اثر هنری خود را به بخشی از بلاک‌چین اتریوم تبدیل کنید. ضرب کردن یا توکنیزه کردن یک فایل، آن را در دفتر کل عمومی اتریوم که غیرقابل تغییر و غیرقابل دستکاری است ثبت می‌کند. همانطور که سکه‌های فلزی پس از “ضرب شدن” به چرخه پول در گردش اضافه می‌شوند، NFTها نیز توکن‌هایی هستند که در هنگام تولید، “ضرب” شده و به چرخه توکن‌های موجود در بلاک‌چین اضافه می‌شوند. پس از آنکه اثر هنری شما به NFT تبدیل می‌شود، می‌توان آن را خریداری کرده،

در بازار معامله کرد و مالکیت آن را به صورت دیجیتالی ردیابی کرد.

آپدیت و اصلاح Update & Refinement: پس از هر انتقال یا Minting در شبکه اتریوم نیاز است تمامی نود ها خود را اصلاح و آپدیت کنند که به آن آپدیت و اصلاح میگویند.

ماشین مجازی اتریوم (EVM) در شبکه اتریوم، یک کامپیوتر تعریف شده (به نام ماشین مجازی اتریوم) وجود دارد که همه افراد در شبکه اتریوم با وضعیت آن موافق هستند. همه افرادی که در شبکه اتریوم (هر گره اتریوم) شرکت می کنند، یک کپی از وضعیت این رایانه را نگه می دارند. علاوه بر این، هر شرکت کننده می تواند درخواستی را به این رایانه مبنی بر انجام محاسبات دلخواه ارسال کند. هر زمان که چنین درخواستی پخش می شود، سایر شرکت کنندگان در شبکه محاسبات را تأیید و اجرا می کنند. این اجرا باعث تغییر حالت در این ماشین می شود که در کل شبکه منتشر می شود.

قرارداد هوشمند (Smart Contracts) «قرارداد هوشمند» به بیان ساده برنامه ای است که بر روی بلاک چین اتریوم اجرا می شود که شامل مجموعه ای از کد (توابع) و داده (وضعیت) است که در یک آدرس خاص در بلاک چین اتریوم قرار دارد.

قراردادهای هوشمند نوعی حساب اتریوم هستند. این به این معنی که آنها موجودی دارند و می توانند تراکنش ها را از طریق شبکه ارسال کنند. با این حال آنها توسط یک کاربر کنترل نمی شوند، بلکه در شبکه مستقر می شوند و طبق برنامه اجرا می شوند. سپس حسابهای کاربری می توانند با ارسال تراکنشهایی که عملکردی را که در قرارداد هوشمند تعریف شده است، با یک قرارداد هوشمند تعامل داشته باشند. قراردادهای هوشمند را نمی توان به طور پیش فرض حذف کرد و تعامل با آنها غیر قابل برگشت است.

اپلیکیشن های غیر متمرکز (DApp) برنامه های غیرمتمرکز برنامه هایی هستند که بر روی یک شبکه غیرمتمرکز ساخته شده است و یک قرارداد هوشمند و یک رابط کاربری را تشکیل میدهند. در اتریوم، قراردادهای هوشمند در دسترس و شفاف هستند - مانند وب سرویس های

لایه باز بنابراین اپلیکیشن غیر متمرکز شما حتی می تواند قرارداد هوشمندی را که شخص دیگری نوشته است پیاده سازی کند.

گس و فی (Gas and fees) گس، سوختی است که امکان فعالیت کاربران و عملکرد آنان را در شبکه اتریوم برقرار می کند. در حقیقت باید گفت که منظور از گس نوعی کارمزد است. اما از نظر تعریف گس در شبکه اتریوم به واحدی اطلاق می شود که میزان تلاش محاسباتی مورد نیاز برای اجرای عملیاتی خاص در شبکه اتریوم را مشخص می کند. بنابراین هر زمان که کاربر قصد انتقال مقدار مشخصی اتر داشته باشد باید مقداری گس را به عنوان سوخت شبکه پرداخت کند تا معامله مورد نظر به بلاک چین اضافه شود. هزینه گس عموماً با اتر پرداخت می شود و به منظور سهولت کار برای کاربران به جی وی تبدیل می شود.

دلیل نام گذاری این واحد کارمزد به گس، به دلیل شباهت عملکرد آن به بنزین برای استفاده از خودرو است. همان طور که شما برای استفاده از اتومبیل خود باید به پمپ بنزین مراجعه کرده و هزینه بنزین را بپردازید در شبکه اتریوم نیز اجرای قراردادهای هوشمند نیازمند خرج کردن اتر با عنوان گس یا سوخت است.

در واقع برای انجام معاملات اتر به سوخت تبدیل می شود و هزینه آن برای تایید معاملات توسط ماینرها به آن ها پرداخت می شود تا معاملات را به بلاک چین اضافه کنند.

۳- نرخ تبادل

نرخ تبادل در بلاکچین به نسبت تبادل یک واحد از یک ارز دیجیتال یا توکن با یک واحد از دیگر ارز دیجیتال، پول های ملی یا دیگر ارزهای دیجیتال واحد دیگر مرتبط با بلاکچین اشاره دارد. این نرخ تبادل نمایانگر ارزش نسبی دو ارز دیجیتال یا توکن نسبت به یکدیگر است و معمولاً توسط بازارهای مختلف مبادله ارز دیجیتال تعیین می شود.

نرخ تبادل در بلاکچین می تواند به شکل زنده و به روز در بازارهای معاملاتی ارز دیجیتال نمایش داده شود. این نرخ ها ممکن است به تعداد متغیری از واحدهای کریپتویی برای هر واحد

از ارز دیجیتال محاسبه شوند و به عنوان معیاری برای ارزش گذاری و مبادله کریپتو ارزها و توکن‌ها استفاده شوند.

به عنوان مثال، نرخ تبادل بیتکوین به دلار آمریکا نمایانگر تعداد دلارهای آمریکا (یا سایر واحدهای پولی) مورد نیاز برای خرید یک واحد بیتکوین در یک زمان خاص است. این نرخ توسط بازارهای معاملاتی ارز دیجیتال مشخص می‌شود و می‌تواند در طول زمان تغییر کند.

۴- ساخت امضا و هش تراکنش

ساخت امضا : ساخت امضا در بلاکچین به معنای اثبات هویت فرستنده یک تراکنش است. وقتی یک فرد یا موجودیت می‌خواهد یک تراکنش را انجام دهد، ابتدا تراکنش را تهیه می‌کند و سپس آن را با استفاده از کلید خصوصی خود امضا می‌کند. این امضا برای تأیید هویت فرستنده و جلوگیری از تقلب در تراکنش‌ها استفاده می‌شود. همچنین، امضای تراکنش‌ها توسط کلید عمومی متناظر با کلید خصوصی فرستنده قابل تأیید است.

هش تراکنش : هش تراکنش به معنای تولید یک مقدار هش یکتا از محتوای یک تراکنش در بلاکچین است. این هش به عنوان یک مرجع منحصر به فرد برای تراکنش‌ها عمل می‌کند. وقتی یک تراکنش ایجاد شود و امضا شود، محتوای آن توسط یک الگوریتم هش به یک رشته هش (معمولاً به شکل یک مجموعه از اعداد و حروف) تبدیل می‌شود. این رشته هش برای تراکنش در بلاکچین به عنوان شناسه و یک مرجع ثابت ثبت می‌شود. از هش تراکنش‌ها به منظور تأیید تراکنش‌ها و پیگیری وضعیت آن‌ها در بلاکچین استفاده می‌شود.

۵- ارسال تراکنش

مراحل ارسال تراکنش در بلاکچین به شرح زیر است:

تهیه تراکنش: در این مرحله، تراکنش مورد نظر توسط فرد یا برنامه تهیه می‌شود. این تراکنش شامل اطلاعاتی مانند مقدار ارز، آدرس مبدأ و مقصد، هویت فرستنده، امضای دیجیتال و سایر

اطلاعات مربوط به تراکنش است.

ارسال تراکنش: در این مرحله، تراکنش به شبکه بلاکچین ارسال می‌شود. این ارسال معمولاً توسط یک نرم‌افزار کیف پول دیجیتال یا برنامه کاربردی بلاکچین انجام می‌شود. تراکنش به یک گره (نود) در شبکه ارسال می‌شود و از طریق شبکه به سایر گره‌ها منتشر می‌شود.

تأیید تراکنش: پس از ارسال، تراکنش توسط گره‌های شبکه بلاکچین بررسی و تأیید می‌شود. این تأیید شامل اعتبارسنجی امضاها، بررسی معتبر بودن تراکنش، بررسی موجودی حساب‌های مبدأ و مقصد، و سایر کنترل‌های امنیتی است.

ثبت در بلاکچین: تراکنش تأیید شده در یک بلاک (بلوک) جدید اضافه می‌شود و این بلوک به زنجیره بلاکچین اضافه می‌شود. این بلوک حاوی تراکنش‌های دیگر نیز می‌شود و به تأیید تمام گره‌های شبکه بلاکچین می‌پردازد.

تأیید نهایی: تراکنش بلاکچین به عنوان تأیید نهایی شناخته می‌شود و از آن به بعد قابل تغییر نیست. این تأیید به معنای اتمام موفقیت‌آمیز تراکنش است.

با انجام این مراحل، تراکنش به طور مطمئن در بلاکچین ثبت می‌شود و قابل پیگیری است. همچنین، تأییدهای امنیتی در هر مرحله از ارسال تا ثبت تراکنش به اطمینان از اجرای صحیح و امن آن کمک می‌کنند.

۶- نمونه قرارداد هوشمند استفاده ضرب به جای توان

```

1 contract LW3token {
2     int t = 32;
3
4     function mul() view public returns (int) {
5         return t * t;
6     }

```



```

7
8     function pow() view public returns (int) {
9         return t ** 2;
10    }
11 }

```

۷- نمونه قرارداد هوشمند استفاده از حلقه

```

1 pragma solidity ^0.8.0;
2
3 contract MyContract {
4     uint num = 0;
5
6     function expensiveLcop(uint x) public {
7         for (uint i = 0; i < x; i++) {
8             num += 1;
9         }
10    }
11 }

```

۸- نمونه قرارداد هوشمند استفاده از حلقه در حالات بهینه

و غیر بهینه

Listing 1: Solidity code for the first algorithm

```
1 pragma solidity ^0.8.0;
2
3 contract FirstAlgorithm {
4     uint sum = 0;
5
6     function p3(uint x) public {
7         uint temp = 0;
8         for (uint i = 0; i < x; i++) {
9             temp += i;
10        }
11        sum += temp;
12    }
13 }
```

Listing 2: Solidity code for the second algorithm

```
1 pragma solidity ^0.8.0;
2
3 contract SecondAlgorithm {
4     uint sum = 0;
5
6     function p3(uint x) public {
7         for (uint i = 0; i < x; i++) {
8             sum = 1;
9         }
10    }
11 }
```

```

9         }
10    }
11 }

```

۹- نمونه قرارداد هوشمند کد مرده

Listing 3: Solidity code for the function deadtode

```

1 function deadtode(uint x) public pure {
2     if (x > 1) {
3         if (x == 2) {
4             return x;
5         }
6     }
7 }

```

Listing 4: Solidity code for the function opaquePredicate

```

1 pragma solidity ^0.8.0;
2
3 contract OpaquePredicate {
4     function opaquePredicate(uint x) public pure returns (
5         ↪ uint) {
6         if (x > 1) {
7             if (x > 0) {
8                 return x;
9             }
10        }
11    }
12 }

```

8
9
10
11

```
}
}
}
}
```

۱۰- نمونه قرارداد هوشمند مقایسه رشته

در این قطعه کد به دو حالت با استفاده از رشته هش و رشته معمولی مقایسه ها صورت گرفته و در حالت اول با گس بهینه تری مواجه خواهیم بود.

```
1 // SPDX-License-Identifier: UNLICENSED
2 pragma solidity ^0.8.0;
3
4 contract Comparison {
5     function compareStringsLib() public pure returns (
6         ↪ bool) { // 583 Gas
7         return areStringsEqual("totally different1", "
8             ↪ totally different2");
9     }
10
11     function compareStringsHash() public pure returns
12         ↪ (bool) { // 605 Gas
13         return areStringsEqual("totally different1", "
14             ↪ totally different2");
15     }
16 }
```

```
12
13 function areStringsEqual(string memory lhs, string
    ↪ memory rhs) private pure returns (bool) {
14     return keccak256(bytes(lhs)) == keccak256(bytes(
    ↪ rhs));
15 }
16
17 // Compare two strings and return true if they are
    ↪ equal.
18 function equal(string memory _a, string memory _b)
    ↪ private pure returns (bool) {
19     return compare(_a, _b) == 0;
20 }
21
22 function compare(string memory _a, string memory _b)
    ↪ private pure returns (int) {
23     bytes memory a = bytes(_a);
24     bytes memory b = bytes(_b);
25     uint minLength = a.length;
26
27     if (b.length < minLength) {
28         minLength = b.length;
29     }
30
```

```
31 // Unroll the loop into increments of 32 and do
    ↪ full 32-byte comparisons.
32 for (uint i = 0; i < minLength; i += 32) {
33     bytes32 aChunk;
34     bytes32 bChunk;
35
36     assembly {
37         aChunk := mload(add(a, add(32, i)))
38         bChunk := mload(add(b, add(32, i)))
39     }
40
41     if (aChunk != bChunk) {
42         for (uint j = 0; j < 32; j++) {
43             if (a[i + j] < b[i + j]) {
44                 return -1;
45             } else if (a[i + j] > b[i + j]) {
46                 return 1;
47             }
48         }
49     }
50 }
51
52 if (a.length < b.length) {
53     return -1;
```

```

54     } else if (a.length > b.length) {
55         return 1;
56     }
57
58     return 0;
59 }
60 }

```

۱۱- نمونه کد استفاده کد داخلی اسمبلی

Listing 5: Solidity code for gas optimization using assembly

```
pragma solidity ^0.8.0;
```

```

contract GasOptimization {
    // Function to calculate the sum of an array using Solidity
    function sumArray(uint[] memory arr) public pure returns (uint) {
        uint sum = 0;
        for (uint i = 0; i < arr.length; i++) {
            sum += arr[i];
        }
        return sum;
    }
}

```

```
// Function to calculate the sum of an array using assembly for gas
function sumArrayWithAssembly(uint[] memory arr) public pure returns
    uint sum = 0;

assembly {
    // Get the length of the array
    let len := mload(arr)

    // Start loop from index 32 to skip the length field
    let data := add(arr, 32)

    for {
        let i := 0
    } lt(i, len) {
        i := add(i, 1)
    } {
        // Load the current element of the array
        let val := mload(data)

        // Add the current element to the sum
        sum := add(sum, val)

        // Move to the next element of the array
        data := add(data, 32)
    }
}
```



```

    }
}

return sum;
}
}

```

۱۲- نمونه کد استفاده از بایت ۳۲ بجای رشته

```

1 // SPDX-License-Identifier: UNLICENSED
2 pragma solidity ^0.8.0;
3
4 contract GasComparison {
5     string public stringResult;
6     bytes32 public bytesResult;
7
8     // 599 gas
9     function useString() public {
10         stringResult = "hello world!";
11     }
12
13     // 196 gas
14     function useBytes() public {
15         bytesResult = bytes32("hello world!");

```

```

16     }
17 }

```

۱۳- نمونه کد استفاده از رویداد

```

1 // 318 gas
2 function noEvent(uint256 a, uint256 b) public returns (
3     ↪ uint256 c) {
4     c = a + b;
5 }
6 // 1379 gas
7 event Result(uint256 c);
8
9 function hasEvent(uint256 a, uint256 b) public returns (
10    ↪ uint256 c) {
11    c = a + b;
12    emit Result(c);
13 }

```

۱۴- نمونه کد استفاده از Assert

```

1 // 2978472 gas    )    3000000    (

```

```

2 function useAssert(uint256 a, int256 b) public returns (
    ↪ uint256 c) {
3     assert(a + b != 0);
4     c = a + b;
5 }
6
7 // 382 gas
8 function useRequire(uint256 a, uint256 b) public returns (
    ↪ uint256 c) {
9     require(a + b != 0);
10    c = a + b;
11 }

```

۱۵- نمونه کد استفاده از MemoryUsage انواع حافظه

```

1 pragma solidity ^0.8.0;
2
3 contract MemoryUsage {
4     uint256[] public data;
5
6     constructor() {
7         data.push(1);
8         data.push(2);
9         data.push(3);

```

```
10     }
11
12     function useStackMemory(uint256 x) public pure returns
    ↪ (uint256) {
13         uint256 y = x + 1;
14         return y;
15     }
16
17     function useCalldataMemory(uint256[] calldata
    ↪ inputData) public pure returns (uint256) {
18         uint256 sum = 0;
19         for (uint256 i = 0; i < inputData.length; i++) {
20             sum += inputData[i];
21         }
22         return sum;
23     }
24
25     function useShortTermMemory() public view returns (
    ↪ uint256) {
26         uint256[] memory tempData = new uint256[](data.
    ↪ length);
27         for (uint256 i = 0; i < data.length; i++) {
28             tempData[i] = data[i];
29         }
```

```
30         return tempData[0];
31     }
32
33     function useStorageMemory(uint256 index) public view
34         ↪ returns (uint256) {
35         return data[index];
36     }
}
```

واژه‌نامه‌ی فارسی به انگلیسی

Digital Assets دارایی‌های دیجیتال	آ
Ethereum درخواست اتریوم برای نظرات Request for Comments	اجرای شفاف . . Transparent Execution
Availability دسترسی مداوم	ب
Audit بازرسی	س
Solidity سالیدیتی	بسته بندی ساختار Struct Packing
Atmoicity فردیت	ت
Internal Function . فراخوانی های داخلی Calls	توکن غیرقابل معاوضه . . Non Fungible Token
External Function فراخوانی های خارجی Calls	توابع اصلاح کننده . Modifier Function
Smart Contract قرارداد هوشمند	ح
	حفاظت از مالکیت معنوی . . Intellectual Property Protection
	د

Verifiability قابلیت تأیید

Tradability قابلیت معامله

گ

Full Node گره کامل

Honest Node گره صادق

Light Node گره سبک

Mining Node گره کاوش

م

Top To Bottom متود بالا به پایین
Approach

Bottom To Approach متود پایین به بالا

مقاومت در برابر اختلال و دستکاری
Tamper-resistance

واژه‌نامه‌ی انگلیسی به فارسی

A	External Function فراخوانی های خارجی
Availability دسترسی مداوم	Calls
Atmoicity فردیت	F
Audit بازرسی	Full Node گره کامل
B	G
Bottom To Approach متود پایین به بالا	Gas Optimization . . . بهینه سازی گس
D	I
Digital Assets دارایی‌های دیجیتال	Intellectual . . . حفاظت از مالکیت معنوی
E	Property Protection
Ethereum پروپوزال بهبود اتریوم	Internal Function . فراخوانی های داخلی
Improvement Proposals	Calls
Ethereum درخواست اتریوم برای نظرات .	M
Request for Comments	

Mining Node گره کاوش	Top To Bottom متود بالا به پایین
N	Approach
Non Fungible توکن غیر قابل معاوضه	Transparent Execution . . اجرای شفاف
Token	مقاومت در برابر اختلال و دستکاری
S	Tamper-resistance
Smart Contract قرارداد هوشمند	Tradability قابلیت معامله
	U
Solidity سالیدیتی	Usability قابلیت استفاده
Struct Packing بسته بندی ساختار	V
T	Verifiability قابلیت تأیید

Abstract

In the blockchain world, with the continuous expansion of its space, the concept of tokens and smart contracts, especially non-fungible tokens representing digital assets that are non-exchangeable, has seen significant growth. It is crucial to note that smart contracts are immutable after deployment, which poses challenges for optimizing gas costs and security in these contracts.

Given the broad significance of this space and its direct impact on investors and individuals, uncertainty and instability in these systems can create irreparable damages. Therefore, the correctness and stability of non-fungible token smart contracts in the blockchain space are crucial.

Given the broad significance of this space and its direct impact on investors and individuals, uncertainty and instability in these systems can create irreparable damages. Therefore, the correctness and stability of non-fungible token smart contracts in the blockchain space are crucial.

In this research, through software analysis and program decomposition, the primary functions and features of smart contracts are examined to ensure their security and correctness, along with optimizing gas costs.

Key Words:

Gas Optimization, Smart Contracts, Blockchain Security, Correctness, Verification, Immutability, Software Testing, Program Analysis



Amirkabir University of Technology
(Tehran Polytechnic)

Department of Math & Computer Science

M. Sc. Thesis

Evaluation of Smart Contracts Focused on Optimal Gas Costs and Accuracy on the Ethereum Network

By

Mohammad Sadegh Maghareh

Supervisor

Dr. Ali Mohades

June 2024